

The Claude Operator's Manual

Everything Anthropic's docs never explained in plain English. The setup, the tools, and the workflows that turn Claude from a chatbot into your operating system.

Elvin Peters

First Edition · 2026

The Claude Operator's Manual

Everything Anthropic's docs never explained in plain English.

Elvin Peters

First Edition · 2026 · Written for Claude as it stands today

How to Use This Manual

This book is built to be read in order the first time and raided for the rest of its life. The early chapters assume nothing. The later ones assume you have read the early ones. If you skip ahead to hooks or subagents and feel lost, the fix is almost always a chapter you jumped over.

The manual comes in four parts. **Part I** is hands-on with the Claude apps and needs no terminal. **Part II** explains how Claude actually works under the hood, the mental model everything else rests on. **Part III** is the operator's power tools in Claude Code, the command-line half where the real leverage lives. **Part IV** wires it all into repeatable workflows. The appendices at the back are pure reference: cheat sheets you will keep open in a second window.

Throughout, recurring boxes flag the things worth stopping for. A **Tip** is a shortcut worth adopting. A **Gotcha** is the thing that quietly bites people. A **Key Term** defines a word you will see again. **Do This** blocks are copy-exact steps, **Worked Examples** are full walk-throughs, and every chapter ends with a **Checklist** so you can confirm the capability actually landed. Code and configuration appear in labeled panels you can copy verbatim.

One promise and one caveat. The promise: read this cover to cover and you will operate Claude at a level most users never reach. The caveat: this product moves fast. Where a detail is the kind that changes between releases, this manual teaches the durable concept and points you to the official docs for the current specifics. It is dated for a reason. Treat the mental models as permanent and the exact flags as perishable.

THE SHIFT

The Operator's Mindset

Most people chat with Claude. They open a box, type a question, read the answer, and close the box. That works, and it is also using a workshop as if it were a vending machine.

THE OPERATOR'S MINDSET

An operator does something different. An operator sets Claude up: gives it memory so it stops forgetting, gives it rules so it stays on the rails, gives it tools so it can act, and gives it workflows so the same good result happens every time without babysitting. The difference between the two is not talent or a secret prompt. It is setup. Everything in this book is setup you can copy.

Hold one idea as you read. Claude is capable far past what a single chat box reveals, and the ceiling is set by how well you configure it, not by how cleverly you phrase one message. Learn the configuration, and the same model that gave you a decent paragraph will run a project. That is the whole game. Turn the page and let us set it up.

Contents

PART I · CLAUDE, HANDS-ON

1	The Claude Landscape	7
2	Talking to Claude Well	14
3	Projects, Knowledge & Artifacts	23
4	Skills & the Skill Creator	32
5	Connecting Claude to Your World	39

PART II · HOW CLAUDE WORKS

6	Inside the Machine	48
7	The API	54

PART III · CLAUDE CODE

8	Meet Claude Code	62
9	Memory & Files	71
10	Settings & Permissions	81
11	Plan Mode	93
12	Slash Commands & Master Commands	102
13	Skills in Depth	112
14	Hooks & Hard Rules	125
15	Subagents	137
16	MCP in Depth	147
17	Worktrees, Checkpoints & Sessions	158

PART IV · PUTTING IT TOGETHER

18	Operator Workflows	173
19	The Operator's Practice	184

APPENDICES

A	The Day-One Setup Checklist	192
B	Slash-Command Reference	194
C	Hook-Event Reference	197
D	settings.json Key Reference	201
E	CLI Flag Reference	206
F	Glossary	211
G	Sources & Further Reading	214

PART I

Claude, Hands-On

Start here, and start in the app. This half needs no terminal and no setup beyond an account. You will learn the surfaces Claude gives you, how to talk to it so it actually delivers, and how to bring your own files, projects, and tools into the conversation. By the end you will reach for the right surface on instinct.

Chapter One

The Claude Landscape

A plain map of every place Claude lives, so you always know which one to open for the job in front of you.

Claude is one account with several front doors. You sign in once, and the same conversations, files, memory, and preferences follow you to every device you own. Open Claude in a browser at your desk, on the desktop app, or on your phone in a coffee line, and you pick up exactly where you left off. There is no separate login to remember and no content to copy from one place to another.

That single account is the good news. The mild confusion is that Claude offers more than one way to work once you are inside. There is ordinary chat. There are Projects. There are Artifacts. There is Cowork. Each is a genuine workspace with its own strengths, and most people only ever discover the first one by accident and stop there.

This chapter is the map. By the end you will know what each surface is in plain language, you will have a one-glance table that tells you which surface to reach for by the kind of job you have, and you will understand the plans well enough to pick the right one or advise a friend on theirs. This is the orientation the rest of Part I builds on.

1.1 One account, several front doors

Start with the physical apps, because those are the doors you actually click to get in. There are three, and they wrap the same Claude account in different

bodies.

The web app lives at `claude.ai` in any browser. This is home base. Every new feature Anthropic ships lands here first, and if you only ever use one door, this is the one. Nothing to install, works on any machine you can log into.

The desktop app for macOS, Windows, and Linux is a native window around that same account, with extra powers a browser tab cannot have. It can read and write files on your computer, drive a browser of its own, and answer a global keyboard shortcut so you can summon Claude from inside any other app without switching windows. When a task touches files on your own machine, the desktop app is the door you want.

The mobile apps for iOS and Android put chat, Projects, and Cowork in your pocket. They sync with everything else on the account, so a task you start on your phone is waiting for you on the desktop later. The one limit worth knowing early: anything that needs to touch local files or control a computer leans on a desktop app being open and reachable somewhere, because a phone has no filesystem for Claude to work in on its own.

I TIP

Install the desktop app even if you mostly live in the browser. The global "Quick Entry" shortcut alone earns its place: you can call Claude from any application, ask a question, and dismiss it without ever leaving what you were doing. Local file access and Desktop Extensions are powers the browser tab simply does not have.

Two more doors are worth naming so you recognize them later. **Claude in Chrome** is a browser extension that lets Claude act inside your own logged-in Chrome tabs, clicking and filling forms on sites where you are already signed in. And the **Developer Platform** at `platform.claude.com` is the door for people building software that calls Claude. If your goal is to write code that talks to the model, that is where you go. You do not need either one to get full value from this half of the book, and both get proper treatment later.

1.2 The workspaces inside the app

Once you are through any of those doors, Claude gives you four ways to work. They are not competing products. They are gears you shift between depending on the job.

Chat is the plain conversation you already know. You type, Claude answers, and the thread remembers itself as you go. It is the fastest path for a question, a rewrite, a quick bit of analysis, or a one-off task with no setup. Most sessions start and end here, and that is perfectly correct.

Projects are persistent workspaces that bundle three things: a set of custom instructions that acts like a standing brief, a library of files you upload once as reference knowledge, and every chat you hold inside that space. Every new conversation in a Project inherits the instructions and the knowledge automatically, so you stop re-pasting the same context every morning. A Project is the right home for a body of knowledge with a consistent voice: a client's brand and all their assets, a research corpus, a course you are building.

TIP

Think of a Project as the low-effort alternative to a full coding setup for anything that is really "a pile of reference material plus a consistent set of rules." Keep the Project instructions general (role, tone, standing constraints) and put the task-specific asks in the individual chat. That split keeps the workspace reusable across many different jobs.

Artifacts are a side panel where Claude builds a standalone deliverable that lives on its own, separate from the chat scrolling past it. A document, a chart, an SVG image, a working single-page web app, a small tool. You describe what you want, Claude produces a first version in a few turns, and then plain-language follow-ups like "make the buttons bigger" edit it in place. When something breaks, a "Fix with Claude" button sorts it out. You can publish an artifact to a shareable link, and other people can view it, even without a Claude account of their own.

Cowork is the newest gear, and the one most worth understanding, because it changes what "asking Claude to do something" means.

KEY TERM

Cowork Claude's autonomous, long-running task mode. You describe a multi-step job, then let it run in the background and come back to a result, instead of steering every step yourself in a live chat. Cowork sessions persist and can be started, checked, or resumed from web, desktop, or mobile on the same account.

The practical shape of Cowork is this. You find the message box, click "Cowork" in the bottom-left corner, and describe a task that would normally take you many back-and-forth turns to babysit. Claude carries it out on its own and you check in when it is done. You can kick a task off on your phone and review it later on your laptop, because the session belongs to the account, not the device. Desktop gives Cowork the fullest set of local powers; web and mobile borrow those powers when a desktop app is open and reachable.

1.3 Which surface for which job

Here is the payoff of the whole chapter on one page. When you have a task and are unsure where to take it, read down the left column until you find the job that matches, then open what the middle column says. The right column tells you why that is the better door.

QUICK REFERENCE · IF YOU WANT TO X, USE Y

If you want to	Use	Why
Ask a quick one-off question, no files needed	Chat (any device)	Zero setup, fastest path to an answer
Work repeatedly against the same knowledge and rules	Projects	Instructions and files load into every chat automatically
Build a document, chart, app, or other deliverable	Artifacts	A standalone thing you can edit, publish, and share

Hand off a multi-step job and not babysit it	Cowork	Runs in the background, resumable across your devices
Have Claude touch files on your own computer	Desktop app	Only surface with real local file access
Click around a website you are already logged into	Claude in Chrome	Acts inside your live browser session
Work from your phone with no laptop nearby	Mobile app	Full chat, Projects, and Cowork in your pocket
Build software that calls Claude programmatically	Developer Platform	Usage-based access for products and integrations

A useful habit forms once this sinks in. Before you type, ask yourself one question: is this a quick answer, a reusable body of work, a thing to build, or a job to hand off? Those four answers point straight at chat, Projects, Artifacts, and Cowork. That single reflex will save you more time than any prompt trick in this book.

1.4 Plans and tiers on one page

Claude comes in a free tier and several paid tiers. The exact prices and usage numbers move often, so treat the shape below as durable and check claude.com/pricing for the current figures before you commit money.

The tiers stack in a sensible order. **Free** is for evaluation. It gives you chat and a handful of Projects at limited usage, which is enough to decide whether Claude belongs in your day. **Pro** is the default recommendation for a serious individual, because it unlocks the surfaces that make Claude an operator's tool, including Cowork and the command-line agent this book's second half is about. **Max** sits above Pro for people who hit Pro's session limits regularly and want more headroom at higher throughput. **Team** is the moment two or more people need shared billing, an admin console, and workplace connectors. **Enterprise** is what you reach for when compliance and security requirements, things like audit logs and controlled data retention, are the actual driver, not headcount alone.

QUICK REFERENCE · WHICH PLAN, AND WHY

Plan	Who it is for	What it adds
Free	Evaluating Claude for the first time	Chat and a few Projects at limited usage
Pro	A serious individual user	Cowork, the command-line agent, all models, more usage
Max	Heavy daily users hitting Pro limits	The Pro feature set at higher throughput
Team	Two or more people in one workspace	Shared billing, admin console, workplace connectors
Enterprise	Organizations with compliance needs	Audit logs, controlled retention, advanced security

 GOTCHA

The free tier is **chat only**. Cowork and the command-line agent both require a paid plan, so if you evaluate Claude on Free and conclude it "cannot" run background tasks, you have only seen the front room of the house. Move to Pro before you judge what Claude can do as an operator's tool.

1.5 A power tool waiting in Part III

One surface deliberately sits outside this beginner-friendly half of the book. **Claude Code** is Anthropic's command-line agent: a version of Claude that reads a real project on your machine, plans, edits files, runs commands, and handles the kind of multi-step work that would be miserable to steer one message at a time. It comes included with the same Pro and Max plans above, so if you are on a paid tier you already have access to it.

You do not need it yet, and you will not touch a terminal anywhere in Part I. Mention it here only so you know the ceiling is higher than it looks. When you are comfortable with the surfaces in this chapter and ready for the most capable way to run Claude, Part III of this book teaches Claude Code from the ground up. For now, the four workspaces and three apps you just mapped are everything you need.

CHAPTER 1 · CHECKLIST

You can now

- ✓ Name the three Claude apps (web, desktop, mobile) and say what each one adds.
- ✓ Explain chat, Projects, Artifacts, and Cowork in one sentence apiece.
- ✓ Define Cowork as a background, resumable task mode and know how to start it.
- ✓ Pick the right surface for a job using the "if you want to X, use Y" reflex.
- ✓ Choose a plan by matching the tier to the person, from Free evaluation up to Enterprise.
- ✓ Recognize that the free tier is chat only and that paid plans unlock the operator surfaces.
- ✓ Know that Claude Code exists as the power tool covered later in Part III.

Chapter Two

Talking to Claude Well

Direct Claude clearly and you will pull dramatically better work out of the same model, no tricks required.

The gap between a mediocre answer and an excellent one is almost never the model. It is the instruction. Two people ask Claude the same kind of question on the same day, and one walks away with a polished, usable result while the other gets something vague and generic. The difference is that one of them told Claude exactly what they wanted and the other left it to guess.

Good news lives inside that fact. You do not need a secret phrase or a jailbreak or a thousand-word template to get expert output. You need to say what you want plainly, give the goal, and set the constraints. This chapter teaches that skill from the ground up, using Anthropic's own guidance, translated into moves you can make from any chat box.

By the end you will know how to state a task so it lands the first time, how to show Claude the shape of the answer you want, how to structure a longer prompt so the model reads it cleanly, when to let Claude think harder before it replies, and how to lock in a tone and format you reuse every day.

2.1 Clear direction beats clever tricks

Anthropic gives one mental model that does most of the work: treat Claude like a brilliant new employee on their first day. They are sharp and capable. They also have no idea what your norms are, what "done" looks like to you, or which details you care about. The more precisely you brief them, the better

the result. Leave things implied and you get a literal reading of a half-specified request.

Three habits carry most of the load. Be specific about the output you want, including its format and length. State the goal, so Claude understands what success means. Give the constraints up front, so it does not have to guess and guess wrong. When the order of steps matters, spell them out as a numbered list so Claude never has to reconstruct your sequence.

TIP

Anthropic's golden-rule test for any prompt: show it to a colleague who has minimal context on the task and ask them to follow it. If they would be confused about what to do, Claude will be too. This one check catches most weak prompts before you ever send them.

There is a second half to specificity that people miss. Tell Claude *why* a rule matters, and it will apply the rule intelligently instead of following it like a robot. A bare "never use ellipses" is a narrow order. Explain that the text will be read aloud by a text-to-speech engine that cannot pronounce them, and Claude generalizes the intent to every similar case you did not think to list. Motivation travels further than a command.

WORKED EXAMPLE · TURNING A WEAK PROMPT INTO A STRONG ONE

Say you want a dashboard mocked up as an artifact. The instinct is to type the shortest thing that names the task.

Weak prompt

```
Create an analytics dashboard
```

Claude will build something, and it will be thin, because you asked for the bare concept and nothing more. Now add the goal and the level of ambition you actually have in mind.

Strong prompt

Create an analytics dashboard. Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation.

Same model, same surface, one added sentence. The second version routinely produces a richer, more complete result, because Claude no longer has to guess how far you wanted it to go. You told it. That is the entire technique.

! GOTCHA

Current Claude models follow you very literally, and that bites when you phrase an action as a question. "Can you suggest some changes?" gets you a list of suggestions, not the edits, even when you clearly meant for Claude to just do it. When you want action, use plain imperative phrasing: "Change this function to improve its performance" or "Rewrite this paragraph to be shorter." Ask for a suggestion and you get a suggestion.

One more phrasing habit pays off constantly. Tell Claude what to do instead of what to avoid. "Do not use markdown" is weaker than "Write your response as smoothly flowing prose paragraphs," because the positive version gives the model a target to hit instead of a fence to avoid. Whenever you catch yourself writing a "do not," try to flip it into the thing you do want.

QUICK REFERENCE · WEAK PHRASING AND ITS STRONGER FORM

Instead of	Write
Summarize this	Summarize this in three bullet points a busy executive could skim
Do not use markdown	Write your response as flowing prose paragraphs
Can you suggest edits?	Edit the draft directly for clarity and length
Make it better	Cut it to 200 words and make the opening line concrete

Never use
jargon

Explain it so a smart 15-year-old would follow, since a beginner will
read it

2.2 Show, do not only tell

Some things are hard to describe in words and easy to demonstrate. The exact tone of a reply email, the precise structure of a product blurb, the way you like edge cases handled: for all of these, an example is worth a paragraph of instruction. Anthropic calls giving worked examples one of the most reliable ways to steer Claude's output, and it costs you almost nothing.

KEY TERM

Few-shot prompting giving Claude a few worked examples of the task, each showing an input and the ideal output, so it learns the pattern by imitation instead of from description alone. "Few" is literal: three to five good examples is the sweet spot.

Good examples share three traits. They are relevant, meaning they mirror your real use case closely. They are diverse, meaning they vary enough to cover the edge cases and stop Claude from latching onto some accidental pattern the examples happen to share. And they are clearly marked, so Claude can tell an example apart from an instruction. In a chat, the simplest way to mark them is a plain label and a blank line, like "Example 1:" above each one.

I TIP

You do not have to write the examples yourself. Paste one or two, then ask Claude to generate three more in the same style, or to check whether your examples are varied enough to be useful. It is happy to critique its own inputs, and this bootstrapping trick turns a single sample into a full set in one turn.

The other way to show instead of tell is to hand Claude the reference material directly. Rather than describing your brand voice or your data, paste the style guide, the source document, the spreadsheet, or the transcript, and ask Claude to work from it. When you do this with a long document, one habit matters more than any other: put the long material near the top of your

message, above the actual question. Claude answers noticeably better when the bulky reference comes first and your instruction comes last, especially when you have pasted several documents at once.

I TIP

With a large pasted document, ask Claude to first pull out the quotes that are relevant to your question, then answer using only those quotes. Grounding the reply in specific passages keeps Claude focused on the material that matters and cuts down on it wandering into the noise of a long file. It is the single most effective move for long-document work.

2.3 Give a prompt a shape Claude can read

Once a prompt grows past a sentence or two, it starts mixing different kinds of content: your instruction, the background context, the document to work on, the examples to imitate. When all of that runs together as one blob, Claude has to guess where one part ends and the next begins, and it sometimes guesses wrong, treating your data as an instruction or your instruction as data. A little structure removes the ambiguity.

The plainest structure is headings. Break your message into labeled parts, like a short memo: a "Context" section, a "Task" section, an "Output format" section. Even that much helps Claude parse a busy request. For anything with a document or variable input embedded in it, Claude reads XML-style tags especially cleanly, because it has been tuned to recognize them.

A structured prompt

```
<context>
I run a small bakery. The tone should be warm and casual.
</context>

<task>
Write three Instagram captions for tomorrow's sourdough special.
</task>

<constraints>
Under 40 words each. Include one emoji. No hashtags.
</constraints>
```

Nothing about those tags is magic. They are just consistent, descriptive labels that fence each type of content off from the others. Use whatever names read clearly to you and keep them consistent: `<context>`, `<document>`, `<examples>`. When your input has natural layers, such as several documents, you can nest tags to match. The point is to make the boundaries obvious so Claude never has to infer them.

The same section that carries your context is also where you set a role. Telling Claude who it is at the top of the conversation measurably shapes everything that follows. "You are a careful copy editor who cuts every unnecessary word" produces different, better-aimed output than the same request with no framing at all. One sentence of role is one of the cheapest upgrades available to you, and it pairs naturally with the context block above.

2.4 When to let Claude think first

For a hard problem, an answer improves when Claude reasons through it before committing to a reply, the same way you would sketch on scratch paper before writing a final answer. Claude offers a mode for exactly this, usually called extended or adaptive thinking, where the model works through the problem step by step and then gives you its conclusion.

KEY TERM

Extended thinking a mode where Claude reasons through a problem internally before answering, which raises quality on tasks that need real multi-step reasoning. On current models Claude increasingly decides for itself when a question warrants it, scaling the effort to the difficulty of what you asked.

The useful judgement is knowing when thinking earns its keep. Reach for it when the task genuinely needs reasoning: a tricky logic or math problem, planning a multi-step piece of work, weighing several options against each other, debugging why something went wrong. On those, the extra deliberation shows up directly in the quality of the answer.

! GOTCHA

Thinking is overkill for simple work. A quick lookup, a short rewrite, a factual question with one clear answer: forcing deep reasoning on these buys you nothing and only makes the reply slower. The heuristic is plain. If the problem is one you could answer instantly yourself, Claude can too, and it does not need to think out loud to get there.

How you switch thinking on, and whether it is even a control you touch, depends on which app and which model you are using, and Anthropic keeps changing the specifics as models improve. The durable takeaway is the concept: harder questions benefit from more deliberation, easy ones do not, and newer models handle more of that decision on their own. For how to enable it on the model in front of you today, check the current thinking documentation instead of the setting you saw in an older guide.

I TIP

You can nudge thinking with plain words even without touching a setting. For a careful answer, add "work through this step by step before you answer." To catch mistakes on math or logic, end with "before you finish, double-check your answer against the requirements." A short verification instruction reliably surfaces errors the first pass missed.

2.5 Styles: lock in a tone once

Everything so far shapes one message. When you find yourself typing the same tone-and-format instructions at the start of every chat, promote them into a reusable style. A style is a saved set of instructions about how Claude should write for you: the voice, the formatting, the length, the things to always do and always avoid. You set it once and Claude applies it to every conversation you point at it, so you stop re-explaining your preferences each morning.

A style is the natural home for the format discipline from earlier in this chapter. If you always want flowing prose instead of bullet-point soup, or a warm first-person voice, or replies that skip the throat-clearing preamble and get straight to the point, those belong in a style. Write them the same way you would write any good instruction: say what to do, explain why when it helps, and be specific about the format you want.

QUICK REFERENCE · WHAT TO REACH FOR WHEN

If you want to	Reach for
Get a clearly-specified one-off task right	A specific prompt: goal, format, constraints
Nail an exact tone or structure that is hard to describe	A few examples (three to five)
Work against a long document or dataset	Paste it at the top, ask for quotes first
Keep instructions and content from blurring together	Headings or XML-style tags
Solve a hard, multi-step reasoning problem	Extended thinking or a "step by step" nudge
Apply the same voice and format across every chat	A saved style

Notice the through-line across the whole chapter. Every one of these moves is a way of removing a guess. A specific prompt removes the guess about what you want. Examples remove the guess about what good looks like. Structure

removes the guess about which words are the task. Thinking removes the guess that comes from answering too fast. A style removes the guess about your voice. Get in the habit of asking, before you hit enter, what is Claude still having to guess here, and your results climb without any cleverness at all.

CHAPTER 2 · CHECKLIST

You can now

- ✓ Brief Claude like a capable new hire: state the goal, the format, and the constraints up front.
- ✓ Run the colleague test on a prompt and fix it before sending if a person would be confused.
- ✓ Phrase actions as imperatives so Claude does the work instead of only suggesting it.
- ✓ Steer output with three to five relevant, varied examples when words alone fall short.
- ✓ Put long reference material at the top of a message and ask Claude to quote before it answers.
- ✓ Separate instructions from content with headings or XML-style tags, and set a role in one sentence.
- ✓ Judge when extended thinking helps a hard problem and when it is wasted on a simple one.
- ✓ Save a reusable style so your tone and format carry across every chat automatically.

Chapter Three

Projects, Knowledge & Artifacts

How to turn one-off chats into a reusable workspace that already knows your context, and into finished deliverables you can publish and share.

A plain chat forgets the moment you close it. Every new conversation starts cold, so you paste the same background, restate the same rules, and re-upload the same reference file you uploaded yesterday. For a quick question that is fine. For a body of work you return to every day, it is a tax you pay over and over, and it quietly caps how good the answers can get.

Two features exist to end that. A **Project** is a persistent workspace that remembers your instructions, your reference files, and every chat you hold inside it, so each new conversation walks in already briefed. An **Artifact** is a standalone thing Claude builds beside the chat: a document, a chart, a small web app, something you can edit, publish, and hand to someone else.

This chapter takes both past the one-line summary from Chapter 1. You will set up a Project the right way, learn what belongs in its standing instructions versus a single chat, load reference knowledge without choking the workspace, build and refine an Artifact through plain follow-ups, and finish with a clear rule for which container fits which job.

3.1 What a Project actually bundles

A Project ties three things together into one durable space. Understanding the three parts is most of the battle, because each one solves a different flavor of the "explain it again" problem.

KEY TERM

Project a persistent Claude workspace that combines custom instructions, an uploaded knowledge library, and all the chats you run inside it. Every new chat in the Project inherits the instructions and the knowledge automatically.

The first part is **custom instructions**: a standing brief that behaves like a system prompt scoped to this one workspace. It defines who Claude is being for you here, the tone to write in, and the rules to hold across every conversation. The second part is the **knowledge library**: files you upload once as reference, so Claude can consult them in any chat without you attaching them again. The third part is the **chats themselves**, all kept together in the Project so the work stays in one place instead of scattered across your history.

The payoff is that instructions and knowledge load into every new conversation for free. You open a fresh chat inside the Project and Claude already knows the brand voice, already has the style guide, already understands the standing rules. That is the whole reason Projects exist: a consistent context you set up once and reuse indefinitely.

► DO THIS · STAND UP YOUR FIRST PROJECT

- 1 Open the Projects area.** Go to claude.ai/projects, or hover the left sidebar and click **Projects**. Choose to create a new one and give it a clear name, like the client, the course, or the research topic it will hold.
- 2 Write the custom instructions.** Keep them general: the role Claude plays, the voice to use, and the constraints that are true in every chat. Section 3.2 covers what to put here and what to leave out.

- 3 **Add reference knowledge.** Upload the documents Claude should treat as source material: the brand guide, the spec, the transcript set. These become searchable in every chat without re-attaching.
- 4 **Start a chat inside the Project.** Ask something that leans on both the instructions and the files. When the answer reflects your voice and your documents without you pasting either, the workspace is working.

Projects are available on every plan, including Free. Free accounts can create a handful of them; paid plans (Pro, Max, Team, and Enterprise) remove that ceiling and let you make as many as you want. If you are running several distinct bodies of work, that alone is a reason to be on a paid tier.

I TIP

If your account has memory turned on, each Project keeps its own **separate** memory. What Claude learns inside a client Project stays inside that Project and never surfaces in an unrelated personal one. That isolation is a confidentiality boundary you can rely on, so keep genuinely separate work in genuinely separate Projects rather than one catch-all space.

3.2 Instructions that belong in the Project, and the ones that do not

The most common Project mistake is stuffing the custom instructions with today's task. The instructions load into every conversation, so anything you write there applies to all of them. That makes the box perfect for durable context and wrong for one-off asks.

Put the **standing** things in the Project instructions. The role Claude is playing (a copy editor, a financial analyst, a patient tutor). The voice and formatting you always want. The hard constraints that never change, like a reading level to hold to, a term to avoid, or a currency to price in. Official guidance is to keep this section concise and focused on general context, because a short, sharp brief is followed more reliably than a long rambling one.

Put the **specific** things in the individual chat. The document you want edited today. The exact question. The one-time exception to a usual rule. Task detail lives in the conversation where it applies, so it does not silently bleed into next week's unrelated work in the same Project.

QUICK REFERENCE · PROJECT INSTRUCTIONS VS THE CHAT

Put in Project instructions	Put in the individual chat
The role Claude plays for this workspace	The specific task you need done today
Tone and formatting you always want	A one-time deviation from the usual tone
Standing rules that hold every time	The exact document or data for this request
Vocabulary, audience, reading level	Follow-up questions and revisions

There is a second lever worth knowing, because people reach for the Project instructions when they actually want it. A **Style** controls how Claude writes, the tone and length and formatting, and you can apply it to any chat regardless of Project. Presets like Concise or Explanatory ship built in, and you can create your own from a writing sample or a description. When you want the same voice everywhere you go, a Style travels with you across Projects. When you want facts, rules, and role fixed for one workspace, that is the Project instructions. They compose cleanly, so a formal Style can run inside a client Project without conflict.

3.3 Loading knowledge without choking the workspace

Project knowledge is reference material Claude can pull from in any chat. You upload the files once and they stay put. Common formats work directly, including PDF, DOCX, CSV, TXT, HTML, RTF, ODT, and EPUB, which covers most of what a working document set is made of.

The knowledge library has room, and it also has limits, so treat it like a shelf rather than a junk drawer. There is a per-file size cap and an overall budget for how much total material a Project can hold in active context, roughly the equivalent of a few hundred pages of English text before Claude has to start

being selective about what it pulls in. The exact figures shift as the product evolves, so check the official Projects documentation for the current per-file size and total capacity rather than trusting a number you memorized months ago.

I UNDER THE HOOD

You do not have to keep your library tiny to keep it fast. On paid plans, as a Project's knowledge grows past what fits comfortably in context, Claude switches to retrieval mode: it searches the library and pulls only the chunks relevant to your current question instead of loading every file every turn. That is why a large, well-organized knowledge base still answers quickly. The practical effect is a much larger effective capacity than the raw context window suggests, and it happens automatically with no toggle to flip.

Two habits keep a knowledge base useful. First, upload the durable reference material, the things Claude should treat as ground truth every time, and leave the disposable, one-off files for a normal chat attachment instead. Second, give files clear names and prune what has gone stale, because a library full of outdated drafts pollutes answers just as surely as no library at all.

! GOTCHA

A scanned or image-heavy PDF loses its visual detail past a certain page count. Claude reads the full text of a long document, but charts, diagrams, and handwriting in a very long scan may not be analyzed the way they are in a short one. The upload still succeeds, so the degrading is silent. If a chart deep inside a 400-page scan matters, pull that page out into its own short file rather than trusting the big upload to see it.

3.4 Artifacts: building something you can keep

A Project organizes your inputs. An Artifact is an output. It opens in a side panel next to the conversation and holds a single standalone deliverable that lives on its own, separate from the chat scrolling past it.

KEY TERM

Artifact a standalone piece of work Claude creates in a side panel: a document, a chart or diagram, an SVG image, a single-page web app, a small tool. You refine it with plain-language follow-ups and can publish it to a shareable link.

You get an Artifact in one of three ways. Describe what you want in a normal chat and Claude opens one when the output calls for it. Ask outright, with something like "make this an artifact." Or start from the Artifacts area, describe the goal, answer a clarifying question or two, and let Claude produce a working first version in a few turns. All three land in the same place: a live panel you can watch take shape.

Plenty of things make good Artifacts. A polished document such as a report, memo, or proposal. A chart or a flowchart. An SVG graphic. A single-page website in HTML, CSS, and JavaScript, static or interactive. A small calculator or tool your team can actually use. The common thread is a self-contained deliverable, something with edges you would want to hand to another person, rather than a passing answer inside a conversation.

The refining loop is where Artifacts earn their keep. Once a first version exists, you steer it in plain language. "Make the buttons bigger." "Add a total row." "Use a warmer color for the header." Each follow-up edits the Artifact in place, so you shape it by describing changes the way you would to a designer sitting next to you. When something breaks, a **Fix with Claude** button takes a pass at the problem, or you can just describe what went wrong and let Claude repair it.

▶ WORKED EXAMPLE · FROM ROUGH ASK TO SHARED TOOL

You want a simple tip calculator for your restaurant staff. You type: "Build a tip calculator where I enter the bill and pick a tip percentage, and it shows the total and the per-person split." Claude opens an Artifact with a working single-page app in a few turns.

You try it and refine by talking. "Add an 18, 20, and 22 percent button." "Let me set the number of people with a plus and minus." "Make the total bigger

and easier to read on a phone." Each request updates the same Artifact. When a button misbehaves, you click Fix with Claude and it sorts itself out.

Satisfied, you click **Publish**. Claude gives you a link. You send it to the team, and they open the calculator in a browser and use it, with no Claude account of their own required. A rough sentence became a real tool your staff uses, and you never wrote a line of code.

Publishing is the last step. A **Publish** button at the bottom of the Artifact panel turns it into a shareable link. Most published Artifacts are view-only for anyone with the link, and viewers do not need a Claude account to open them. Other Claude users can **Remix** a published Artifact, taking their own editable copy to build from without touching your original.

I TIP

If your first attempt is close and you want to try a bolder direction without losing it, fork the conversation. Forking branches the thread so you can explore a different version while the original Artifact stays intact. It is the safe way to gamble on a big change when you already have something decent in hand.

! GOTCHA

An Artifact that itself calls Claude, the kind that acts like a small assistant with built-in knowledge, behaves differently when shared. A viewer has to sign in to use it, because it needs to talk to the model on their behalf. A plain static Artifact opens for anyone with the link. And Artifact availability on mobile has moved around over time, so if you rely on building them from your phone, confirm current mobile support in the app before you count on it.

3.5 What to use when

With chat, Projects, and Artifacts all in hand, the only real skill left is choosing the right container before you start. The choice comes down to two questions. Are you returning to this context repeatedly, and are you producing a standalone thing you will keep?

PROJECTS, KNOWLEDGE & ARTIFACTS

Reach for a **Project** when the answer to the first question is yes: you have a body of knowledge and a consistent set of rules you will lean on again and again. A client and all their brand assets. A research corpus you keep querying. A course you are building over weeks. The setup cost pays for itself the second time you open a chat and Claude already knows everything. Reach for a plain chat when it is a one-off with no reusable context, where the setup would cost more than the task.

Reach for an **Artifact** when the answer to the second question is yes: the goal is a deliverable with edges, something to edit, publish, and hand off. A document, a chart, a tool, a page. The two features stack, so the strongest setup is often both at once. You build inside a Project so Claude has your context, and the chat produces an Artifact you publish and share.

QUICK REFERENCE · WHICH CONTAINER FOR THE JOB

If the job is	Use	Why
A quick question with no reusable context	Plain chat	Zero setup, fastest path to an answer
Repeated work against the same knowledge and rules	Project	Instructions and files load into every chat automatically
Producing a document, chart, app, or tool to keep	Artifact	A standalone thing you can edit, publish, and share
A reusable deliverable that needs your standing context	Artifact inside a Project	Your knowledge and voice, producing a shareable output
The same voice applied everywhere you write	Style	Travels with you across chats and Projects

One reflex captures all of it. Before you type, ask whether you will come back to this context and whether you want to keep what comes out. A yes to the first points at a Project. A yes to the second points at an Artifact. A no to both means a plain chat is exactly right, and there is no shame in that being most of your day.

CHAPTER 3 · CHECKLIST

You can now

- ✓ Name the three things a Project bundles: custom instructions, a knowledge library, and its chats.
- ✓ Stand up a Project and confirm its instructions and files load into every new chat automatically.
- ✓ Split standing context into Project instructions and keep one-off asks in the individual chat.
- ✓ Upload reference knowledge in the right formats and keep the library lean, named, and pruned.
- ✓ Create an Artifact, refine it with plain-language follow-ups, and repair it with Fix with Claude.
- ✓ Publish an Artifact to a shareable link and know when a viewer needs an account to use it.
- ✓ Choose the right container for a job, and combine a Project and an Artifact when both fit.

Chapter Four

Skills & the Skill Creator

How to give Claude reusable, on-demand expertise that shows up only when a job needs it, including expertise you build yourself without writing a line of code.

You have probably noticed yourself pasting the same instructions into Claude over and over. The exact way you like a meeting summary formatted. The checklist you run every blog post through. The tone rules for replying to a customer. Each time, you retype the briefing before you get to the actual task, and each time Claude follows it well and then forgets it the moment the chat ends.

A skill is how you stop retyping. It is a small package of know-how you save once and Claude pulls in automatically whenever a task calls for it. Think of it as a specialist you keep on call. You do not brief the whole staff on every job. You bring in the tax specialist only when there are taxes to do, and the rest of the time that expertise sits quietly out of the way.

This chapter is the friendly introduction. You will learn what a skill is in plain terms, how to use skills that already exist inside the app, and how to build your own using a guided helper called the skill creator, all without opening a code editor. The deep, technical craft of authoring skills by hand lives in Chapter 13. Here you get the concept and the easy path.

4.1 What a skill actually is

A skill is a packaged capability: a named bundle of instructions, and sometimes a few supporting files, that teaches Claude how to do one specific kind of task well. You could think of it as a recipe card Claude can reach for. It has a name, a short description of what it is for, and the steps to follow when the moment arrives.

KEY TERM

Skill a reusable, self-contained set of instructions Claude loads only when a task matches it. It teaches one procedure or capability, saves you from re-explaining it, and stays out of the way until it is needed.

The quiet magic is in those last four words: until it is needed. A skill loads *on demand*. Claude keeps a short list of the skills available to it, just the names and one-line descriptions, so it knows what it has on hand. The full contents of any single skill stay closed until a task actually calls for it. The moment your request matches, Claude opens that skill, reads the whole procedure, and follows it.

That "loads on demand" property is why skills are worth learning. Everything you give Claude to read up front, standing instructions and reference material, takes up room in its working attention, and a head crammed with a dozen procedures it does not need right now is a distracted head. A skill keeps its detail folded away until the relevant task appears, so Claude stays focused on the job in front of it and still has the full procedure the instant it matters. You get the depth without paying for it every second.

I TIP

A good rule for when to make a skill: the second time you catch yourself pasting the same instructions or checklist into a fresh chat, that procedure wants to be a skill. Once is a one-off. Twice is a pattern, and patterns are exactly what skills are for.

Skills are also a shared standard, not a Claude-only invention. They follow an open format called Agent Skills, which means a skill is a portable little artifact you own and can carry with you. For this chapter that mostly means one comforting thing: what you learn here is a durable idea, and the skill you build today is yours to keep.

4.2 Using skills that already exist

You do not summon a skill with a special command or a menu. You just ask for what you want in plain language, the way you always do. Behind the scenes Claude reads your request, compares it against the descriptions of the skills available to it, and if one clearly fits the job, it opens that skill and works from its instructions.

So the trigger is your words. Ask for a task that a skill covers, phrased naturally, and the skill activates on its own. This is why a skill's description matters so much: that one line is what Claude matches your request against. A vague description gets missed; a clear one that names the kind of task and when to use it gets found. You will feel this directly the first time you build your own in the next section.

On the app, the skills available to Claude are the ones you have turned on for your account. You manage them from the Customize panel in the desktop app, or from the skills settings on claude.ai. Turning a skill on adds it to Claude's on-call list for every chat. Turning it off takes it back out. This is worth knowing because it explains a common surprise: a skill a teammate showed you will not fire in your account until you have actually enabled it for yourself.

I TIP

Not sure whether a skill did the work? Ask. A quick "which skill did you use for that?" gets you a straight answer, and it is the fastest way to learn which of your requests reliably trigger which skills. Over a few days you develop a feel for the phrasing that lands.

GOTCHA

A skill is guidance, and Claude decides when it fits. If a skill you expected does not fire, the usual cause is a description that does not clearly match how you asked, or the skill was never enabled for your account. Re-phrase your request to name the task plainly, or check that the skill is switched on. The deep controls for forcing exactly when a skill runs are a Chapter 13 topic.

4.3 Building your own with the skill creator

Here is the part that surprises people. You do not have to write a skill by hand to have a good one. Anthropic ships a helper called the skill creator whose entire job is to build skills for you. You describe the capability you want in ordinary language, it asks a few clarifying questions, and it assembles a proper, well-formed skill from your answers.

Think of the skill creator as a friendly interviewer. You bring the knowledge, the way you like a task done, and it handles turning that into the right shape: a clear name, a description written to trigger cleanly, and the step-by-step instructions underneath. It spares you the file details entirely, which is exactly what you want at this stage.

► DO THIS · MAKE YOUR FIRST SKILL

- 1 Enable the skill creator.** Turn it on from the Customize panel in the desktop app, or from your `claude.ai` skills settings, the same place you manage every other skill. Now it is on call like any other capability.
- 2 Describe what you want, plainly.** Start a chat and tell it the job. For example: "Build me a skill that turns my messy meeting notes into a clean summary with a decisions section and an action-items list, one owner per action." Speak in outcomes, the way you would brief a competent assistant.

- 3 **Answer the questions it asks.** The skill creator will probe for the parts you left implied: the exact format you want, the tone, the edge cases, when this skill should and should not kick in. Every answer sharpens the result, so be specific.
- 4 **Review the draft it produces.** Read the description especially, because that line is what decides when your skill fires later. Make sure it names the task the way you would naturally ask for it. Adjust anything that feels off.
- 5 **Save it and test it.** Once saved, open a fresh chat and ask for the task in your own words, without mentioning the skill by name. If it activates and produces what you wanted, you are done. If it stays quiet, tighten the description and try again.

That test at the end is the whole game, and it splits into two separate questions worth keeping apart in your head. First, did the skill *trigger* when you asked naturally? Second, was the *output* actually what you wanted? A skill can fire perfectly and still produce the wrong format, or produce great work only when you name it outright. Fixing the first is a description problem. Fixing the second is an instructions problem. Knowing which one you are looking at saves you a lot of guessing.

I TIP

Keep each skill about one thing. A skill that tries to be your summarizer and your email drafter and your code reviewer all at once triggers unpredictably, because no single request matches its sprawling description. One skill, one job, one clear description. If you have three jobs, make three skills.

4.4 Chat, Projects, or a skill: what to use when

You now have three ways to give Claude context, and they are easy to confuse. Plain chat, Projects from Chapter 1, and skills. They are not rivals. They cover different shapes of need, and the fastest way to see the difference is to ask what each one keeps and when it shows up.

Plain chat keeps nothing past the conversation, which is perfect for a one-off. A Project keeps a body of knowledge and a standing set of instructions

that load into every chat inside that one workspace, which is perfect for ongoing work against the same material. A skill keeps a single reusable procedure that loads on demand, in any chat, only when a task calls for it. That last combination, everywhere yet only when needed, is what a skill uniquely gives you.

QUICK REFERENCE · CHAT VS PROJECT VS SKILL

	Plain chat	Project	Skill
Holds	Just this conversation	Files, instructions, and its chats	One reusable procedure
Loads when	You are in the thread	Every chat in that workspace	A matching task appears, in any chat
Best for	A quick one-off with no setup	Ongoing work on the same body of knowledge	A repeatable how-to you want everywhere
Reach for it when	You will not need this again	The material and rules stay constant	You keep re-explaining the same procedure

A simple way to choose. If you will never repeat the task, stay in chat. If the constant is a pile of reference material and a consistent voice, build a Project. If the constant is a *procedure*, the steps of how a task should be done, make it a skill. The tell is in what repeats. Repeating knowledge points to a Project. Repeating a method points to a skill.

The three also stack. A skill activates happily inside a Project chat, so your brand Project can carry all the reference knowledge while a summarizer skill handles the formatting whenever you ask for a recap. You can even lean on more than one skill in a single task, letting Claude reach for each as the work moves through stages. You do not orchestrate any of this by hand. You ask for the outcome, and the right pieces show up. Getting deliberate about combining several skills into a repeatable workflow is where Chapter 13 picks up.

4.5 Where this goes next

Everything so far has been the app-level view: enable a skill, ask in plain language, build your own with a guided helper. That is genuinely enough to

make Claude noticeably more capable this week, and most people never need more than this.

There is a deeper layer for when you do. Skills can be authored directly, with fine control over the exact words that trigger them, which tools they are allowed to touch, and how several of them combine into a single dependable workflow. That is the operator's craft, and it is the whole subject of Chapter 13 in Part III. When a skill you built is close but you want it sharper, or when you are ready to wire skills together into something that runs like a small machine, that is the chapter waiting for you. For now, the guided path is all you need.

CHAPTER 4 · CHECKLIST

You can now

- ✓ Explain a skill in one sentence as a reusable capability Claude loads only when a task needs it.
- ✓ Say why "loads on demand" matters: it keeps Claude focused and its depth out of the way until it counts.
- ✓ Trigger an existing skill by asking for the task in plain language, and confirm which skill fired.
- ✓ Enable and disable skills for your account from the Customize panel or your claude.ai skills settings.
- ✓ Build your own skill end to end with the skill creator, without writing any files.
- ✓ Test a new skill on the two separate questions of triggering and output quality.
- ✓ Choose correctly among plain chat, a Project, and a skill based on what actually repeats.

Chapter Five

Connecting Claude to Your World

How to give Claude safe, direct access to your other apps and the live web, so it can read and act where your real work already lives.

By default, Claude knows only what you type into the box. It cannot see your calendar, your Google Drive, the ticket you filed in Linear this morning, or the invoice sitting in your inbox. Everything it works on, you first have to copy in by hand. That copying is the quiet tax of using any assistant that lives in a sealed room, and it caps how much real work you can hand off.

Connecting Claude removes the wall. Once you link an outside tool, Claude can read from it and, in many cases, act inside it, without you pasting a thing. Ask it what is on your calendar next week and it looks. Ask it to pull the three newest tickets and summarize them and it does. This is the shift from an assistant that talks about your work to one that reaches into it.

This chapter is the plain-English version of that idea. You will learn what a connector is, what the technology underneath is called and why it matters, how to add a connection from the app safely, and what Claude in Chrome lets you do inside your own browser. The deep, technical build-it-yourself version lives in Chapter 16. Here, you get the friendly tour and the safety habits that keep it from biting you.

5.1 What a connector actually is

A connector is a bridge between Claude and one of your other tools. You set it up once, grant it permission, and from then on Claude can talk to that tool on your behalf. Think of it as introducing Claude to an app it did not previously know existed, and handing it a limited key to work there.

KEY TERM

Connector a saved link between Claude and an outside service, such as Google Drive, Gmail, a calendar, or a project tracker. Once connected, Claude can read data from that service and often take actions in it, using the access you granted.

Two kinds of access matter here, and it is worth being clear about the difference from the start. Some connectors are read-only: Claude can look at your files or messages but cannot change anything. Others let Claude act: create a calendar event, draft an email, add a row to a sheet. The action kind is where connectors become genuinely powerful, and also where a little care goes a long way, which is why the whole back half of this chapter is about safety.

The payoff is concrete. Instead of describing your data to Claude, you point Claude at the source and let it fetch what it needs. A research corpus in Drive, the state of your sprint in a tracker, the thread you are trying to reply to: all of it becomes something Claude can pull in live, in the moment, without a copy-paste relay running through your keyboard.

5.2 MCP, explained gently

Behind almost every connector is one piece of technology with an intimidating name and a simple job. It is called the Model Context Protocol, or MCP. You do not need to understand its internals to use it, any more than you need to understand USB to plug in a keyboard. That comparison is not an accident.

KEY TERM

MCP (Model Context Protocol) an open standard, created by Anthropic and given to the wider community, for connecting an AI application to outside tools and data. Anthropic describes it as "a USB-C port for AI applications": one universal adapter so any tool can plug into any AI that speaks the standard.

Here is why that adapter idea is the whole point. Before a shared standard existed, every company that wanted its app to work with an AI had to build a custom, one-off integration for each AI, and every AI had to build one for each app. That is a tangle that scales badly. MCP replaces the tangle with a single shape. A company writes one MCP connector for its service, and that connector works with Claude, and with other AI tools that speak MCP, without redoing the work each time. When you browse a directory of connectors, you are really browsing everything that has spoken this one common language.

I TIP

You will hear "MCP server" used for the thing on the other end of a connector. In the app, a connector and the MCP server it points at are effectively the same idea wearing two names. "Connector" is the friendly, product-facing word; "MCP server" is what the engineers call the program that answers when Claude reaches out.

That is as far into the machinery as Part I goes, and it is genuinely all you need to use connectors well. When you want the full picture, the host-and-server model, every kind of transport, how permissions and authentication really work, and how to build a connector of your own from scratch, Chapter 16 in Part III covers it end to end. For now, hold the one image: a universal adapter that lets your tools plug into Claude.

5.3 Adding a connection from the app

Adding a connector is a short, guided flow, and Anthropic curates a browsable directory of reviewed ones so you rarely start from nothing. You pick the service you want, Claude walks you through granting access, and from that

point the tool is available in your chats. Two families of connection are worth telling apart.

The first is a remote connector, which links Claude to a service that lives in the cloud, like Gmail or a project tracker. You add these from your connector settings on claude.ai, and once connected they work from both the web app and the desktop app on your account. The second is a Desktop Extension, a one-click package that installs a connector to something running on your own computer. Desktop Extensions install into the Claude desktop app on Windows and Mac, and they are the friendly path when the tool you want to reach lives locally rather than in the cloud.

▶ DO THIS · CONNECT YOUR FIRST TOOL SAFELY

- 1 Open your connector settings.** On claude.ai, go to your connector settings and browse the directory of available connectors, or open the desktop app and look for its extensions area for tools that run on your own machine.
- 2 Pick a tool you already trust.** Start with something first-party and low-stakes, like a calendar or a file store you own, before you reach for anything obscure.
- 3 Read the permission screen before you approve.** When you connect, a screen tells you what access you are granting: which account, and whether Claude can only read or can also act. Read it. This is the moment that decides what Claude can touch.
- 4 Grant, then test small.** Approve the access, then ask Claude one gentle, read-only question against the new connector to confirm it works before you trust it with anything that changes data.

That permission screen is the heart of the whole process. Granting a connector is a real decision about access, in the same weight class as letting any third-party app into your Google or Microsoft account. The connection persists until you remove it, so what you approve today is live tomorrow. Treat the approval moment with the attention it deserves and the rest is easy.

GOTCHA

A connector that can act does so with **your** identity and permissions. If you connect an email tool that can send, Claude can send mail as you. If you connect a tracker that can edit, Claude can change tickets that your teammates see. The lesson is a simple discipline: connect only tools you trust, grant only the access a task genuinely needs, and never approve a connector you cannot vouch for because a web page or a message told you to.

GOTCHA

A connector can pull in outside content, and that content can carry instructions aimed at Claude rather than at you. A malicious document or web page might contain hidden text that tries to get Claude to leak your data or take an action you never asked for. This is called prompt injection, and it is the core reason Anthropic's own guidance is to verify you trust a connector before adding it. Anything you connect that fetches external content is a doorway, so be sure of who built it.

TIP

On Team and Enterprise plans, connectors are usually managed by an administrator, not by individual users. If a connector you expect is missing, or you cannot add one yourself, that is by design. Ask whoever runs your workspace to enable it rather than hunting for a workaround. Centralized control is what keeps a whole organization's connections vetted.

5.4 Claude in Chrome

Connectors reach into services through a clean, defined door. Claude in Chrome does something more direct and more powerful. It is a browser extension that lets Claude act inside your own Chrome tabs, on the sites where you are already logged in, by clicking buttons and filling in forms the way you would with your own hands.

The distinction matters. A connector talks to a service through a structured connection built for machines. Claude in Chrome drives the actual website,

using your live, signed-in session. If a site has no connector but you can do a task in it by clicking around, Claude in Chrome can often do that same task by clicking around for you. It is the tool for the long tail of web work that no formal connector covers, from filling a multi-step form to pulling numbers out of a dashboard that only exists as a page.

That reach is exactly why it deserves the most caution in this chapter. Claude in Chrome is operating in your real browser, with your real logins, on pages that can contain anything. Everything the two gotchas above warned about is sharper here, because the model is acting directly on live sites rather than through a reviewed connector.

GOTCHA

A web page Claude is reading in Chrome can carry hidden instructions crafted to hijack it, the same prompt-injection risk as before, now on the open web where you did not choose the content. Watch what Claude does in the browser rather than walking away, keep it on sites you trust, and stay especially alert on any page that touches money, credentials, or account settings. When a task involves logging in, entering a password, or a payment, do that part yourself. Handing those specific actions to any automated browser is a line worth keeping.

TIP

Reach for Claude in Chrome when the job is genuinely web-shaped and no connector exists for it: scraping a report off a page, filling a tedious form, walking through a checkout you have already decided on. When a proper connector *does* exist for the service, prefer it. A connector's structured door is steadier and safer than driving the website by hand, so save the browser for the tasks that have no other path.

5.5 What to connect when

The best way to decide what to connect is to work backward from the jobs you keep doing by hand. Every connection below removes a specific copy-

paste chore. Read down the left column until you find a job you actually have, and connect the tool that unlocks it.

QUICK REFERENCE · COMMON CONNECTIONS AND THE JOB EACH UNLOCKS

Connect	So Claude can	Reach for it when
A file store (Drive, OneDrive)	Read your documents and pull facts straight from them	Your reference material already lives in the cloud
Email	Read threads and draft replies in context	You spend real time triaging or answering mail
A calendar	Check availability and set up events	Scheduling and planning eat your day
A project tracker (Linear, Jira, Asana)	Read tickets, summarize status, update items	Your work is tracked as tasks and issues
A team chat (Slack)	Search discussion and pull decisions back out	Important context is buried in message history
A knowledge base (Notion, Confluence)	Read your docs and wiki as living reference	Your team's knowledge lives in a shared workspace
Claude in Chrome	Act inside any site you are logged into	The task is web-shaped and no connector fits

One habit turns all of this into a discipline instead of a pile of permissions. Connect the tools you use constantly and leave the rest disconnected until a real job asks for them. A short list of well-chosen, trusted connections beats a long list you cannot remember approving. When a connection stops earning its place, remove it. The point is a Claude that reaches exactly as far into your world as you have deliberately let it, and no further.

CHAPTER 5 · CHECKLIST

You can now

- ✓ Explain in one sentence what a connector is and why it beats copy-pasting your data in.
- ✓ Tell a read-only connector from one that can act on your behalf, and why the difference matters.
- ✓ Name MCP as the open standard underneath connectors and give the USB-C adapter analogy.
- ✓ Add a connector from the app or a Desktop Extension from the desktop app, and read the permission screen.
- ✓ Recognize the approval moment as a real access decision and grant only what a task needs.
- ✓ Describe what Claude in Chrome does and name the extra caution a live browser demands.
- ✓ Pick the right connection for a recurring job and keep your connected list short and trusted.
- ✓ Know that Chapter 16 holds the full technical MCP deep dive when you want it.

PART II

How Claude Actually Works

Two short chapters on the machinery under the surface. How a language model turns your words into its next words, what a token and a context window really are, and how software talks to Claude through the API. This is the mental model the rest of the book stands on. Understand it once, and every feature that follows makes immediate sense.

Chapter Six

Inside the Machine

How a language model actually turns your words into its words, and the three facts about it that change how you work forever.

You do not need to know how an engine works to drive a car. You do need to know how one works to be a great mechanic. This short chapter is the mechanic's view of Claude. It has no terminal, no configuration, and no steps to copy. It has a mental model, and that model is what makes every technique in the rest of this book feel obvious instead of magical.

Three ideas carry the whole thing. A model predicts one piece of text at a time. It can only see a fixed amount of text at once. And it remembers nothing between conversations on its own. Hold those three, and you will understand why Claude behaves the way it does, why it sometimes fails, and why the setup work in this book pays off. We take them one at a time.

6.1 It predicts the next token, over and over

At its core, a large language model does one thing. Given a stretch of text, it predicts what comes next. It produces a small piece of text, adds that piece to the running text, and predicts again. It repeats until the answer is done. Everything Claude does, every essay, every plan, every line of code, is that single loop running at speed.

The model learned to make these predictions by reading an enormous amount of writing and getting very good at the question "what word tends to

follow these words." That is why a good prompt works. You are setting up the beginning of a passage such that the most natural continuation is the answer you want. You are steering a prediction, not querying a database.

I UNDER THE HOOD

The prediction is not a single certain answer. At each step the model produces a spread of likely next pieces with probabilities attached, then samples one. That controlled randomness is why the same prompt can give slightly different answers on two runs, and why a model can be fluent and confident while still being wrong. It is choosing plausible continuations, and plausible is not the same as true.

This is also the honest explanation for hallucination. When a model does not have the facts, it does not stop. It keeps predicting the most plausible-sounding continuation, and a confident fabrication is often more plausible-sounding than an admission of ignorance. The behavior that makes Claude fluent is the same behavior that occasionally makes it invent. Knowing that, you verify anything that matters, and you give Claude the real source material so the plausible continuation and the true one become the same sentence.

6.2 Tokens are the unit of everything

The "pieces" the model reads and writes are called tokens. A token is a chunk of text, usually a common word or a fragment of a longer one. As a rough feel, a token is about three quarters of a word in English, so a page of text is several hundred tokens. The model never sees letters or words the way you do. It sees a stream of tokens.

KEY TERM

Token the basic unit of text a model reads and writes. Roughly a short word or a piece of a longer one. Your input is measured in tokens, the reply is measured in tokens, and both cost and limits are counted in tokens.

Tokens matter to you for two practical reasons. They are the unit of cost, because paid API access and usage limits are measured per token in and per

token out. And they are the unit of the limit we discuss next, the context window. When someone talks about a model handling "200K context," they are counting tokens. Getting a feel for token size is enough. You do not need to count them by hand.

6.3 The context window is finite working memory

Everything the model considers when it makes its next prediction has to fit inside one space called the context window. That space holds your current message, the earlier turns of the conversation, any files or instructions that were loaded, and the reply as it is being written. It is the model's entire working memory for that moment, and it has a fixed size.

KEY TERM

Context window the fixed budget of tokens a model can hold in view at once. Instructions, conversation history, uploaded files, and the answer in progress all share this one space.

A fixed budget has consequences you feel every day. A very long conversation eventually fills the window, and older turns have to be summarized or dropped to make room. When that happens, details from early in a marathon session can quietly fade. Loading a giant file spends a large part of the budget on that file, leaving less room for reasoning. And two unrelated topics crammed into one long thread compete for the same space, which is why a fresh conversation for a fresh task often gives sharper results than continuing a bloated one.

TIP

Treat the context window as a desk, not a warehouse. Put on it exactly what the current task needs and clear off what it does not. Starting a new chat for a new job, keeping loaded files relevant, and summarizing where you can are all the same move: protecting a scarce, valuable space so the model can think clearly inside it.

! GOTCHA

A bigger context window is not a filing cabinet you should stuff full. Even when a model can technically hold a huge amount of text, burying the important part in the middle of a wall of loosely related material makes it easier to miss, not harder. Relevance beats volume. Give Claude the pages that matter, not every page you have.

6.4 Every request starts from a blank slate

Here is the fact that surprises people most. The model itself is stateless. Between one request and the next it retains nothing. It does not quietly remember your last conversation, your name, or the correction you made an hour ago. Each time you send a message, the model is handed a block of text and asked to continue it, with no private memory of anything before.

So how does a chat feel continuous? Because the tool around the model re-sends the earlier turns every single time. What looks like memory is really the whole conversation being packed back into the context window on each request, so the model can read the history fresh and continue it. The "memory" lives in the text you resend, not in the model.

I UNDER THE HOOD

This one fact explains an enormous amount of the rest of this book. A memory file like CLAUDE.md works precisely because the model is stateless: the file is text that gets loaded into the context of every session, standing in for the long-term memory the model does not have. Projects, knowledge, and system instructions are all the same trick. They are durable text that gets re-presented so a stateless model appears to remember. Once you see that memory is something you supply as context, the entire operator toolkit clicks into place.

6.5 The mirror principle

Put the three facts together and a working rule falls out. The model continues the text it is given, using only what fits in its window, with no memory beyond that. Its output is therefore a reflection of its input. Vague, thin, or contradictory input yields vague, thin, or contradictory output. Specific, well-organized, well-sourced input yields the opposite. The model is a mirror with a talent for language.

This is not a motivational slogan. It is a direct consequence of next-token prediction on a finite context. It means the highest-leverage thing you control is the quality of what you put in front of the model: the clarity of your instructions, the relevance of the files you load, the structure of your prompt, the rules you make standing. Chapter 2 turned this into technique. This chapter tells you why that technique is not a trick. It is how the machine works.

I TIP

When an answer disappoints, resist the urge to blame the model and re-ask the same way louder. Look at the input instead. Was the goal stated plainly? Was the needed material actually in the window? Was the context clean or cluttered with an unrelated earlier topic? Nine times in ten, fixing the input fixes the output.

6.6 What this buys you

You now hold the model everything else stands on. When a later chapter tells you to keep CLAUDE.md short, you know why: it spends context. When it tells you to start a fresh session for a fresh task, you know why: a clean window thinks better. When it tells you to give a subagent its own context, or to hand Claude the source file instead of describing it, you will recognize the same three facts underneath. The techniques will feel less like rules to memorize and more like the obvious thing to do.

The next chapter takes this one level down, to the software layer. It shows how a program talks to Claude through the API, which is the same mental model with a precise shape around it.

CHAPTER 6 · CHECKLIST

You can now

- ✓ Explain in one sentence that a language model generates text by predicting one token at a time.
- ✓ Define a token and say why tokens are the unit of both cost and limits.
- ✓ Describe the context window as finite working memory shared by instructions, history, files, and the reply.
- ✓ Explain why a model is stateless and how re-sent context creates the feeling of memory.
- ✓ Connect statelessness to why memory files, Projects, and system instructions exist.
- ✓ State the mirror principle and use it to diagnose a weak answer by inspecting the input.

Chapter Seven

The API

How software talks to Claude, explained as a mental model you can hold whether you write code or just want to understand what the apps are doing for you.

Every Claude surface you have met so far, the chat box, Projects, Cowork, the command line, is a friendly face on one plain mechanism underneath. A program sends Claude some text and gets some text back. That mechanism is the API, and this chapter is its mental model. You will not write a full application here. You will understand the shape of the thing so the apps stop being magic and, if you ever want to build, you know where the door is.

An operator benefits from this even with no intention of coding. Once you see the request and response underneath, you understand why a system instruction behaves differently from a chat message, what "tool use" really means when Claude checks the weather or edits a file, and why picking the right model is a real decision with a real cost. This is the physics. The apps are the engineering built on top of it.

7.1 An API is a doorway for programs

A person talks to Claude through a window on a screen. A program talks to Claude through an API, which stands for application programming interface. The name is heavier than the idea. An API is simply an agreed-upon doorway: a program sends a request in a defined format to an address on the internet,

and it gets a response back in a defined format. That is the whole transaction. Request goes out, response comes back.

KEY TERM

API a defined doorway that lets one program ask another for something. For Claude, your program sends a request containing the conversation so far, and the response contains Claude's reply. Anthropic's is reached through the Developer Platform.

The request Claude expects is not mysterious. At its heart it carries three things: which model you want to use, the conversation to continue, and a few dials that shape the reply. The response carries Claude's next message and some accounting, including how many tokens the exchange cost. Once you can picture those two envelopes, the request and the response, you can picture every Claude product working.

7.2 System and user: who is saying what

A request does not send one blob of text. It sends a short structured conversation made of messages, and each message has a role. The two roles you meet first are the user and the assistant. User messages are what the human said. Assistant messages are what Claude said. A back-and-forth chat is just this list growing by two entries each turn, then getting sent again so Claude can continue it. This is statelessness from Chapter 6 made concrete: the history is resent every time.

Standing above that list is the system prompt. It is not part of the conversation. It is the standing brief that frames the whole exchange: who Claude should be, the rules to follow, the format to produce. The model treats it as the governing context for everything the user then says. When you set custom instructions on a Project, or a style, or a CLAUDE.md, you are writing something that ends up in this system position. That is why those instructions feel more like settings than like messages. They sit in a different slot.

■ a request, simplified to its shape

```
{
  "model": "a-claude-model",
  "system": "You are a precise assistant for a law office.",
  "messages": [
    { "role": "user",      "content": "Summarize the attached
contract." },
    { "role": "assistant", "content": "Here is the summary ..." },
    { "role": "user",      "content": "Now list the termination
clauses." }
  ]
}
```

I TIP

The exact field names and options change as the platform evolves, so treat the block above as the shape, not the spec. When you are ready to build, the current request format lives in the official API reference on the Developer Platform. Learn the shape [here](#), look up the details [there](#).

7.3 The reply, and why it can stream

Claude's reply comes back as tokens, produced in the same one-at-a-time loop from Chapter 6. A program can wait for the whole reply and receive it in one piece, or it can stream, receiving each token the instant it is generated. Streaming is why the chat apps type the answer out in front of you instead of pausing and then dumping a finished wall of text. The answer is genuinely being written live, and streaming just forwards each piece as it lands.

Streaming buys responsiveness, which matters most for long replies where waiting in silence would feel broken. For a short, quick call a program might skip it and take the whole answer at once. It is a delivery choice, and the content is identical either way.

7.4 Tool use: how Claude reaches beyond text

A model that only predicts text cannot check today's weather, query your database, or edit a file. Tool use is the bridge, and it is the single most important idea in this chapter, because it is the engine under Cowork, the command line, connectors, and every agent in this book.

The mechanism is a conversation, not a magic power. You tell Claude, in the request, which tools exist and what each one takes. When Claude decides it needs one, it does not run anything itself. It pauses and returns a structured message that says, in effect, "please call the weather tool with this city." Your program runs that tool, gets the real result, adds the result to the conversation, and sends it back. Claude reads the result and continues. The loop repeats until the job is done and Claude gives a final answer.

I UNDER THE HOOD

That loop is the whole definition of an agent. A model that can request a tool, receive the result, and decide what to do next is an agent, and everything from a subagent to Cowork to Claude Code is that loop wrapped in a good harness. The model never touches your files or the network directly. It asks, your surrounding program acts, and the result comes back as more context. Permissions and hooks, which arrive later in the book, live exactly at that "your program acts" step, which is why they can gate what an agent is allowed to do.

KEY TERM

Tool use the pattern where the model, instead of answering, requests that a named tool be run with specific inputs. The surrounding program runs it and returns the result as context. Also called function calling. It is how a text model takes real action.

7.5 Thinking as a visible step

Modern Claude models can do an explicit reasoning pass before they answer, working through a hard problem in a scratch space and then producing the final reply. Through the API this extended thinking is something a program can request and, within limits, size, spending more effort on genuinely hard reasoning and less on simple calls. It is the same capability the apps expose when they take a longer, more deliberate pass on a tough question.

The operator's takeaway is a judgement call, not a setting to memorize. Deep reasoning helps on planning, multi-step logic, and thorny analysis. It is wasted effort on a quick lookup or a short rewrite. Reach for more thinking when the problem is hard, and keep it light when the task is routine. The current controls and their exact behavior are on the Developer Platform, since this is an area that moves quickly.

7.6 Models come in tiers, and the tier is a decision

Anthropic offers a family of models rather than a single one, and they trade the same two things against each other: capability and cost. The pattern has been durable even as the names and numbers change. There is a fast, inexpensive tier for high-volume, straightforward work. There is a balanced tier for everyday tasks. And there is a most-capable tier for the hardest reasoning, priced accordingly. Every tier is billed per token in and per token out, so both the model you pick and the amount of context you send move the bill.

QUICK REFERENCE · CHOOSING A MODEL TIER BY THE JOB

Tier	Reach for it when	Trade-off
Fast and inexpensive	High volume, simple classification, quick extraction, cheap drafts	Lowest cost and latency, least raw capability
Balanced	Everyday writing, coding, and analysis	The usual default for real work

**Most
capable**

Hardest reasoning, planning, and high-
stakes output

Highest capability, highest
cost

! GOTCHA

Reaching for the most capable model for everything is a quiet way to overspend. Much real work runs beautifully on the balanced tier, and a lot of bulk work runs fine on the fast one. Because the model names and prices change often, confirm the current lineup on the Anthropic models page before you standardize on one. Match the tier to the difficulty of the task, not to a habit.

7.7 Why this matters even if you never write code

You may never open a code editor, and this chapter still pays off. Every app in Part I is a request and a response with a nice interface around it. A Project is a saved system prompt plus attached knowledge. Cowork is a tool-use loop running on your behalf. A connector is a set of tools Claude is allowed to call. Once you see the API underneath, the products stop being separate mysteries and become variations on one clear idea.

And if you do decide to build, you now know the map. A request with a system prompt and a message list, a response that can stream, a tool-use loop for real action, a thinking step for hard problems, and a tier to choose. The official documentation fills in every exact detail. What you have is the thing documentation cannot give you: the shape of the whole, held in your head.

CHAPTER 7 · CHECKLIST

You can now

- ✓ Describe an API as a defined doorway where a program sends a request and gets a response.
- ✓ Explain the difference between the system prompt and the user and assistant messages.
- ✓ Say what streaming is and why the chat apps type replies out live.
- ✓ Explain the tool-use loop and why it is the engine under agents, Cowork, and connectors.
- ✓ Describe extended thinking and judge when deeper reasoning is worth the effort.
- ✓ Choose a model tier by matching capability and cost to the difficulty of the task.
- ✓ See each Part I app as a specific arrangement of the same request, response, and tools.

PART III

Claude Code: The Operator's Power Tools

This is where Claude stops being a place you visit and becomes part of how you work. The terminal half of the manual goes deep on the machinery: memory, permissions, plan mode, commands, skills, hooks, subagents, and the connective tissue that turns a chat assistant into an operating system for your projects.

Chapter Eight

Meet Claude Code

How to leave the chat window behind and move to the command line, where Claude stops answering questions and starts doing the work.

The Claude app is a conversation. You ask, it answers, you copy the answer somewhere useful. That loop is excellent for thinking out loud and terrible for getting real work shipped, because you are the one moving every file, running every command, and pasting every result back in. You are the hands. Claude is only the head.

Claude Code closes that gap. It is the same model, given a terminal, your files, and the permission to act. It reads your codebase, edits files in place, runs your tests, and reports back with the results already in hand. The chat app hands you a plan. Claude Code executes the plan and shows you the diff. This is the surface where the leverage in this manual actually lives, and every later chapter assumes you are here.

This chapter is the on-ramp. By the end you will know what Claude Code is, when to reach for it over the app, how to install and sign in on any platform, the two ways to run it, how it lives inside your editor, and how to keep an eye on what it costs. The deeper machinery, memory in Chapter 9, permissions in Chapter 10, plan mode in Chapter 11, waits for you on the far side of a working install.

8.1 What Claude Code is, and when to graduate to it

Claude Code is Anthropic's agentic coding tool. The word that matters is *agentic*: it operates on your project instead of only talking about it. Point it at a folder and it can list the files, read them, write new ones, run shell commands, and loop on the output until the job is done. The conversation is still there. The difference is that every turn can end in an action instead of a suggestion.

KEY TERM

Claude Code Anthropic's agentic tool for working on real projects. It runs the same models you know from the app, wrapped in a harness that can read and edit your files, run commands, and use tools on your behalf, all inside a workspace you control.

It shows up in several places, and they share one brain. There is a command-line tool named `claude` that runs in any terminal. There is a native VS Code extension with a graphical panel. There is a JetBrains plugin for IntelliJ, PyCharm, WebStorm, and the rest. There is a desktop app, a web version, and a CI mode for GitHub Actions and GitLab. Most of these surfaces read the same `~/.claude/` configuration and share the same conversation history, so a rule you write once tends to follow you everywhere.

The recurring question this manual keeps asking is *what to use when*, and the line between the app and Claude Code is the first place to answer it. Reach for the app when the work is conversational: brainstorming, drafting an email, reasoning through a decision, learning a concept, anything where the output is text you will read and use yourself. Reach for Claude Code the moment the work touches files on your machine. Refactoring a module, running a test suite, wiring up a script, fixing a bug across a dozen files, automating a task you do every week: that is agent work, and doing it through copy-paste in a chat window is the slow road.

I TIP

You do not need to be a developer to benefit from Claude Code. If you can open a terminal and point it at a folder, Claude can handle the rest: organizing files, running a data script, editing a batch of documents. The command line is the doorway. What happens past it is a conversation, the same one you already know how to have.

8.2 Installing and signing in

Getting Claude Code onto your machine takes one command, and the exact command depends on your operating system. The recommended path is the native installer, a small script that downloads a self-contained binary and keeps it updated in the background so you rarely think about upgrades again. Anthropic publishes the current one-liner for macOS, Windows, and Linux on the official install page, and because those commands change with releases, the durable move is to copy the live version straight from the docs, since one printed in a book goes stale.

```
claude --version
```

That is the command you run *after* installing, from any terminal, to confirm it worked. It should print a version string. If it does, the binary is on your path and you are ready to sign in. If the command is not found, the install did not finish or your shell has not picked up the new path yet, and a fresh terminal usually fixes the second case.

I TIP

Run `claude doctor` right after installing. It is a read-only health check that validates your install and your settings without starting a session, and it flags common problems with the specific fix attached. When anything feels off later, this is the first command to run, before you start guessing.

There are other ways in. Homebrew on macOS, WinGet on Windows, npm for anyone who lives in the Node ecosystem, and the standard Linux package

managers all carry Claude Code. They install the same binary. The one thing to know is that most of these alternatives do not auto-update by default, so you upgrade them yourself with the package manager's own command. If you want the version to quietly keep itself current, the native installer is the one to pick. When you choose a package manager, check its current requirements on the setup page first, since details like the minimum Node version for the npm route shift between releases.

GOTCHA

Never install the npm package with `sudo`. A global install run as root creates permission problems that are genuinely painful to unwind later. If npm complains about permissions, the fix is to configure a user-owned global directory, never to reach for `sudo`. The native installer sidesteps this entirely by installing into your home directory.

With the binary installed, authentication is the last step. Change into any project folder and run `claude`. On the first launch it opens a browser window to log in. If the browser does not open on its own, the prompt tells you how to copy the login URL, and on a remote or containerized setup where you get a code instead of a redirect, you paste that code back at the prompt. Sign in once and the credentials are stored securely for next time.

What you sign in *with* depends on your account. Most individuals log in with a Claude Pro, Max, Team, or Enterprise subscription, and that is the recommended path. If you work off the API instead, you can authenticate with a Claude Console account backed by prepaid credits. Larger organizations can point Claude Code at a cloud provider like Amazon Bedrock or Google Cloud, or at a self-hosted gateway for corporate single sign-on. Inside a running session, `/login` switches accounts and `/logout` signs you out.

TIP

For a script or a CI job that runs without a human at the keyboard, do not paste your password anywhere. Run `claude setup-token` to generate a long-lived token through the normal browser flow, then hand that token to the automation through an environment variable. It is the clean way to give an unattended process access without exposing a credential in plain text.

8.3 The two ways to run it: interactive and one-shot

Claude Code runs in two modes, and knowing which one a task wants is most of the skill of driving it well. The default is interactive: you type `claude`, a session opens, and you have a back-and-forth conversation that stays alive until you leave. The other is one-shot, also called headless or print mode, where you hand Claude a single instruction on the command line, it does the work, prints the result, and exits.

KEY TERM

Headless mode running Claude Code with the `-p` (or `--print`) flag so it executes one prompt and exits without an interactive session. Because it returns plain output and a clean exit, it is the form you drop into scripts, pipelines, and any automation that calls Claude and moves on.

Interactive mode is where you spend real working time. It holds context across turns, lets you course-correct mid-task, and gives you the slash commands, permission prompts, and checkpoints that the rest of Part III is built on. This is the mode for anything exploratory or anything long, which is to say most of your day.

One-shot mode is for everything scriptable. The `-p` flag turns Claude into a command you can call from a shell script, a cron job, or a larger program, and you can pipe content into it the same way you pipe into any other Unix tool. The examples below show the shapes side by side.

terminal

```
# Interactive: open a session, optionally with a first prompt
claude
claude "explain the auth flow in this repo"

# One-shot: run once, print the answer, exit
claude -p "summarize what changed in the last commit"

# Pipe content in and let Claude work on it
cat error.log | claude -p "find the root cause in this log"
```

Two flags come up so often they are worth learning on day one. `--continue` (or `-c`) picks up the most recent conversation in the current folder, so closing your terminal by accident costs you nothing. `--resume` (or `-r`) reopens an older session, either by name or through a picker of your recent conversations. Between them you almost never lose a thread.

QUICK REFERENCE · WAYS TO START CLAUDE CODE

Command	What it does
<code>claude</code>	Open an interactive session in the current folder.
<code>claude "task"</code>	Open interactively with a first prompt already sent.
<code>claude -p "query"</code>	Run one prompt, print the result, exit. The scriptable form.
<code>claude -c</code>	Continue the most recent conversation in this folder.
<code>claude -r</code>	Resume an earlier session by name or from a picker.
<code>claude --model sonnet</code>	Start on a specific model by its alias.

I TIP

Once a session is open, type `/help` to see every command available to you, and type `/` alone to browse the full menu of built-in commands, skills, and plugin commands. When you are learning, these two are your map. Everything the manual references from here lives one slash away.

▶ DO THIS · YOUR FIRST SESSION

- 1 Change into a real project folder.** Run `cd` to the directory you want to work in, then start Claude with `claude`. The session header shows your model, your working directory, and the current version so you know where you are.
- 2 Run `/init`.** Claude reads your codebase and writes a starter `CLAUDE.md`, a memory file that teaches future sessions how your project works. Chapter 9 is entirely about this file. For now, just let it write the first draft.
- 3 Ask it something concrete.** Try *"walk me through how this project is structured"* and watch it read files and answer from what it actually finds, not from a guess.

4 **Leave and come back.** Exit with `/exit`, then reopen with `claude --continue` to confirm the thread is right where you left it.

8.4 Living inside your editor

The terminal is home base, and for many people the editor is where Claude Code feels most natural. Two first-class integrations bring it into the tools you already use, and both connect to the same account and the same configuration as the CLI.

In VS Code, Claude Code installs as a native extension. You search for it in the Extensions view, install it, and a graphical panel appears. The payoff is inline diff review: before you accept an edit, you see exactly what changes in the editor's own diff view, and you approve or reject it there. You can `@`-mention specific files and folders to pull them into the conversation, review a plan with inline comments, and browse your session history without leaving the window.

In JetBrains IDEs, the story is the same shape with a native feel. You install the plugin from the JetBrains Marketplace, restart the IDE, and launch Claude Code from a keyboard shortcut or a toolbar button. It runs inside the IDE's own terminal, so the permission modes and flags you learn in the next few chapters behave exactly as they do in a plain terminal, and diffs open in the IDE's native diff viewer.

! GOTCHA

Installing the VS Code extension does **not** put `claude` on your shell path. The extension bundles a private copy of the CLI that only powers its own panel. If you want to run `claude` in VS Code's integrated terminal, you have to install the standalone CLI separately. The JetBrains plugin is stricter still: it bundles no CLI at all, so you always install both the plugin and the standalone tool.

TIP

The IDE integrations and the CLI share `~/.claude/settings.json` for the things that matter across surfaces: permissions, environment variables, hooks, and MCP servers. Set a rule once and it applies whether you launch from the terminal or the editor panel. The editor has a few of its own display preferences on top, but the operating rules are shared, which is exactly what you want.

8.5 Keeping an eye on cost

Claude Code does more per turn than a chat message, so it is worth knowing what it costs and where the meter runs. The command to watch is `/usage`. Run it inside any session and it shows a breakdown of the current session's token counts with a dollar estimate, and on a subscription plan it also shows how much of your plan limit you have consumed, with a toggle between a daily and a weekly view.

GOTCHA

Older tutorials call this command `/cost`. That name still works as an alias, so nothing breaks if you type it out of habit. The current, canonical name is `/usage`, and that is the one to learn. Naming like this shifts between releases, so if a command you read about somewhere is missing, check `/help` for what it was renamed to before assuming it is gone.

The dollar figure `/usage` shows is a local estimate based on list pricing, so treat it as a running gauge for staying aware, not as your invoice. The authoritative number lives on your Console or account billing page. For a session, the estimate is more than accurate enough to tell you when a task is getting expensive and worth splitting up.

Keeping cost down comes from a few durable habits. Run `/clear` between unrelated tasks so a fresh job does not drag the whole history of the last one along with it. Use `/compact` to summarize a long session when the context window fills. Pick a cheaper model with `/model` when the work does not need

MEET CLAUDE CODE

your strongest one. And for an unattended run where you want a hard ceiling, the `--max-budget-usd` flag stops the session once it has spent a set amount, which is a useful seatbelt for anything you are not watching live.

I TIP

The single biggest lever on cost is context discipline, and it is the subject of the very next chapter. A lean, well-built memory file means Claude walks into each session already oriented, so you spend fewer tokens re-explaining your project every morning. Chapter 9 shows you how to build one. Cost control and good memory turn out to be the same skill.

CHAPTER 8 · CHECKLIST

You can now

- ✓ Say what makes Claude Code agentic, and draw the line between when to use the app and when to use the CLI.
- ✓ Install Claude Code on your platform from the official docs, and verify it with `claude --version` and `claude doctor`.
- ✓ Sign in with the right account type, and generate a long-lived token for an unattended script.
- ✓ Choose between interactive mode and one-shot `-p` mode based on whether the task is exploratory or scriptable.
- ✓ Recover any thread with `--continue` and `--resume` instead of starting over.
- ✓ Run Claude Code inside VS Code or a JetBrains IDE, and avoid the trap that the bundled CLI is not on your path.
- ✓ Track spend with `/usage` and rein it in with `/clear`, `/compact`, `/model`, and a budget cap.

Chapter Nine

Memory & Files

How to make Claude remember your project, your rules, and your preferences so you never explain them twice.

Every Claude Code session begins with an empty head. Close the terminal, reopen it tomorrow, and the model remembers nothing about your project, your naming conventions, or the correction you made three times last week. That blank slate is a feature. It keeps each session clean and predictable. It also means the burden of context is on you, and re-explaining yourself every morning is a tax you should refuse to pay.

Memory is how you refuse to pay it. Claude Code gives you two persistent mechanisms that load automatically at the start of every conversation, so the assistant walks in already knowing how your world works. Getting them right is the single highest-leverage setup move in this entire manual. A well-built memory file is the difference between an assistant you manage and one that manages itself.

This chapter covers both mechanisms, the exact files and folders they live in, the order Claude reads them, the import syntax that keeps them tidy, and the two-file pattern that professional operators use to separate stable rules from day-to-day state.

9.1 Two kinds of memory

Claude Code remembers things in two complementary ways, and they answer two different questions. One is the memory *you* write. The other is the memory

Claude writes for itself.

KEY TERM

CLAUDE.md a plain Markdown file you author by hand. It holds your instructions: coding standards, architecture notes, workflows, hard rules. Claude loads it in full at the start of every session and treats it as standing context.

KEY TERM

Auto memory notes Claude writes about your project based on corrections and preferences it picks up as you work. Stored per repository under `~/.claude/projects/<project>/memory/`, with a `MEMORY.md` index loaded each session. This is the assistant learning your habits without being told twice.

You will spend most of your effort on CLAUDE.md, because that is the lever you control directly. Auto memory runs quietly in the background and gets better the more you correct Claude in plain language. The table below is the whole distinction on one page.

QUICK REFERENCE · CLAUDE.MD VS AUTO MEMORY

	CLAUDE.md files	Auto memory
Who writes it	You	Claude
Contents	Instructions and rules	Learnings and patterns
Scope	Project, user, or organization	Per repository, shared across worktrees
Loaded each session	In full	First ~200 lines (or 25 KB) of <code>MEMORY.md</code>
Reach for it when	Standards, workflows, architecture	Build commands, debugging insights, discovered preferences

! GOTCHA

Both kinds of memory are **context, not enforcement**. Claude reads them and usually follows them, the same way you follow a well-written brief. There is no guarantee. If you need to *hard-block* an action no matter what the model decides, a memory file is the wrong tool. Use a `PreToolUse` hook instead (Chapter 14). Memory shapes behavior; hooks enforce it.

9.2 The memory hierarchy and who wins

CLAUDE.md is not one file. It is a stack of files at different scopes, from machine-wide policy down to your personal notes for this one project. Claude loads all of them and concatenates them into a single block of context. Knowing the order they load in is what lets you keep a personal preference from quietly overriding a team rule, or the reverse.

QUICK REFERENCE · MEMORY SCOPES, BROADEST TO NARROWEST

Scope	Location	Purpose
Managed policy	C:\Program Files\ClaudeCode\CLAUDE.md (Windows)	Organization-wide rules, set by IT. Applies to everyone on the machine.
	/Library/Application Support/ClaudeCode/CLAUDE.md (macOS)	
	/etc/claude-code/CLAUDE.md (Linux / WSL)	
User	~/.claude/CLAUDE.md	Your personal preferences across every project you touch.
Project	./CLAUDE.md or ~/.claude/CLAUDE.md	Team-shared instructions, committed to source control.
Local	./CLAUDE.local.md	Your private notes for this one project. Add it to .gitignore.

The rule for who wins is simple once you see the mechanism. Files load root first and leaf last, so the **most specific instruction is read last**, closest to the model's recent-context bias. Nothing is overridden by force. Everything is concatenated. When two files say contradictory things, the later, more specific one tends to carry the day because it is the last thing Claude read.

I UNDER THE HOOD

At launch, Claude Code walks *up* the directory tree from wherever you started it, loading a `CLAUDE.md` and any adjacent `CLAUDE.local.md` from every folder along the way. Start it in `foo/bar/` and it pulls `foo/bar/CLAUDE.md`, then `foo/CLAUDE.md`, and so on up to your home and project roots. All of it is stitched together in load order.

Two behaviors are worth committing to memory. First, the project-root `CLAUDE.md` **survives compaction**: when a long session compacts, Claude re-reads it from disk and re-injects it, so your core rules never fall out of context. Second, block-level HTML comments (`<!-- like this -->`) are stripped before the file is injected, so maintainer notes cost you zero tokens while staying visible when you open the file yourself.

! GOTCHA

A `CLAUDE.md` sitting in a *subdirectory below* where you launched is **not loaded at startup**. It loads on demand, only when Claude actually reads a file inside that subfolder, and it does not get re-injected after compaction the way the root file does. If a rule absolutely must be in play from the first message, put it in the project-root `CLAUDE.md`, not in `packages/api/CLAUDE.md`.

9.3 Imports: keep one file, split the content

A single `CLAUDE.md` that tries to hold everything becomes a wall of text that Claude scans past and you stop maintaining. Imports fix that. Anywhere inside a `CLAUDE.md` you can write `@path/to/file` and Claude pulls that file's contents in at load time, as if it were written inline.

CLAUDE.md

```
See @README for the project overview and @package.json for available
commands.
```

```
# Additional instructions
```

- Git workflow: @docs/git-instructions.md
- My personal preferences: @~/ .claude/my-project-instructions.md

Two details save you from the common mistakes. Relative paths resolve against the file that contains the import, not your current working directory, so an import behaves the same no matter where you launched Claude. And imports can themselves import, chaining several levels deep. The exact depth limit is the kind of number that shifts between releases, so treat the docs as the source of truth and keep your chains shallow on purpose.

TIP

The home-directory import above is the clean way to share personal preferences across git worktrees. A gitignored `CLAUDE.local.md` only exists in the one worktree where you created it. A line like `@~/.claude/my-project-instructions.md` in the project `CLAUDE.md` follows you into every worktree, because it points at your home folder.

GOTCHA

The first time a *project* `CLAUDE.md` imports a file that lives outside your working directory (that home-folder path counts), Claude Code shows an approval dialog listing the external files. This protects you from an import someone else committed into a shared repo. Decline it and those imports are disabled permanently for that project, and the dialog will not return. Imports in your own user-scope files load without the prompt, since they are already yours.

GOTCHA

Imports organize your files. They do **not** save context. An imported file is loaded in full at launch, so ten tidy imports cost exactly as many tokens as one giant file. Use imports for maintainability. When you actually need to cut token cost, trim the content or move procedures into skills, which load only when needed (Chapter 13).

9.4 The commands that manage it: `/init`, `/memory`, `/context`

You rarely need to create these files by hand. Three slash commands cover the whole lifecycle, from generating a first draft to confirming what actually loaded.

▶ DO THIS · STAND UP MEMORY ON A NEW PROJECT

- 1 **Run `/init` in the project root.** Claude reads your codebase and writes a starting `CLAUDE.md`: detected build commands, test setup, and conventions it can infer. If a `CLAUDE.md` already exists, `/init` proposes improvements instead of overwriting your work.
- 2 **Read the draft and cut it down.** Delete anything Claude can already see from the code itself. Keep the pitfalls, the non-obvious conventions, and the reasons behind them.
- 3 **Add rules as you catch them.** When you correct Claude the second time on the same thing, that correction belongs in the file. Open it with `/memory`, which lists every memory file across scopes and opens the one you pick in your editor.
- 4 **Confirm it loaded.** Run `/context` and check the memory-files list. If your `CLAUDE.md` is not there, Claude never read it, and nothing else in this chapter matters until it is.

I TIP

To teach auto memory, just talk. Say *"always use pnpm, not npm"* or *"remember the API tests need a local Redis instance,"* and Claude saves it to auto memory for next time. When you specifically want something in `CLAUDE.md` instead, say *"add this to CLAUDE.md"* or edit the file through `/memory`.

9.5 The two-file pattern operators actually use

There is a tension baked into CLAUDE.md. It works best when it is stable and short, ideally under two hundred lines. Real projects also have volatile state: what is blocked today, what is running, what you decided yesterday. Cram that into CLAUDE.md and you are rewriting your stable rules file every few hours. The fix is to split stable from volatile and stitch them back with an import.

▶ WORKED EXAMPLE · A BOOTLOADER PLUS A STATUS FILE

Keep two files. `CLAUDE.md` is the bootloader: identity, hard rules, the map of the project, the things that are true every day. It stays short and you almost never touch it. `SESSION_STATUS.md` is the live state: what is blocked, what is running, the last decision. It changes constantly. The bootloader imports the status file on its first line, so both load every session while you only ever edit the volatile one.

■ CLAUDE.md

```
# Project Bootloader
@SESSION_STATUS.md

## Hard rules
- Never commit secrets or .env files.
- Use plan mode for changes under `src/billing/`.
```

Now your morning ritual is editing one short status file, and Claude walks into every session knowing both the permanent rules and today's state. This is exactly the shape of a professional setup: a thin, stable brain and a separate, fast-moving notepad.

9.6 Splitting rules by topic and path

On a larger project, even a well-imported CLAUDE.md gets crowded. The `.claude/rules/` folder lets you break instructions into topic files that Claude

MEMORY & FILES

discovers automatically. Every `.md` file in there loads at launch with the same priority as your project `CLAUDE.md`.

The powerful move is path scoping. Give a rules file a `paths` header and it loads only when Claude is working with matching files, so a big set of API rules stays out of context until Claude actually opens an API file.

```
.claude/rules/api.md

---
paths:
  - "src/api/**/*.ts"
---

# API development rules
- Every endpoint must validate its input.
- Return typed errors, never raw exceptions.
```

I TIP

Brace expansion works in these patterns, so `"src/**/*.{ts,tsx}"` covers both extensions in one line. User-level rules in `~/.claude/rules/` apply to every project and load before project rules, which means your project rules take priority when they overlap. That ordering is deliberate: local intent beats global default.

9.7 Keeping a memory file healthy

A memory file is a living document, and a bloated one actively hurts you. Past roughly two hundred lines per file, you pay in both tokens and obedience: longer files consume more context and lower how reliably Claude follows any single instruction inside them. Treat brevity as a feature you maintain, not a nice-to-have.

Two habits keep it in shape. Be concrete, because specific rules beat vague ones every time. And prune on a schedule, because a rule that made sense in March may be dead weight by July.

QUICK REFERENCE · WRITE RULES THAT STICK

Write this	Not this
------------	----------

Use 2-space indentation.

Format code properly.

Run `npm test` before committing.

Test your changes.

API handlers live in `src/api/handlers/`.

Keep files organized.

I TIP

Recent Claude Code builds ship a `/doctor` command that proposes trims for a checked-in CLAUDE.md. It cuts what Claude can already derive from the codebase, like directory layouts and dependency lists, and keeps the pitfalls, the rationale, and your non-default conventions. Run it when a file starts feeling heavy.

One last refinement. You can steer what a compaction preserves by adding a short section to CLAUDE.md that tells Claude what matters most when it summarizes a long session.

I CLAUDE.md

```
# Compact instructions
```

```
When compacting, focus on test output and code changes.
```

CHAPTER 9 · CHECKLIST

You can now

- ✓ Name the two memory mechanisms and say which one you write and which one Claude writes.
- ✓ Place a CLAUDE.md at the right scope, from managed policy down to a gitignored local file.
- ✓ Predict which instruction wins when two files disagree, using the root-first, leaf-last load order.
- ✓ Use `@` imports to split content cleanly, and explain why they do not cut token cost.
- ✓ Stand up memory on a fresh project with `/init`, `/memory`, and `/context`.
- ✓ Build the two-file pattern: a stable bootloader that imports a volatile status file.
- ✓ Scope rules to file paths with `.claude/rules/`, and keep every file lean and specific.

Chapter Ten

Settings & Permissions

How to configure Claude Code once, safely, and control exactly which actions it may take without asking you first.

Out of the box, Claude Code asks before it does anything consequential. It wants a nod before it runs a shell command, before it edits a file, before it fetches a URL. That is the right default for your first hour and a slow way to work by your fortieth. The whole point of settings is to move the routine yeses out of your way while keeping a hard wall around the things you never want touched.

Two systems do this work together. Settings files are where you write standing configuration: which model to run, what environment to inject, and, most importantly, the permission rules that decide what runs freely and what gets blocked. Permission modes are the session-wide posture you toggle on top of those rules, from cautious to trusting. Learn both and you stop babysitting prompts without ever handing Claude a blank check.

This chapter maps the settings files and the order they load, names the keys worth knowing at a durable level, and then goes deep on the permission model: the allow, ask, and deny tiers, the rule syntax that scopes a permission to one tool and one pattern, and the modes that set the tone for a whole session. Memory (Chapter 9) gives Claude context. Hooks (Chapter 14) give you deterministic enforcement. Settings sit in between, and they are where most of your day-to-day control actually lives.

10.1 Where settings live and who wins

Settings are not one file. They are a small stack of JSON files at different scopes, from a policy your IT department deploys down to a gitignored file that is yours alone on this one repo. Claude Code reads all of them and layers them, so knowing the stack is what lets a team standard hold while you still keep personal overrides on top.

QUICK REFERENCE · SETTINGS FILE LOCATIONS

Scope	Location	Applies to
Managed	C:\Program Files\ClaudeCode\managed-settings.json (Windows) /Library/Application Support/ClaudeCode/managed-settings.json (macOS) /etc/claude-code/managed-settings.json (Linux / WSL)	Every user on the machine. Deployed by IT, cannot be overridden.
User	~/.claude/settings.json	You, across every project on this machine.
Project	.claude/settings.json	Everyone who clones the repo. Committed to git.
Local	.claude/settings.local.json	You, on this one repo. Gitignored automatically.

When the same key is set in more than one file, the more specific file wins, with one exception covered in the next section. The order runs from managed at the top, which nothing can override including a command-line flag, down through command-line arguments, then local, then project, then user settings at the bottom. Set a default model in your user settings and override it for one repo in that repo's project settings, and the project value carries there while your user default holds everywhere else.

I UNDER THE HOOD

Managed settings are parsed tolerantly. If one entry is malformed, Claude Code strips just that entry with a warning and still enforces the rest, so a single typo can never disable an entire org policy. Your user, project, and local files are strict by contrast: invalid JSON or a bad schema rejects the **whole file**. Run `/doctor` to see what got stripped or rejected, and add the schema line below so your editor catches mistakes before Claude does.

I TIP

Put `"$schema": "https://json.schemastore.org/claude-code-settings.json"` as the first line of any settings file. Editors that understand JSON schema then give you autocomplete on every key and flag typos inline. It is the cheapest insurance against a strict-parse rejection you will not notice until Claude ignores your config.

To confirm what is actually active in a running session, use `/status` and read the "Setting sources" line, which lists every file that loaded. On Windows, `~/.claude/` resolves to `%USERPROFILE%\.claude/`. And if you ever corrupt a file, Claude Code keeps timestamped backups of the five most recent versions of each config.

10.2 The keys worth knowing

The full settings schema runs to dozens of keys, and the list grows with every release. Chasing all of them is a waste. A small set carries almost all the value, and the rest you look up in the docs on the rare day you need them. Here is a starter file that shows the shape and the keys you will reach for first.

```
.claude/settings.json

{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "deny": ["Read(/.env)", "Read(/.env.*)", "Read(/secrets/**)"],
    "allow": ["Bash(npm run lint)", "Bash(npm run test *)"]
  },
  "env": {
    "NODE_ENV": "development"
  },
  "cleanupPeriodDays": 30
}
```

The keys break down by what they govern. `permissions` is the heart of the file and gets the next two sections to itself. `env` injects environment variables into every session and subprocess Claude spawns, which is how you set a build flag or a test variable once instead of on every command. `model` sets the default model, and `fallbackModel` gives it a short chain to fall back to when the primary is overloaded. `cleanupPeriodDays` controls how long old session data lingers before Claude Code deletes it at startup. And the `attribution` object customizes the git commit and pull-request trailers Claude adds, which is where you go if you want to change or remove the "Generated with Claude Code" line.

! GOTCHA

Two keys are read **once at startup**: `model` and `outputStyle`. Change either in a file mid-session and nothing happens until you run `/clear` or restart. Almost everything else, including all your permission rules and hooks, reloads live the moment you save the file. If a model change seems to be ignored, this is why.

I TIP

Do not try to memorize the full key list, and do not trust one you memorized last quarter. New keys land and old ones get renamed release to release. When you need something beyond the handful above, read the current settings reference in the docs and let the schema line drive your editor's autocomplete. Treat the durable mental model here as the map and the docs as the live territory.

10.3 The permission model: allow, ask, deny

Everything Claude Code can do runs through one permission system, and it has exactly three verdicts. A tool call is allowed and runs without a prompt, or it triggers an ask and waits for your yes, or it is denied and never runs at all. You shape those verdicts by writing rules into three arrays inside the `permissions` object.

KEY TERM

Permission rule a string of the form `Tool` or `Tool(specifier)` that matches some set of tool calls. Placed in the `allow`, `ask`, or `deny` array, it tells Claude Code to run, prompt for, or block every call that matches.

The verdicts are checked in a fixed order that you must internalize: **deny, then ask, then allow**. The first array that matches a call decides it, and the specificity of a rule does not change that order. This has a sharp consequence. A broad deny beats a narrow allow every time. If you deny `Bash(aws *)` and also allow `Bash(aws s3 ls)`, the call is blocked, because deny is checked first and wins. Deny rules cannot carry allowlist exceptions. When you want most of a family blocked with a few holes, allow the holes and leave the rest to prompt, and reserve deny for the things that must never run.

```

.claude/settings.json

{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git commit *)",
      "WebFetch(domain:docs.anthropic.com)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Bash(curl *)",
      "Read(./env)",
      "Read(./secrets/**)"
    ]
  }
}

```

The specifier in the parentheses is what scopes a rule to a pattern. A bare tool name with no parentheses matches every use of that tool, so `Bash` and `Bash(*)` mean the same all-encompassing thing. Add a specifier and you narrow it. For Bash the specifier is a command pattern where `*` is a wildcard: `Bash(npm run *)` covers every npm script, `Bash(git commit *)` covers any commit invocation. For file tools the specifier is a path, so `Read(./env)` matches that one file and `Read(./secrets/**)` matches everything under that folder. For web access it is a domain: `WebFetch(domain:docs.anthropic.com)`.

! GOTCHA

The space in a Bash pattern is load-bearing. `Bash(ls *)` matches `ls -la` but does **not** match `ls of`, because the space forces a word boundary. Drop the space and `Bash(ls*)` matches both, which is almost never what you want. When you mean "this command followed by any arguments," keep the space, or use the `:*` suffix, which means the same thing.

GOTCHA

Trying to restrict a command by pinning its URL or path in the pattern is a classic footgun. A rule like `Bash(curl http://github.com/ *)` does not reliably confine curl to GitHub. It misses `https://`, misses a `-X GET` placed before the URL, misses redirects, and misses a shell variable holding the address. The safe move is to deny `curl` and `wget` outright and route web access through `WebFetch(domain:...)`, which matches on the real hostname. For anything that must truly hold, back it with a `PreToolUse` hook (Chapter 14).

Two behaviors of the file tools deserve a place in memory. Bash matching is smarter than a plain string compare: Claude Code strips wrappers like `timeout` and `nice` before matching, so `Bash(npm test *)` still covers `timeout 30 npm test`, and a compound command joined by `&&` or a pipe requires *each* subcommand to match on its own. And a set of read-only shell commands, the likes of `ls`, `cat`, `grep`, and read-only `git` forms, runs with no prompt in every mode and cannot be configured. On the file side, a `Read` deny on a path also blocks editing that path, which is how a single `Read(./secrets/**)` deny keeps Claude from both reading and rewriting your secrets.

TIP

The `/permissions` command opens an interactive view of every rule in play and which file each one came from, and lets you add or remove entries without hand-editing JSON. When a call gets blocked or prompted and you cannot see why, this is the first place to look. Note that a "Yes, don't ask again" choice during a session writes a fresh allow rule into `.claude/settings.local.json` at the repo root, so your local file quietly grows an allowlist as you work.

10.4 Permission modes: the session-wide posture

Rules decide individual calls. A permission mode sets the posture for the whole session, the baseline of what runs without asking before any rule is

SETTINGS & PERMISSIONS

consulted. You set it at launch with the `--permission-mode` flag, persist it with `permissions.defaultMode` in a settings file, or cycle it mid-session with `Shift + Tab`. You do not set it by asking Claude in chat, because the mode governs the harness, not the model.

QUICK REFERENCE · PERMISSION MODES

Mode	Runs without asking	Reach for it when
<code>default</code>	Reads only. Everything else prompts.	Getting started, or any sensitive work.
<code>acceptEdits</code>	Reads, file edits, and common filesystem commands in your working directory.	Iterating on code you are actively reviewing.
<code>plan</code>	Reads and read-only exploration. No file edits at all.	Exploring a codebase before you change it.
<code>bypassPermissions</code>	Everything, including writes to protected paths.	An isolated container or VM, and nowhere else.

Most work lives in the first three. You start in `default`, where only reads run free and Claude checks with you before it changes anything, which is exactly the caution you want on unfamiliar or sensitive code. When you trust the direction of a task and are watching the diffs go by, `acceptEdits` stops the constant edit prompts and lets Claude work at speed. Before a large or risky change, `plan` mode lets Claude read and explore to its heart's content while guaranteeing it cannot write a single file, so you get a plan before any code moves. The `Shift + Tab` cycle walks you through these, and the status bar shows the active one, such as `plan mode on`.

KEY TERM

bypassPermissions a mode that skips every permission prompt, including writes to normally protected paths. Its flag form is `--dangerously-skip-permissions`, and the name is a warning. It offers no protection against a prompt-injection attack, so it belongs only in a throwaway container or VM with no network access.

GOTCHA

Never run `bypassPermissions` on your real machine to save a few clicks. It hands Claude your full user permissions with no gate, and a single hostile instruction hidden in a file or a web page can then do real damage. It is designed for isolated, disposable environments only. It also cannot be entered mid-session if you did not enable it at launch, refuses to start as root on Linux and macOS, and shows a one-time warning the first time you use it interactively. Those are guardrails around a dangerous mode, not an endorsement of running it loosely.

TIP

Claude Code has been adding further modes aimed at longer autonomous runs and locked-down scripted use, gated by account, plan, and model support and still moving fast between releases. If you run automated or CI workloads, or want more hands-off operation on a paid plan, check the current permission-modes docs for what is available to your setup instead of relying on a fixed list. For interactive daily work, the four modes above are the durable core, and `default` plus `acceptEdits` will cover most of it.

UNDER THE HOOD

Even `bypassPermissions` holds a few lines it will not cross. It still prompts for rules you explicitly marked `ask`, still honors connector and MCP tools an organization flagged as requiring interaction, and still refuses a catastrophic `rm -rf /` or `rm -rf ~` as a circuit breaker against a model error. There is also a set of protected paths, directories like `.git` and `.claude` and files like your shell rc, that no normal mode will auto-approve a write to. That protection is exactly what bypass mode removes, which is the whole reason to keep it caged.

10.5 Safe defaults and the habits that keep you out of trouble

A good permission setup is boring by design. It blocks the handful of things you never want touched, pre-approves the handful of commands you run constantly, and leaves everything else to prompt. The starting configuration below does that and travels well, because it lives in project settings and so applies to every contributor who clones the repo.

```
.claude/settings.json
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)",
      "Bash(curl *)",
      "Bash(wget *)"
    ],
    "allow": [
      "Bash(npm run lint)",
      "Bash(npm run test *)"
    ]
  }
}
```

The habits matter as much as the file. Put your credential deny rules in project settings, not in your personal ones, so every contributor inherits the same wall around `.env` and `secrets/`. Prefer narrow Bash allow rules over a bare `Bash` allow, since a bare allow signs off on every shell command Claude will ever propose, which defeats the point of having a permission system at all. Start new or sensitive projects in `default` mode and graduate to `acceptEdits` only once you trust the task's direction. And reach for `plan` mode before any change large enough that you would want to read the approach before the code.

***I* TIP**

Layer the two systems for real security. A permission `deny` rule blocks a call at the settings level, and a `PreToolUse` hook (Chapter 14) blocks it deterministically in the harness even when a rule might be worded around. Using both on your most dangerous actions, a deny rule plus a hook guard, is a belt-and-suspenders setup that holds where either one alone might have a gap. Settings are your first line. Hooks are the enforcement floor underneath them.

***!* GOTCHA**

Watch what accumulates in `.claude/settings.local.json`. Every "don't ask again" you click during a hurried session appends an allow rule there, and over weeks that file can quietly widen into a permission set you never audited. Open it now and then, read what has collected, and prune anything you would not have written on purpose. An allowlist you did not review is an allowlist you do not really control.

None of this is set-and-forget in the strict sense, because the keys and modes keep evolving. The durable skill is the shape: a stack of files where the specific overrides the general, three verdicts checked deny-first, and a session posture you dial from cautious to trusting as your confidence grows. Hold that model and the specific keys become lookups, not memory work, and you can walk into any version of Claude Code and configure it safely on the first try.

CHAPTER 10 · CHECKLIST

You can now

- ✓ Name the four settings scopes and predict which file wins when a key is set in more than one.
- ✓ Add the schema line and use `/status` and `/doctor` to see what actually loaded.
- ✓ Reach for the durable settings keys, and know that `model` and `outputStyle` need a restart to take effect.
- ✓ Write allow, ask, and deny rules, and predict outcomes using the deny-then-ask-then-allow order.
- ✓ Scope a rule to one tool and one pattern, and avoid the space and fragile-URL footguns.
- ✓ Choose a permission mode for the task, and explain why `bypassPermissions` stays in a container.
- ✓ Stand up a safe starting configuration and keep `settings.local.json` pruned over time.

Chapter Eleven

Plan Mode

How to make Claude fully research a change and hand you a written plan for approval before it touches a single file.

The most expensive mistake in an agentic session is a confident wrong turn. Claude reads your one-line request, decides it understands, and spends twenty minutes and a pile of tokens editing eight files in a direction you never wanted. Now you are unwinding the work instead of doing it. The fix is to make Claude think first and show you its thinking before it earns the right to write anything.

Plan mode is that gate. It is a permission mode that puts Claude into a read-only research phase: it can explore your codebase freely, but every edit is blocked until you read a proposed plan and sign off on it. You get the full benefit of Claude investigating a large or unfamiliar change, with a guarantee that nothing changes on disk while it does.

This chapter covers what plan mode is and exactly when to reach for it, every way to enter and leave it, how to steer a planning session toward a plan you actually want, the approval flow that turns a plan into execution, and the handful of ways it surprises people.

11.1 What plan mode is, and when to use it

Plan mode tells Claude to research and propose, never to change. While it is active, Claude reads files and runs read-only shell commands to explore, then writes a plan describing the approach it recommends. It does not edit source

PLAN MODE

files. Those edits stay blocked until you explicitly approve the plan. The status bar shows `|| plan mode on` the whole time so you always know you are in it.

KEY TERM

Plan mode a read-only permission mode where Claude investigates a task and writes a proposed plan without making any edits. Editing is unblocked only after you review and approve the plan.

The reason to care is that plan mode is the cheapest insurance you can buy against re-work. Claude settles on an approach and shows it to you **before** spending time and tokens building it, so you catch a wrong direction while it is still a paragraph of text rather than a diff across your repo. Anthropic's own cost-reduction guidance names it directly: cycle to plan mode before implementation to prevent expensive re-work when the initial direction is wrong. It is a token saver and a safety mechanism in the same feature.

So when do you actually reach for it? Force plan mode before any change that is large, risky, or ambiguous: a multi-file refactor, a first change in a codebase you do not know, or anything where a wrong first move is painful to reverse. Reach for it also when you are exploring and do not intend to write code yet at all. A prompt like "analyze the database schema" or "figure out how auth flows through this app" benefits from the read-only guarantee while Claude digs. Skip it for the small and obvious: a one-line fix, a rename, a typo in a comment. Making Claude write a plan to change a log message is ceremony you do not need.

I TIP

Treat plan mode as your default posture for the first prompt in an unfamiliar repository. You lose nothing by letting Claude explore under a read-only guarantee, and you gain a clear picture of what it intends before it touches anything. Once you trust the direction, approve and let it run.

11.2 Getting into plan mode

There are several ways in, and which you pick depends on whether you want plan mode for one prompt, for this session, or as a standing default. The fastest is a keyboard cycle you will use constantly.

Mid-session, press `Shift Tab` to cycle through the permission modes. Plan mode sits in that rotation, after the default manual mode and the accept-edits mode. Keep pressing the pair until the status bar reads `|| plan mode on`. The exact order of the cycle depends on which optional modes your setup has enabled, so watch the status bar rather than counting presses. It tells you where you landed.

To plan a single request without changing the session's mode, prefix that one prompt with `/plan`. Claude plans just that request and the session stays wherever it was. To start a session already in plan mode, launch with the CLI flag.

```
terminal
claude --permission-mode plan
```

To make plan mode the standing default so every session opens in it, set it in your settings JSON. Put it in `.claude/settings.json` to default the whole project, or in `~/.claude/settings.json` to default every project you touch.

```
.claude/settings.json
{
  "permissions": {
    "defaultMode": "plan"
  }
}
```

QUICK REFERENCE · WAYS IN AND OUT OF PLAN MODE

You want	Do this
Enter mid-session	Press <code>Shift Tab</code> until the status bar shows <code> plan mode on</code>

PLAN MODE

Plan one prompt only	Prefix the prompt with <code>/plan</code>
Start a session in it	Launch with <code>claude --permission-mode plan</code>
Make it the default	Set <code>"defaultMode": "plan"</code> in settings JSON
Leave without approving	Press <code>Shift</code> <code>Tab</code> again to cycle out
Approve and execute	Accept the plan at the <code>ExitPlanMode</code> prompt (see 11.3)

TIP

In VS Code, plan mode gets a richer interface than the plain terminal. Pick **Plan** from the mode indicator at the bottom of the prompt box, or set `claudeCode.initialPermissionMode` to `"plan"` in your VS Code settings. The plan opens as a full Markdown document where you can leave inline comments as feedback before Claude proceeds. For heavy plan-driven work on a complex change, it is worth being in VS Code specifically for that editing surface.

GOTCHA

In the desktop app, a mode you pick from the selector is remembered per folder and takes precedence over your `defaultMode` for that folder. Plan is the exception. It applies to the current session only and does not get stuck as the folder default, so you will not accidentally trap every future session in plan mode by choosing it once.

11.3 Driving the session and approving the plan

A planning session is a conversation, not a black box. While Claude is in plan mode it can read files and run the shell commands Claude Code recognizes as read-only, the built-in set like `ls`, `cat`, `grep`, `find`, and `git log` or `git diff`. Those run without pausing to ask you, so Claude explores at full speed while only the state-changing actions stay gated. You steer the same way you steer any session: tell it what to focus on, what to leave alone, and what a good answer looks like.

When Claude has settled on an approach, it calls an internal tool named `ExitPlanMode`, which surfaces the plan text along with an approval prompt. That prompt is where you decide what happens next, and the options come down to two paths: approve and start executing, or send Claude back to keep planning.

QUICK REFERENCE · THE EXITPLANMODE APPROVAL CHOICES

Choice	What it does
Approve, auto-accept edits	Exits plan mode and starts executing with edits accepted automatically. The exact wording shifts with your setup, reading as auto mode or bypass permissions depending on what the session has enabled.
Approve, manually approve edits	Exits plan mode and drops into manual mode, so you review each edit as Claude makes it.
No, keep planning	Stays in plan mode. You tell Claude what to change and it revises the plan.
No, refine on the web	Sends the plan to Anthropic's browser-based review flow for a deeper editing pass.

Approving does two things at once. It exits plan mode and it switches the session into whichever permission mode the choice implies, so Claude begins editing immediately under those rules. If you want to plan again later in the same session, you cycle back in with `Shift Tab` or prefix your next prompt with `/plan`. One approval does not put you in plan mode forever.

***I* TIP**

Before you accept, you can edit the plan yourself. Press `Ctrl G` at the approval prompt to open the proposed plan in your default text editor, make changes, and hand the edited version back for Claude to execute. This is the cleanest way to tighten a plan that is almost right without a round of back-and-forth.

I TIP

When you approve, the session auto-names itself from the plan content, unless you already set a name with `--name` or `/rename`. Some builds also add an approval option that clears the planning context as it accepts, freeing the tokens you spent exploring once the plan is locked in. If your settings expose a control for that, turning it on keeps a long planning session from weighing down the execution that follows.

11.4 Where plan mode surprises people

Plan mode behaves exactly as advertised in a normal session. The surprises live at the edges, in specific configurations and in the difference between a behavioral instruction and a hard technical wall.

! GOTCHA

Plan mode is **not a hard sandbox by itself**. In a session started with bypass permissions available, such as one launched with `--dangerously-skip-permissions`, Claude Code does not enforce the edit-blocking even while you are cycled into plan mode. Claude is still instructed to plan without editing, but an edit or shell command it attempts during planning runs without a prompt. If you need a real guarantee of zero side effects, do not combine plan mode with a bypass-permissions session. The exact interaction here has been revised across recent releases, so confirm the current behavior in the permissions docs before you rely on it.

A second surprise is what "read-only" covers. Read-only shell commands genuinely run without interrupting you, which is what lets Claude explore fluidly. A command that would change state, or a write to a protected path like `.git` or `.claude`, is treated differently. Depending on your configuration it either prompts you for approval right there in the middle of planning or is routed to Claude Code's own classifier to decide. The mechanics of that routing are governed by a setting whose name and default have moved between builds, so read the concept as "state-changing actions during

planning are gated somehow" and check the current docs for exactly how your version handles them.

GOTCHA

Leaving plan mode is not the same as approving a plan. Press `Shift Tab` to cycle out and you abandon planning entirely, returning to the previous mode with no plan approved and nothing executed. Approval happens only through the `ExitPlanMode` prompt. If you meant to accept a plan and instead cycled out, Claude did not start the work.

TIP

If you reject a plan several times in a row and the approval flow starts behaving oddly, press `Esc` or run `/clear` to reset, then restate the task cleanly. A session that has churned through many rejected plans can occasionally get stuck, and a fresh start is faster than fighting it. And if you approve a plan that goes wrong during execution, `/rewind` or a double `Esc` rolls both the code and the conversation back to a checkpoint (Chapter 12).

11.5 A planning session end to end

Here is the whole loop on a change worth planning: adding rate limiting to an API you have not worked in before. It is multi-file, easy to get architecturally wrong, and exactly the kind of task where a wrong first direction costs you an hour.

▶ WORKED EXAMPLE · FROM REQUEST TO APPROVED PLAN

You press `Shift Tab` until the status bar shows `|| plan mode on`, then type your request:

 your prompt

```
Add rate limiting to the public API endpoints. Explore how
requests are handled now and propose an approach before changing
anything.
```

PLAN MODE

Claude goes to work read-only. It runs `grep` to find the route handlers, `cat` s the middleware stack, checks `package.json` for a limiter library already in the tree, and reads how config is loaded. None of this prompts you, because all of it is read-only. A minute later it calls `ExitPlanMode` and surfaces a plan: add a shared limiter middleware, wire it into the three public routes, pull the limits from existing config, and add tests for the throttled path.

You read it and notice it plans to store counters in memory, which will not survive the app running on more than one instance. Rather than approve and fix it mid-flight, you choose **No, keep planning** and say so:

I your redirect

```
Good, but we run multiple instances. Back the counters with the
Redis we already use for sessions, not in-memory.
```

Claude revises the plan to use Redis and re-presents it. This time it is right. You press `Ctrl G`, tighten one wording detail in your editor, save, and approve with manual edit review so you can watch the first few changes land. Plan mode exits, the session switches to manual mode, and Claude starts implementing the plan you actually wanted. You spent two minutes steering and saved an hour of unwinding an in-memory design.

CHAPTER 11 · CHECKLIST

You can now

- ✓ Explain plan mode as a read-only research phase that blocks edits until you approve a plan.
- ✓ Decide when to force plan mode and when it is overkill for a small, obvious change.
- ✓ Enter plan mode four ways: the `Shift Tab` cycle, a `/plan` prefix, the `--permission-mode plan` flag, and `defaultMode` in settings.
- ✓ Steer a planning session and redirect a wrong plan with "No, keep planning" instead of fixing it mid-execution.
- ✓ Approve a plan through `ExitPlanMode` and pick between auto-accept and manual edit review.
- ✓ Edit a proposed plan yourself with `Ctrl G` before it runs.
- ✓ Avoid the traps: plan mode is not a hard sandbox under bypass permissions, and cycling out abandons rather than approves.

Chapter Twelve

Slash Commands & Master Commands

How to take the prompt you keep re-typing and turn it into a single one-word command you own, version, and share.

Type a forward slash in a Claude Code session and a menu drops open. Every entry on it is a shortcut: a piece of work someone decided was worth naming so nobody would ever have to spell it out again. Some of those entries ship with the CLI. The interesting ones are the entries you write yourself, because a command you author is a prompt you never have to type twice.

This is the shell-alias move applied to an agentic session. You have a deploy checklist you paste every Friday, a pull-request rubric you keep re-explaining, a commit-message convention you correct by hand. Each of those is a candidate for a command. You capture it once in a file, give it a name, and from then on it is one word at the prompt.

This chapter covers the built-in commands that steer a session, the file-per-command model for authoring your own, the frontmatter and prefixes that make a command dynamic, and the payoff at the top: a master command that bundles an entire workflow behind a single slash. It closes on where commands are heading, which is straight into Skills (Chapter 13).

12.1 What the built-in commands are for

A slash command is recognized only at the very start of your message. Type the name, and any text after it becomes the command's arguments. Type `/` by itself and Claude Code lists everything available in this session: built-in commands, bundled skills, the custom commands you wrote, prompts exposed by connected MCP servers, and anything a plugin added.

Built-in commands are the control panel for the session itself. They do things the model cannot do from inside a normal reply, like switching model, rolling back to a checkpoint, or showing you what is eating your context window. You will lean on a small core of them every day. The table below is that durable core.

QUICK REFERENCE · BUILT-IN COMMANDS WORTH KNOWING BY HEART

Command	What it does
<code>/help</code>	Shows help and the current list of available commands. Your source of truth when this book and the CLI disagree.
<code>/init</code>	Reads the codebase and writes a starting <code>CLAUDE.md</code> (Chapter 9).
<code>/memory</code>	Opens and edits your memory files across scopes.
<code>/context</code>	Visualizes context-window usage as a grid and warns when you approach the limit.
<code>/compact</code>	Summarizes the conversation to free context, with optional focus instructions.
<code>/model</code>	Switches the model and saves it as the default.
<code>/clear</code>	Starts a fresh conversation with an empty context (aliases <code>/reset</code> , <code>/new</code>).
<code>/resume</code>	Returns to an earlier conversation.
<code>/rewind</code>	Rolls code and conversation back to a checkpoint.
<code>/permissions</code>	Sets approval rules for tools and file access (Chapter 11).
<code>/hooks</code>	Read-only browser of configured hooks (Chapter 14).

SLASH COMMANDS & MASTER COMMANDS

`/usage` Token usage and cost for this session (alias `/cost`).

Beyond that core sit dozens more: background and parallel-work commands like `/background`, `/fork`, and `/subtask`; review commands like `/code-review`, `/security-review`, and `/simplify`; session-portability commands like `/teleport` and `/export`. Many of these are themselves bundled skills instead of fixed CLI logic, which is a distinction Chapter 13 makes concrete.

I TIP

Do not memorize the full built-in list. It changes almost every release: commands get added, renamed, and quietly promoted from fixed logic into skills. Treat `/help` as the live index and the official commands reference as the spec. When this book names a command, trust the behavior and confirm the exact spelling against `/help` in your version.

12.2 Authoring your own: one file, one command

Your own commands live in a folder Claude Code scans automatically. The model is simple and it is the reason custom commands are worth the ten minutes it takes to learn them: **one Markdown file becomes one slash command, and the filename is the command name**. A file at `.claude/commands/deploy.md` creates `/deploy`. The body of the file is the prompt that runs when you type it.

Where the file lives decides who can use it, and the two scopes mirror the memory hierarchy from Chapter 9. A command in your project's `.claude/commands/` is committed to the repo and shared with your team. A command in `~/.claude/commands/` in your home folder follows you into every project on the machine. Same file format, different reach.

QUICK REFERENCE · WHERE CUSTOM COMMANDS LIVE

Scope	Path	Use it for
Project	<code>.claude/commands/<name>.md</code>	Team workflows committed to source control.

Personal `~/.claude/commands/<name>.md` Your habits, available in every project.

Here is the whole thing at its simplest. Create the file, write a prompt, and you have a command. No configuration, no registration step.

```
! .claude/commands/pr-review.md
```

```
Review the current diff as a senior engineer would.
Flag correctness bugs first, then security issues, then style.
For each finding, give the file, the line, and a one-line fix.
End with a single verdict: ship, or do not ship.
```

Type `/pr-review` and that prompt runs verbatim. The command shows up in the `/` menu the moment the file exists, with no restart required. You just turned a paragraph you were pasting into every review into a word.

I TIP

Subdirectories organize commands without changing their names. A file at `.claude/commands/frontend/component.md` still creates `/component`. The `frontend` folder shows up in the command's description in the menu so you can tell two `component` commands apart, and the folder name is not part of what you type.

12.3 Making a command dynamic: arguments, shell, and files

A static prompt is useful. A command earns its keep once it takes input and pulls in live context. Three features do that work: placeholders for arguments you pass in, a `!` prefix that runs shell commands, and a `@` prefix that inlines files. Learn these three and most of what a command can do is in reach.

Start with arguments. Text you type after the command name is available inside the file through placeholders. `$ARGUMENTS` is the whole argument string as you typed it. For individual pieces, `$0` is the first argument and `$1` the second, using shell-style quoting so `"hello world"` stays one argument. An `argument-hint` in the frontmatter tells the autocomplete what to expect.

SLASH COMMANDS & MASTER COMMANDS

```
.claude/commands/fix-issue.md
```

```
---
argument-hint: [issue-number] [priority]
description: Fix a GitHub issue
---
Fix issue #${0} with priority ${1}.
Check the issue description and implement the necessary changes.
```

Running `/fix-issue 123 high` substitutes `${0}` with `123` and `${1}` with `high` before Claude ever sees the text. An indexed placeholder with no matching argument is left in the text unchanged, so design your prompts to read sensibly when an argument is missing.

KEY TERM

Frontmatter a small block of YAML fenced by `---` at the top of a command file. It configures the command without appearing in the prompt: the description shown in the menu, the argument hint, which tools are pre-approved, and more.

The second feature is the one that makes commands feel alive. A backtick-wrapped command with a leading `!`, written as `!`git status``, runs that shell command **before** Claude sees the file, and its output replaces the placeholder as plain text. This is preprocessing. Claude does not run the command; it reads the result you baked in. The effect is a prompt that carries a live snapshot of your working tree.

```
.claude/commands/summarize-changes.md
```

```
---
description: Summarizes uncommitted changes and flags anything risky.
---
## Current changes
!`git diff HEAD`

## Instructions
Summarize the changes above in two or three bullet points,
then call out anything that looks risky to ship.
```

For several commands at once, open a fenced block with ````!` and put one command per line. The `!` is recognized only at the start of a line or right after

whitespace, so a stray exclamation mark inside a sentence is safe. On Windows you can route these blocks through PowerShell by setting `shell: powershell` in the frontmatter.

GOTCHA

Shell injection runs on your machine with your permissions, the instant the command expands. A command file committed to a shared repo can carry a `!`...`` block that executes when a teammate runs it. Read a command before you trust it, the same way you would read a CI script. Organizations can turn the feature off entirely with a settings toggle, which replaces every shell block with a disabled-by-policy notice.

The third feature is the `@` prefix. Writing `@package.json` in the body inlines that file's contents at the point where you wrote it, so the command carries the actual file into the prompt.

```
.claude/commands/review-config.md

---
description: Review configuration files
---
Review the following configuration files for issues:
- Package config: @package.json
- TypeScript config: @tsconfig.json
- Environment config: @.env
```

GOTCHA

The `@file` reference inside a command body is documented for the legacy command format. The current Skills reference leans on `!`command`` injection and does not itself demonstrate `@` inside a skill file. The two formats share one rendering engine, so `@file` should work in both. Verify it in a live session before you depend on it in a skill, and reach for `!`cat package.json`` if you want a guaranteed inline.

QUICK REFERENCE · THE COMMAND SYNTAX YOU WILL USE

Syntax	Meaning
<code>\$ARGUMENTS</code>	The full argument string exactly as typed.

SLASH COMMANDS & MASTER COMMANDS

<code>\$0</code> , <code>\$1</code>	First and second arguments, shell-quoted.
<code>!`cmd`</code>	Runs <code>cmd</code> before Claude sees the file; output replaces the placeholder.
<code>```\n!`</code> block	Multi-line shell injection, one command per line.
<code>@path</code>	Inlines the contents of a file at that spot in the body.
<code>description:</code>	Frontmatter. The one-line summary shown in the <code>/</code> menu.
<code>allowed-tools:</code>	Frontmatter. Tools pre-approved for the turn that runs the command.

12.4 Master commands: a whole workflow behind one slash

Everything so far builds to this. A master command is a single file that bundles an entire procedure: it gathers live context with shell injection, pre-approves the exact tools it needs, and lays out numbered steps for Claude to follow. You stop supervising a multi-step task and start invoking it. The whole ritual collapses into one word.

The frontmatter field that makes this safe is `allowed-tools`. It pre-approves a specific, narrow set of tools for the single turn that runs the command, so Claude does not stop to ask permission mid-flow. The approval clears on your next message, which keeps a broad grant from leaking into the rest of the session. Scope it tightly, using rule syntax like `Bash(git add *)` so only the operations the command actually performs are waved through.

▶ WORKED EXAMPLE · A ONE-WORD COMMIT COMMAND

The most re-typed task in most projects is the git commit: check the status, read the diff, write a sensible message, stage, commit. A master command does the whole thing. The shell blocks gather the current state into the prompt, the `allowed-tools` list pre-approves exactly the three git operations involved and nothing else, and the body tells Claude what to produce.

I .claude/commands/git-commit.md

```
---
allowed-tools: Bash(git add *), Bash(git status *), Bash(git commit
*)
description: Create a git commit
---

## Context
- Current status: !`git status`
- Current diff: !`git diff HEAD`

## Task
Create a git commit with an appropriate message based on the
changes.
```

Type `/git-commit` and the status and diff are already in the prompt, the git tools are already approved, and Claude writes the message and makes the commit in one pass. The command is committed to the repo, so every engineer on the team commits the same way. You encoded a convention into a file, and the file enforces it by making the right path the easy one.

I TIP

Build a master command by promoting a session you already ran well. When a multi-step task goes smoothly, look at the prompts and approvals it took, then bottle that sequence into a command file. The best master commands are recordings of a workflow that already worked, not speculative designs.

I TIP

Keep `allowed-tools` as narrow as the task allows. A commit command needs three git operations, so grant three. A grant of `Bash(*)` pre-approves every shell command the turn might invent, which hands away the very oversight the permission system exists to give you (Chapter 11). Precision here is a security posture, not a formality.

I UNDER THE HOOD

Claude can invoke your commands on its own, not only when you type them. The mechanism is the `Skill` tool, which executes a command or skill inside the main conversation, and it is governed by the same permission rules as anything else, written as `Skill(git-commit)` for an exact match. If a command should be yours to fire and never triggered automatically, set `disable-model-invocation: true` in its frontmatter. That both keeps Claude from auto-running it and keeps it out of subagents and scheduled tasks.

12.5 The road to Skills

There is one architectural fact you need before the next chapter, because it changes how you should think about everything above. In current Claude Code, custom commands were merged into Skills. The docs state it plainly: a file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and run the same way.

Your `.claude/commands/*.md` files keep working exactly as this chapter describes. That format is the legacy path, fully supported, and it is the right place to start because a single Markdown file is the least there is to learn. A skill is the same idea with more room: its own directory for supporting scripts and templates, richer frontmatter, and the ability for Claude to load it automatically when a task calls for it instead of waiting for you to type the slash.

KEY TERM

Skill the current, recommended form of a custom command. It is a folder containing a `SKILL.md` plus any files the procedure needs, so it can carry code and assets a single command file cannot. When a skill and a command share a name, the skill wins.

The practical guidance is a clean split. Reach for a `.claude/commands/` file when the whole procedure fits in one prompt with some shell injection and a few pre-approved tools, which is most of what you will ever automate. Reach for a skill when the procedure needs bundled scripts, wants to run in its own

subagent, or should trigger itself when relevant. Chapter 13 is that chapter, and everything you learned here about arguments, shell injection, file references, and `allowed-tools` carries straight into it.

CHAPTER 12 · CHECKLIST

You can now

- ✓ Name the built-in commands that steer a session, and use `/help` as the live index instead of a memorized list.
- ✓ Author a custom command as a single Markdown file, and place it at project or personal scope to control who gets it.
- ✓ Pass input into a command with `$ARGUMENTS` and indexed placeholders, and hint the autocomplete.
- ✓ Pull live context into a prompt with `!`shell`` injection and inline files with `@path`.
- ✓ Pre-approve exactly the tools a command needs with `allowed-tools`, scoped tight enough to stay safe.
- ✓ Build a master command that bundles a full workflow, like a one-word git commit, behind a single slash.
- ✓ Explain how commands merged into Skills, and decide when a plain command file is enough and when to reach for a skill.

Chapter Thirteen

Skills in Depth

How to author a real skill from scratch and chain several of them into a workflow that runs the same way every single time.

Chapter 4 gave you the tourist's view of skills: a folder Claude loads on demand, a tidy way to stop pasting the same checklist into chat for the tenth time. That view is correct, and it is enough to make a first skill useful. This chapter is the resident's view. You are going to open the folder, read every line of the file inside it, and learn which frontmatter switch controls which behavior.

The reason to go this deep is leverage. A CLAUDE.md rule sits in context all session and costs you tokens whether you use it or not (Chapter 9). A skill's body loads only when it fires, so a long, precise procedure costs almost nothing until the moment you need it. That single economic property is what lets you build a shelf of heavy, specialized skills without drowning every session in context.

We will cover the anatomy of a skill folder, every frontmatter field and what it controls, how invocation is triggered and scoped, the `skill-creator` tool that scaffolds and grades skills for you, and the real prize: stringing several skills into one repeatable workflow. We close with the question this whole manual keeps returning to. Given a plain prompt, a custom command, a single skill, and a chained workflow, which do you actually reach for?

13.1 The anatomy of a skill

A skill is a folder, and the only required thing inside it is a file named `SKILL.md`. Everything else is optional supporting material: a template for Claude to fill in, example outputs that show the expected format, scripts Claude can execute. The folder is the unit. The file is the brain.

my-skill/ (folder layout)

```
my-skill/
├─ SKILL.md           # Main instructions (required)
├─ template.md        # Template for Claude to fill in (optional)
├─ examples/
│   └─ sample.md      # Example output showing expected format
                        (optional)
└─ scripts/
    └─ validate.sh    # Script Claude can execute (optional)
```

Claude does not automatically read the supporting files. You reference them from inside `SKILL.md` so Claude knows what each one contains and when to load it. This keeps the main file short and pushes bulky reference material into files that load only when the task calls for them.

SKILL.md (referencing supporting files)

```
## Additional resources
- For complete API details, see [reference.md](reference.md)
- For usage examples, see [examples.md](examples.md)
```

The `SKILL.md` file itself has two parts. A block of YAML frontmatter between two `---` fences carries the metadata: the name, the description, the controls we cover in the next section. Below the closing fence is plain Markdown, and that is the instruction set Claude follows when the skill fires. Here is a complete, working skill, the kind you would keep in your personal skills folder.

```

~/.claude/skills/codebase-visualizer/SKILL.md

---
name: codebase-visualizer
description: Generate an interactive collapsible tree visualization of
your
  codebase. Use when exploring a new repo, understanding project
  structure,
  or identifying large files.
allowed-tools: Bash(python3 *)
---

# Codebase Visualizer

Generate an interactive HTML tree view that shows your project's file
structure with collapsible directories.

## Usage

Run the visualization script from your project root:

python3 ${CLAUDE_SKILL_DIR}/scripts/visualize.py .

This creates codebase-map.html in the current directory and opens it in
your default browser.

```

Read that file top to bottom and the design becomes clear. The frontmatter tells Claude what the skill is for and pre-approves the one tool it needs. The body tells Claude exactly what to do when it runs. The `${CLAUDE_SKILL_DIR}` token resolves to the folder this file lives in, so the script runs correctly no matter which directory you launched Claude from. One folder, one file, one job done the same way every time.

I TIP

Keep `SKILL.md` lean, on the order of a few hundred lines at most. The moment a skill fires, its whole body enters the conversation and stays there for the rest of the session, so every line is a recurring token cost. Write "what to do," not "why." Push long reference tables and examples into separate files that load only when the task reaches for them. The current line-count guidance is in the skills docs; treat brevity as the goal regardless of the exact number.

13.2 The frontmatter, field by field

Every field in the frontmatter is optional. A skill with nothing but a body will still run when you type its name. In practice one field does most of the work and the rest are precision controls you add as a skill grows up.

The one field that earns its keep is `description`. This is the text Claude reads to decide whether a skill is relevant to what you just asked. It is the trigger. Write it as "what this does plus when to use it," and put the primary use case first, because the listing that holds every skill's description gets truncated past a fixed character budget. A vague description is the single most common reason a good skill never fires when it should.

KEY TERM

description the trigger-matching text Claude reads to decide when a skill applies. If you omit it, Claude Code falls back to the first paragraph of the body. It is the highest-leverage line in the whole file, so write it deliberately.

Below the description sits a set of controls that fall into four natural groups: identity, invocation, tools, and execution. The table gives you the whole surface on one page. You will reach for four or five of these regularly and leave the rest at their defaults.

QUICK REFERENCE · SKILL.MD FRONTMATTER FIELDS

Field	What it controls
-------	------------------

SKILLS IN DEPTH

<code>name</code>	Display name in listings. In personal and project skills the invoked command name still comes from the directory, so this is cosmetic there.
<code>description</code>	The trigger text. What it does plus when to use it. Recommended on every skill.
<code>when_to_use</code>	Extra trigger phrases and example requests, appended to the description and counted against the same budget.
<code>argument-hint</code>	Autocomplete hint shown in the menu, such as <code>[issue-number]</code> .
<code>arguments</code>	Named positional arguments for <code>\$name</code> substitution in the body.
<code>disable-model-invocation</code>	<code>true</code> means only you can run it by name; Claude cannot trigger it on its own, and its description leaves Claude's context entirely.
<code>user-invocable</code>	<code>false</code> hides it from the <code>/</code> menu while Claude can still invoke it. For background knowledge, not commands.
<code>allowed-tools</code>	Tools pre-approved without a permission prompt, for the invoking turn only.
<code>disallowed-tools</code>	Tools removed from Claude's pool while the skill is active, cleared on your next message.
<code>model</code> / <code>effort</code>	Override the model or reasoning effort for the rest of the current turn only. Not saved.
<code>context</code> / <code>agent</code> / <code>background</code>	Run the skill in a forked subagent, choose which agent type, and decide whether it blocks. See 13.3.
<code>paths</code>	Glob patterns that limit auto-activation to matching files.
<code>hooks</code> / <code>shell</code>	Hooks scoped to this skill's lifecycle (Chapter 14), and which shell runs its inline commands.

Two of these are worth internalizing now because they answer a real daily question: who is allowed to fire this skill? The next section pulls them apart.

GOTCHA

If the YAML frontmatter is malformed, Claude Code loads the body with empty metadata. Your skill still runs when you type `/its-name` by hand, so it looks fine to you. Claude, though, now has no `description` to match against and will never trigger it automatically. When a skill mysteriously refuses to auto-fire, suspect a broken frontmatter fence or a bad indent first, and launch with `--debug` to see the parse error.

13.3 Invocation control: who fires it, and where it runs

A skill can be triggered two ways. You invoke it explicitly by typing `/skill-name`, or Claude invokes it on its own when your request matches the skill's description. Three frontmatter settings decide which of those paths are open, and they map cleanly to the two jobs skills do: commands you fire deliberately, and knowledge Claude reaches for silently.

QUICK REFERENCE · INVOCATION CONTROL

Frontmatter	You invoke	Claude invokes	Description in context
(default)	Yes	Yes	Always. Full body loads on invoke.
<code>disable-model-invocation: true</code>	Yes	No	No. Body loads only when you invoke.
<code>user-invocable: false</code>	No	Yes	Always. Full body loads on invoke.

Set `disable-model-invocation: true` on anything with side effects or timing you want to own: a `/commit`, a `/deploy`, a skill that posts to Slack. You want your finger on that trigger, not the model's. Set `user-invocable: false` for the opposite case, a skill that is really background knowledge Claude should consult but that is not a command a human would ever type, like a `legacy-system-context` reference.

SKILLS IN DEPTH

Triggering is only half of invocation. The other half is where the skill runs. By default a skill runs inline, in your conversation, with full access to everything already in context. Add `context: fork` and it runs somewhere else entirely: a fresh subagent with its own clean context and no view of your conversation history. The skill body becomes that subagent's entire prompt.

```
~/claude/skills/deep-research/SKILL.md

---
name: deep-research
description: Research a topic thoroughly across the codebase
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:
1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Summarize findings with specific file references
```

A forked skill does its work in the background by default, so you keep typing while it runs and the result arrives when it is done. The `agent` field picks which subagent type does the work. The built-in `Explore` and `Plan` agents deliberately skip your `CLAUDE.md` and git status to stay lean, which means a forked skill running on `Explore` sees only its own body plus that agent's system prompt. That isolation is the point. It keeps a big research sweep from polluting your main context with a hundred file reads.

! GOTCHA

`context: fork` only pays off when the skill body contains an actual task, a set of steps to carry out. A skill that holds nothing but reference material ("use these API conventions") gives the forked subagent no instruction to act on, so it spins up, finds nothing to do, and returns nothing useful. Fork skills that *do* something. Keep pure reference knowledge inline where the main model can consult it.

I UNDER THE HOOD

A background fork's file edits land **outside** your session checkpoints, which means `/rewind` will not undo them. If a forked skill writes to disk and you need to roll it back, reach for git, not rewind. A few situations also force a fork to run inline and block regardless of the background setting: non-interactive `-p` and SDK runs, a second invocation of the same skill while the first is still going, and a scheduled task that fires the skill as its prompt.

You can also let the file path decide when a skill wakes up. A `paths` header limits auto-activation to matching files, so a skill full of API rules stays dormant until Claude actually touches an API file. That is the same path-scoping idea you met with rules files in Chapter 9, applied to whole skills.

13.4 Scaffolding with the skill-creator

You can write a `SKILL.md` by hand, and for a three-line skill you should. Past that, Anthropic ships an official tool that scaffolds the folder, drafts the frontmatter, and, more valuably, measures whether the skill actually works. It is a plugin called `skill-creator`.

I Install the skill-creator plugin

```
# add the official marketplace once, if you have not already
/plugin marketplace add anthropics/claude-plugins-official
/reload-plugins

# then install the creator
/plugin install skill-creator@claude-plugins-official
```

At the beginner level you use it to lay down a correct folder and a first draft. At the advanced level you use it for the thing that separates a skill that triggers from a skill that works, which are two different problems. A skill can fire perfectly on the right prompt and still produce a mediocre answer, and the only way to know is to measure both separately.

The creator automates that measurement. It keeps a set of test cases, runs each one in an isolated subagent so results do not contaminate each other,

grades the output against assertions you write, and aggregates the pass rate, time, and token cost into a report. It runs blind A/B comparisons between two versions of a skill so you can see whether an edit helped. And it auto-tunes the `description` text against a list of prompts that *should* trigger the skill and a list that should *not*, which is exactly the tuning that is miserable to do by hand.

I TIP

Measure triggering and correctness with the same technique: baseline comparison. Run the same prompts twice from a fresh session, once with the skill enabled and once with it off, and read the difference. Toggling a skill off for a test does not require deleting it; the `/skills` menu lets you flip its visibility, so you can A/B without touching the file. If enabling the skill does not change the output, the skill is not earning its context cost yet.

13.5 Stringing skills into a repeatable workflow

One skill does one job well. The real operator move is composing several so that a multi-step process runs the same way every time, no matter who kicks it off or how tired they are. There are two ways to compose, and a good workflow uses both.

The first is stacking. Claude Code lets you invoke several skills at the start of a single message, and each one loads its instructions into the same turn. You are assembling a briefing on the fly: load the coding standards, load the review checklist, load the deploy procedure, then state the task. All of that guidance is now in context together for the work that follows.

The second is orchestration through one parent skill. You write a single skill whose body *is* the running order, and it invokes the others in sequence, so no human has to remember which step comes next. That parent skill becomes the one command a person has to remember, and the steps inside it never drift.

▶ WORKED EXAMPLE · A SHIP-IT WORKFLOW FROM THREE SKILLS

Say every release in your project follows the same three beats. Run the tests, write a conventional commit message from the diff, and draft the release note.

memory in the right order, which means one distracted afternoon you will run them in the wrong order or skip the middle one.

Promote the sequence into a parent skill. Its body is nothing more than the ordered steps, written as standing instructions that name each child skill. Because the whole procedure lives in one file, the order is fixed and the workflow is now a single command: `/ship`.

```
.claude/skills/ship/SKILL.md

---
name: ship
description: Run the full release routine: test, commit, and draft
  the release note. Use when the user says they are ready to ship.
disable-model-invocation: true
---

# Release routine

Carry out these steps in order. Stop and report if any step fails.

1. Run the project test suite. If anything fails, stop here and
  show me the failures. Do not continue.
2. Invoke the /commit skill to stage the changes and write a
  conventional commit message from the diff.
3. Invoke the /release-note skill to draft a short note describing
  what changed for the changelog.
4. Show me the commit message and the draft note. Wait for my
  approval before pushing anything.
```

Two design choices make this robust. The `disable-model-invocation: true` line means only a human starts a release, never the model on a hunch. And the explicit "stop if a step fails" instruction gives the workflow a guard rail, so a red test suite halts the train instead of committing broken code. Now anyone on the team ships the same way. They type `/ship` and follow the prompts.

! GOTCHA

Skills are context, and context is guidance the model usually follows, the same caveat we made about memory in Chapter 9. A parent skill that lists four steps is a strong nudge, and it is still a nudge. If a step in your workflow **must** happen or must never happen, do not trust the ordering in a skill body to guarantee it. Enforce that step with a hook (Chapter 14), which the harness runs deterministically. Use skills to make the routine easy and consistent, and hooks to make the non-negotiable parts actually non-negotiable.

I TIP

Give each step in a workflow a clear failure instruction, like the "stop and report" lines above. A skill body is read as standing instructions for the rest of the session, so Claude does not re-read the file between steps. Spell out what to do when something breaks inside the same body, because there is no second chance to add the guard rail once the workflow is running.

13.6 What to use when

By now you have four tools that all shape how Claude behaves: a plain prompt, a custom command from Chapter 12, a single skill, and a chained workflow. They overlap enough to be confusing and differ enough to matter. Here is the decision, stated plainly.

Reach for a **plain prompt** when the task is a one-off. If you will not do this exact thing again next week, typing it out is faster than building anything. The moment you find yourself pasting the same instructions a second and third time, that is the signal to promote it.

Reach for a **custom command** or a **single skill** when a procedure has stabilized into something you repeat. The two are close cousins now that commands and skills share the same machinery. A skill is the richer form: it can carry supporting files, invocation-control frontmatter, and the ability for Claude to load it on its own when a request matches. Use a bare command for a simple, self-invoked shortcut, and a skill when you want bundled scripts, auto-triggering, or forked execution.

Reach for a **chained workflow** when the job has several steps that must run in a fixed order and you want the same result every time, regardless of who runs it. That is the `/ship` pattern from the last section: one command a human remembers, wrapping a sequence that does not drift.

QUICK REFERENCE · WHICH TOOL FOR THE JOB

Tool	Reach for it when	Cost and reach
Plain prompt	A genuine one-off you will not repeat.	Zero setup. Nothing persists.
Custom command	A simple repeated shortcut you fire yourself.	One Markdown file. You invoke by name.
Single skill	A repeated procedure that wants bundled files, auto-triggering, or a forked run.	A folder. Body loads only on invoke; Claude can fire it too.
Chained workflow	A multi-step job that must run the same way every time.	A parent skill invoking children. One command, fixed order.

The through-line across all four is deferred cost. A prompt costs you nothing to keep because it does not stick around. A skill costs you nothing until it fires, which is why you can afford a whole shelf of them. Build the shelf deliberately, keep each skill's description sharp so the right one triggers, and let the workflow skills tie the routine ones together into something you run without thinking.

CHAPTER 13 · CHECKLIST

You can now

- ✓ Lay out a skill folder and write a complete `SKILL.md` with working frontmatter and a body.
- ✓ Name the frontmatter fields that matter and say what each one controls, starting with `description`.
- ✓ Control who fires a skill with `disable-model-invocation` and `user-invocable`.
- ✓ Run a skill in an isolated subagent with `context: fork`, and know when a fork is the wrong choice.
- ✓ Use the `skill-creator` plugin to scaffold, grade, and A/B test a skill's triggering and output.
- ✓ Compose several skills into one repeatable workflow with a parent skill that fixes the running order.
- ✓ Choose correctly among a plain prompt, a custom command, a single skill, and a chained workflow.

Chapter Fourteen

Hooks & Hard Rules

How to turn a behavioral request Claude might follow into a deterministic rule the harness enforces every single time.

Back in Chapter 9 we drew a hard line: memory is context, not enforcement. Your CLAUDE.md is a brief the model reads and usually respects, the way a careful colleague respects a well-written brief. Usually is the operative word. Write "never touch the production config" in bold at the top of the file, and on a long enough timeline Claude will still, one distracted afternoon, propose an edit to it. Context shapes behavior. It does not guarantee it.

Hooks are the guarantee. A hook is a command the Claude Code harness runs by itself at a fixed point in the session, on a schedule you define, with zero input from the model. The model does not choose to run it and cannot choose to skip it. When a hook says no, the action stops, full stop, in every permission mode. This is the layer where a preference becomes a law.

This chapter covers the mechanism end to end: the lifecycle events a hook can fire on, the matchers that scope it to specific tools, the exit codes and JSON that let it allow or block an action, and the handler types that do the actual work. Then three copy-exact recipes you can paste today, and the handful of ways hooks bite the people who deploy them carelessly.

14.1 Context versus enforcement

Everything in Part III so far has been advisory. CLAUDE.md, skills, rules files: all of them are text Claude reads and weighs against everything else in its context window. That design is correct for guidance. It is the wrong tool the moment "should not" becomes "must not."

KEY TERM

Hook a user-defined command that Claude Code's harness executes automatically at a specific lifecycle event, such as before a tool runs or after an edit completes. The harness runs it, not the model, so it fires deterministically whenever its conditions are met.

Hold the two mechanisms side by side. A CLAUDE.md rule is read once at launch and competes for the model's attention with everything else you loaded. A hook sits outside the model entirely, in the harness, and executes on its trigger with the reliability of any other line of code. One is a request. The other is a gate.

You reach for a hook in exactly three situations. When you need to **block** an action Claude cannot be talked out of, like an edit to `.env` or an `rm -rf`. When you want to **automate** a side effect that should happen every time without being asked, like formatting a file the instant it is written. And when you want to **inject** fresh information at a fixed moment, like re-stating a deploy freeze at the top of every prompt. Guidance that merely needs to be seen belongs in memory. Rules that must hold belong in a hook.

14.2 The event model

A hook is bound to a lifecycle event. The event is the "when." Claude Code exposes a long and growing list of them across the whole arc of a session, from the moment it starts to the moment it ends, wrapped around every tool call in between. You do not need all of them. A handful carry the vast majority of real work, and they are the durable ones worth memorizing.

QUICK REFERENCE · THE HOOK EVENTS YOU WILL ACTUALLY USE

Event	Fires	Can it block?
<code>SessionStart</code>	When a session begins or resumes	No. Context injection only
<code>UserPromptSubmit</code>	When you submit a prompt, before Claude sees it	Yes
<code>PreToolUse</code>	Before a tool call executes	Yes. Can deny the call
<code>PostToolUse</code>	After a tool call succeeds	Yes. Cannot undo the action
<code>Notification</code>	When Claude Code sends a notification, such as an idle or permission prompt	No
<code>PreCompact</code> / <code>PostCompact</code>	Before and after context compaction	No
<code>Stop</code>	When Claude finishes responding	Yes. Block to make it keep going
<code>SessionEnd</code>	When the session terminates	No

I TIP

The full event catalog runs past twenty entries and covers subagents, worktrees, MCP elicitation, config changes, and more. It is also clearly still expanding release to release. When you need an event outside the core set above, read the current hooks reference in the docs. Do not trust a list you memorized last quarter. Event names and their exact capabilities are the kind of detail that shifts between builds.

The event answers "when." A **matcher** answers "for what." On the tool events, the matcher is a string tested against the tool name, so a `PreToolUse` hook with matcher `"Bash"` fires only before Bash calls and ignores every edit. An omitted matcher, an empty string, or `"*"` matches everything. A plain value like `Bash` is an exact match, and `Edit|Write` matches either tool. Anything with characters beyond letters, digits, and a few separators is treated as a JavaScript regex, so `mcp__memory__.*` scopes a hook to every tool on one MCP server. That trailing `.*` is required, not decorative.

HOOKS & HARD RULES

The event fires, the matcher narrows, and then your hook gets to decide. The decision travels back to the harness through the hook's **exit code**. This is the single most important mechanic in the chapter, and the one people get wrong.

QUICK REFERENCE · WHAT A HOOK'S EXIT CODE MEANS

Exit code	Effect
0	Success. No objection, the action proceeds. On <code>PreToolUse</code> this does not itself approve the call; the normal permission flow still runs.
2	Blocking. The action is stopped and whatever your script wrote to stderr is fed back to Claude as feedback. On events that cannot block, exit 2 just surfaces stderr to you and execution continues.
Anything else	Non-blocking error. The transcript shows a hook error and the first line of stderr, then execution continues anyway.

 GOTCHA

Exit 1 is **not** the same as exit 2. This is the most common hook-authoring mistake there is. A script that means to block but returns `exit 1` does not block anything. It logs an error, the action goes through, and you find out the hard way. If a hook is meant to stop something, it must exit with code **2**, and nothing else.

For anything richer than a yes or no, a hook can instead exit 0 and print structured JSON to stdout. That JSON is where a `PreToolUse` hook returns a formal `permissionDecision` of `allow`, `deny`, or `ask`, with a reason string that Claude reads. Do not mix the two channels. Claude Code ignores your JSON entirely when you exit 2, so pick one: exit 2 with a stderr message for a blunt block, or exit 0 with JSON for structured control. Never both.

 UNDER THE HOOD

All hooks matching an event run **in parallel**, and identical commands are deduplicated. When several `PreToolUse` hooks weigh in on the same call, the most restrictive decision wins, ordered `deny` over `defer` over `ask` over `allow`. Any context text each hook returns is kept and combined. One guardrail cannot be overruled by a more permissive sibling, which is exactly the property you want from a safety layer.

14.3 Handlers and where they live

The event and matcher decide when a hook runs. The **handler** is what runs. The workhorse handler type is `command`, which executes a shell command or script. Four others exist for specialized needs: `http` POSTs the event to a URL, `mcp_tool` calls a tool on a connected MCP server, `prompt` asks a small model a yes-or-no question, and `agent` spins up a subagent that can inspect files first. The last two are experimental. For anything you depend on, use a `command` handler.

Hooks are configured in your settings JSON, under a single top-level `hooks` object. Each event name is a key, and its value is an array of matcher groups. Each group pairs a matcher with a list of handlers. The shape is easier to read than to describe.

```
.claude/settings.json

{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/check.sh"
          }
        ]
      }
    ]
  }
}
```

Where you put that block decides who it applies to, and the scopes mirror the memory hierarchy from Chapter 9. A hook in `~/.claude/settings.json` follows you across every project on the machine. One in `.claude/settings.json` is committed to the repo and shared with the team. One in `.claude/settings.local.json` is gitignored and yours alone. Managed

policy settings apply organization-wide and are set by an admin. Plugins and skills can bundle their own hooks that are live only while that component is active.

***I* TIP**

The `/hooks` command opens a read-only browser of every hook currently in play: the event, the matcher, the handler, and the source scope it came from. It is the fastest way to answer "why did that just get blocked" and to confirm a committed project hook is actually loaded. To add or change a hook, edit the settings JSON directly. The `/hooks` view only shows you what exists.

Command handlers come in two forms. Supply an `args` array and the command runs in exec form, spawned directly with no shell and no quoting worries. Omit `args` and it runs in shell form, where pipes, `&&`, and globs all work, at the cost of having to quote your path placeholders. Both forms can reference `${CLAUDE_PROJECT_DIR}` to locate your scripts no matter where the session started.

***!* GOTCHA**

Command hooks default to a generous timeout, on the order of ten minutes, while a couple of events cap much lower: a `UserPromptSubmit` hook is held to about thirty seconds because you are waiting on it before every prompt. A slow validation script parked on a fast event will make Claude feel broken. Keep hooks that run on the hot path quick, and verify the current timeout numbers in the docs before you rely on a specific one. They vary by event.

14.4 Recipe A: a hard gate that blocks a dangerous edit

Here is the canonical use. You have files that Claude must never modify, and you want a real gate, one that holds even when the request is ignored. A `PreToolUse` hook on the edit tools inspects the target path and exits 2 the moment it sees a protected file. Because `PreToolUse` fires before any permission check, this gate holds in every mode.

► WORKED EXAMPLE · BLOCK EDITS TO PROTECTED FILES

The hook reads the tool call as JSON on stdin, pulls the target path with `jq`, and checks it against a list of forbidden patterns. A match writes a reason to stderr and exits 2, which cancels the edit and hands the reason back to Claude.

```
.claude/hooks/protect-files.sh

#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

PROTECTED_PATTERNS=( ".env" "package-lock.json" ".git/" )

for pattern in "${PROTECTED_PATTERNS[@]}; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$pattern'"
    >&2
    exit 2
  fi
done

exit 0
```

Make it executable with `chmod +x .claude/hooks/protect-files.sh`, then wire it to the edit tools. The matcher `Edit|Write` covers both the surgical and the whole-file writers.

```
.claude/settings.json

{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command":
"\${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-files.sh" }
      ] }
    ]
  }
}
```

From now on any attempt to edit `.env` dies before it starts, and Claude reads the reason and moves on. No permission mode, not even bypass, gets around it.

14.5 Recipe B: an auto-formatter that runs after every edit

The second classic use is the automatic side effect. You want every file Claude writes to be formatted, immediately, without asking. A `PostToolUse` hook on the edit tools does it. It fires after the write succeeds, reads the path, and runs your formatter over it. This one needs no separate script, because a shell-form command handler can pipe straight through.

► DO THIS · FORMAT ON WRITE WITH PRETTIER

- 1 Add a `PostToolUse` group for the edit tools. The matcher is the same `Edit|Write` pair, so it runs after either kind of write.
- 2 Pipe the path to your formatter. Omitting `args` gives you shell form, so the pipe and `xargs` work as written.
- 3 Trust it to run every time. There is no model decision involved. The edit lands, the formatter runs, the file is clean.

`.claude/settings.json`

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": "jq -r '.tool_input.file_path'
| xargs npx prettier --write" }
      ] }
    ]
  }
}
```

! GOTCHA

A `PostToolUse` hook **cannot undo** the action it observes. By the time it runs, the write has already happened. It is a place to react to a change, not to veto one. Anything that must prevent an action has to live on `PreToolUse`, before the tool executes. Reserve `PostToolUse` for formatting, linting, logging, and other cleanup that assumes the change is already real.

14.6 Recipe C: a context injector for live state

The third use is deterministic context. A CLAUDE.md line is static, written once and read at launch. Some facts change by the minute: which git branch you are on, whether a deploy freeze is active, what the current sprint is. A `UserPromptSubmit` hook computes that fresh on every prompt and feeds it to Claude before the model starts thinking. It injects context by printing JSON with a nested `additionalContext` field.

▶ WORKED EXAMPLE · INJECT THE LIVE BRANCH INTO EVERY PROMPT

The script reads the current branch, then emits the required JSON with `jq`. Claude sees the added line at the top of your prompt, every time, with the value correct as of that second.

```
.claude/hooks/inject-branch.sh

#!/bin/bash
BRANCH=$(git branch --show-current 2>/dev/null || echo "unknown")
jq -n --arg branch "$BRANCH" '{
  hookSpecificOutput: {
    hookEventName: "UserPromptSubmit",
    additionalContext: ("Current branch: " + $branch + ". Verify
before pushing to main.")
  }
}'
```

Wire it to `UserPromptSubmit`. That event takes no matcher, so it always fires, on every prompt you send.

```
.claude/settings.json

{
  "hooks": {
    "UserPromptSubmit": [
      { "hooks": [ { "type": "command", "command":
        "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/inject-branch.sh" } ] }
    ]
  }
}
```

The `additionalContext` field has to sit inside `hookSpecificOutput`. Placed at the top level of the JSON it is silently ignored, and you will spend an afternoon wondering why your context never arrives. The same pattern works on `SessionStart` with a `compact` matcher, which is the standard way to re-inject core conventions right after a long session compacts and threatens to forget them.

TIP

Injected text and other hook output are capped at around ten thousand characters, with the overflow written to a file. That is plenty for a branch name or a freeze notice, and far too little to dump a whole changelog. Keep injected context to the few facts that actually change the model's next move.

14.7 Where hooks bite

Hooks are code that runs itself, which makes them powerful and makes them dangerous in the same breath. Three failure modes account for most of the pain, and all three are avoidable once you have seen them.

The first is scoping a hook too broadly. A matcher of `"*"` on `PreToolUse` runs your script before every tool call in the session, including the dozens of harmless reads and searches. If that script is slow or occasionally exits 2 by mistake, you have quietly kneecapped the whole session. Match the narrowest set of tools that achieves the goal. A file guard belongs on `Edit|Write`, not on everything.

! GOTCHA

Hooks execute **arbitrary shell commands with your full user permissions** the instant their conditions are met. A malicious or careless hook committed into a shared repo's `.claude/settings.json` is a supply-chain risk, the same class of thing as a CI script. Read any hook before you trust it, whether you wrote it or pulled it. Project and local scope hooks require you to accept the workspace trust dialog first, which is a gate, not a guarantee.

The second is the exit-code trap from earlier, worth stating twice because it costs so much time. A block is exit code 2 and only exit code 2. Every other non-zero code is a non-blocking error that logs and lets the action through. If your gate is not gating, check its exit code before you check anything else.

! GOTCHA

Hooks can **tighten but never loosen**. A `PreToolUse` hook returning `deny` cannot be overridden by switching permission mode, which is precisely why it is trustworthy. The reverse does not hold. A hook returning `allow` does not defeat a `deny` rule in your permission settings, and cannot silence prompts your organization marked as required. Treat hooks as a way to add restrictions on top of the permission system, never as a way to punch holes through it.

The third is a quieter one that surfaces on shell-form command hooks. When Claude Code spawns a shell to run your command, that shell may source your profile. An unconditional `echo` in your `.bashrc` or `.zshrc` then prepends its output to your hook's stdout, corrupting any JSON you meant to emit and producing baffling parse failures. Guard profile output so it only runs in interactive shells, and the problem disappears.

I TIP

For hard security, lean on the permission system as the primary control and use hooks to reinforce it, not to replace it. The best-effort argument filters that let a hook peek at a command's contents are documented as fail-open on purpose, so a determined command can slip past them. A permission `deny` rule plus a `PreToolUse` guard is a belt-and-suspenders setup that holds where either alone might not.

CHAPTER 14 · CHECKLIST**You can now**

- ✓ Explain why a hook enforces a rule that a CLAUDE.md line only requests, and choose the right tool for each.
- ✓ Name the core lifecycle events and say which can block an action and which only observe or inject.
- ✓ Scope a hook to specific tools with a matcher, and know when a value is treated as a regex.
- ✓ Use exit code 2 to block and exit 0 with JSON for structured control, without ever confusing 1 for 2.
- ✓ Configure a `command` handler in the right settings scope, from user-wide down to a gitignored local file.
- ✓ Build a `PreToolUse` hard gate, a `PostToolUse` auto-formatter, and a `UserPromptSubmit` context injector from copy-exact recipes.
- ✓ Avoid the three ways hooks bite: over-broad matchers, the exit-code trap, and trusting an unreviewed hook to run arbitrary shell.

Chapter Fifteen

Subagents

How to hand a scoped job to a fresh, focused helper agent, keep the noise out of your main context, and fan several of them out to work at once.

A long task makes a mess of your conversation. Grep across a large repo, read twenty log files, run a failing test suite, and your main context fills with output you will never look at again. Every token of that noise sits there for the rest of the session, crowding out the things you actually care about and quietly raising the cost of every reply. There is a cleaner way to do heavy work: send it somewhere else.

A subagent is that somewhere else. It is a separate Claude Code instance, running inside your session, with its own context window, its own system prompt, its own tools, and its own model. You hand it a task. It goes off and does the messy part on its own, then returns a short summary. All the intermediate churn stays in its context and never touches yours. Your main conversation gets the answer and stays clean.

This chapter covers the whole capability: the mental model that tells you when to delegate, how to define your own reusable agent types as files, the three ways to invoke one, how to fork and resume, and how to fan several subagents out in parallel with the real limits that governs.

15.1 The mental model: a clean pair of hands

Picture a subagent as a competent colleague you can summon for one job. You give them a task description and nothing else from your desk. They have their own workspace, their own reference books, and no memory of the forty things you have been juggling all morning. They do the job, walk back, and tell you the result in a sentence or two. Everything they read and ran to get there stays on their desk, not yours.

KEY TERM

Subagent a separate Claude Code instance spawned by your main conversation to do a scoped task. It has its own context window, system prompt, tool permissions, and model. It works independently and returns only a summary, so the intermediate detail never enters your main context.

Two properties make this worth reaching for. The first is **context preservation**. A verbose task delegated to a subagent costs your main conversation only the final summary, not the hundreds of lines of file contents and command output it took to produce it. The second is **focus**. A subagent starts cold, with a single job and a system prompt written for exactly that job, so it is not weighing your task against everything else loaded in a busy session. A clean, narrow context tends to produce sharper work.

Claude Code ships three built-in subagents you already meet elsewhere in this manual: Explore for read-only searching, Plan for designing an approach, and general-purpose for open-ended multi-step work. You can also define your own, which is where the real leverage lives.

So when do you actually delegate, and when do you just do the work inline? The line is about volume and independence. Delegate when a subtask will generate a lot of output you will not reference again (log triage, a wide search, a test run), when it is genuinely independent and could run alongside other work, or when it is a repeatable job worth saving as a named specialist. Do it inline when the task is small, when you need every intermediate step visible in the main thread, or when the result depends tightly on the conversation you

are already having. A one-line grep does not need a subagent. Reviewing every changed file in a large diff does.

TIP

The single biggest lever for API spend is routing high-volume grunt work to a cheaper, faster model. Point a log-triage or test-running subagent at Haiku through its `model` field and keep your main conversation on a stronger model. You get the throughput where it is needed and the reasoning where it counts, and you pay accordingly.

15.2 Custom agent types: a saved specialist

A custom subagent is a saved configuration you invoke by name. Define a "code-reviewer" once and you can summon it in any session without re-explaining what a good review looks like. The definition is a plain Markdown file with a YAML frontmatter block on top. The frontmatter is the config. The Markdown body below it becomes the subagent's system prompt.

`.claude/agents/code-reviewer.md`

```
---
name: code-reviewer
description: Expert code review specialist. Use proactively right after
writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
---
```

You are a senior code reviewer ensuring high standards of quality and security.

When invoked:

1. Run `git diff` to see recent changes.
2. Focus on the modified files.
3. Begin the review immediately.

Organize feedback by priority: critical issues, warnings, then suggestions.

Include a specific fix for each issue you raise.

SUBAGENTS

Only two fields are required. `name` is a lowercase, hyphenated identifier that must be unique across all your agents, and `description` is the free text Claude reads to decide when to delegate automatically. The body below the frontmatter is the important part: it is the entire system prompt this subagent runs with. It does not inherit the full Claude Code system prompt, so write it to stand on its own.

The optional fields are where you shape the specialist. A few earn their keep on almost every agent you write.

QUICK REFERENCE · THE FRONTMATTER FIELDS YOU WILL REACH FOR

Field	What it does
<code>name</code>	Required. Lowercase and hyphens, unique across your agents. How you invoke it.
<code>description</code>	Required. Tells Claude when to delegate. Write it specific and action-oriented; add "use proactively" to bias toward automatic use.
<code>tools</code>	Comma-separated allowlist. Omit to inherit everything available to subagents. Restrict it to make a real permission boundary.
<code>model</code>	<code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> , a full model ID, or <code>inherit</code> . Default is <code>inherit</code> .
<code>permissionMode</code>	The permission posture the subagent runs under, such as <code>plan</code> or <code>acceptEdits</code> .
<code>memory</code>	Gives the subagent a persistent directory it can read and write across sessions. <code>project</code> scope is git-shareable.

The `tools` field is worth dwelling on, because it is a genuine constraint, not a suggestion. Give a reviewer `tools: Read, Grep, Glob` with no `Edit` or `Write` , and it physically cannot modify a file, no matter what the task text asks for. That is how you build a read-only researcher you can trust to look but never touch. Leave `tools` off entirely and the subagent inherits the full pool available to subagents, which is the right default for a general worker and the wrong one for a specialist you want boxed in.

Where the file lives decides who can use it. A file in `.claude/agents/` is scoped to the current project and can be committed so the whole team shares it. A file in `~/.claude/agents/` is user-level and follows you across every

project on the machine. Both directories are scanned recursively, so you can organize agents into subfolders without changing their identity. Only the `name` field defines who an agent is.

TIP

Claude Code watches both agent directories and picks up new or edited files within seconds, with no restart. The one exception is the very first file you add to a brand-new `agents/` directory that did not exist when the session started: the file watcher only covers folders that were present at launch, so that first agent needs a restart to be seen. Every agent after it loads live.

GOTCHA

The `/agents` command used to open an interactive wizard for creating and editing subagents. In recent Claude Code builds that wizard was removed, and running `/agents` now just reminds you to ask Claude to write the file or to edit the `agents/` directory yourself. The file format and locations are unchanged. Verify what your installed version does before you rely on a terminal UI that may no longer be there. The durable path is to ask Claude in chat, for example "create a read-only code-improver subagent in my user agents folder that uses Sonnet," and let it write the file.

15.3 Invocation: calling, forking, resuming

Once an agent exists, there are three escalating ways to put it to work, each giving you more control over which agent runs.

The lightest is **natural language**. Just name the agent in a sentence and let Claude decide whether to delegate: `Use the code-reviewer subagent to look at the auth changes`. Claude reads the agent's `description` against the task and delegates when it matches. This is also how automatic delegation works with no prompting at all, which is why a specific, action-oriented description matters so much.

The firmest is the **@-mention**. Type `@`, pick the agent from the typeahead, and that exact subagent is guaranteed to run for the task. Use this when you want a particular specialist and do not want Claude to make the call for you.

SUBAGENTS

The third pattern promotes a subagent to run the **whole session**. Launch with `claude --agent code-reviewer` and the main thread itself adopts that agent's system prompt, tools, and model, replacing the default Claude Code persona entirely. It persists across `--continue` and `--resume`. Set it as a project default with an `"agent"` key in `.claude/settings.json` when a repo should always run under one persona.

terminal

```
# Let Claude decide whether to delegate
Use the code-reviewer subagent to review the billing changes.

# Force this exact agent to run the task
@code-reviewer check the auth changes before I push.

# Make the whole session run as the agent
claude --agent code-reviewer
```

A named subagent always starts cold, with none of your conversation history. Sometimes that is exactly wrong. You have built up context over an hour and you want a helper that already knows all of it. That is a **fork**.

KEY TERM

Fork a special subagent that inherits your entire conversation so far (system prompt, tools, model, and full history) instead of starting fresh. You launch one with `/subtask`. Only its final result re-enters your context, so the main window stays clean while the fork does the detour.

Start a fork with a command like `/subtask draft unit tests for the parser changes so far`. It runs in the background by default, works with the full context you have built, and drops only its final result back into your session when it finishes. A fork cannot spawn further forks, and this command was named `/fork` in some earlier builds, so check what your version calls it.

The last move is **resuming**. Subagent transcripts persist on disk and survive even a compaction of your main conversation. That means a completed subagent is not gone. Ask Claude to continue the prior subagent's work and it reopens that agent with its full history intact, picking up exactly where it left off with no cold restart. The built-in Explore and Plan agents are one-shot and

cannot be resumed, so reach for general-purpose or a custom subagent when you know you will want to continue a line of work later.

15.4 Parallel fan-out and its limits

The property that turns subagents from a tidiness trick into a force multiplier is that several can run at once. Independent tasks do not have to wait in line. Ask Claude to research the auth module, the database layer, and the API surface, and it can spawn three subagents that each explore their area concurrently while you keep working, then synthesize the three summaries into one answer.

Subagents run in the background by default in current builds, which is what makes this feel seamless. A background subagent works alongside your main session; when it needs a permission decision, the prompt surfaces in your main thread naming which subagent is asking, and you approve it or press `Esc` to deny just that one call. A foreground subagent, by contrast, blocks your conversation until it finishes. You can push a running subagent to the background yourself with `Ctrl + B`.

Fan-out is powerful, so it comes with guardrails you should know before you lean on it. There are three separate limits, and they answer three different questions.

QUICK REFERENCE · THE THREE FAN-OUT LIMITS (DEFAULTS, ALL CONFIGURABLE)

Limit	Default	What it caps
Concurrent	20	How many subagents run at the same instant. Raise with <code>CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS</code> .
Per session	200	Total spawned in one session before it stops. Raise with <code>CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION</code> ; <code>/clear</code> resets the count.
Nesting depth	3	How many layers of subagents-spawning-subagents are allowed below the main conversation. Set with <code>CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH</code> ; <code>1</code> disables nesting.

SUBAGENTS

These numbers are exactly the kind of default that shifts between releases. The nesting depth in particular has moved more than once. Treat the table as the current shape of the mechanism, and check the subagents docs for the precise numbers your version ships with before you build a workflow that depends on a specific one.

! GOTCHA

A background subagent runs with a **reduced tool set**. The harness strips it down to a core kit of file, search, shell, and web tools plus all MCP tools, and removes the rest even if you listed them in the agent's `tools` field. Beyond that, every subagent loses the interactive tools that only make sense in the main thread, such as asking the user a question or entering plan mode. If a subagent seems to be missing a tool you granted it, background mode is the first thing to check.

I UNDER THE HOOD

The tool that spawns a subagent is called `Agent`. It was renamed from `Task` in an earlier build, and old `Task(...)` references in settings still work as aliases. This matters when you want to restrict delegation: a permission rule like `"deny": ["Agent(Explore)"]` blocks a specific agent type, and denying the bare `Agent` tool disables all subagent delegation at once. A coordinator agent can also be limited to spawning only named workers by listing them in its own `tools`, for example `Agent(worker, researcher)`.

15.5 A worked fan-out, end to end

Here is the pattern in full: several readers over different areas, running at once, then a single synthesis. This is the shape most real fan-outs take.

▶ WORKED EXAMPLE · THREE REVIEWERS OVER ONE LARGE PULL REQUEST

A teammate opened a big pull request that touches three areas: the authentication flow, the database migrations, and the public API. Reviewing it inline would drag every changed file into your main context and mix three unrelated concerns into one muddy thread. Instead you fan out. You keep one

read-only reviewer agent and point three copies of it at three slices of the diff, in one instruction.

I prompt to the main conversation

```
Review this PR in parallel. Spawn three code-reviewer subagents:
one for the auth changes, one for the database migrations, and
one for the API endpoints. Have each report critical issues,
warnings, and suggestions for its own area only. Then combine
their reports into a single prioritized summary for me.
```

Claude spawns three subagents from the same read-only `code-reviewer` definition. Each one runs concurrently, reads only its slice of the diff, and burns through the file contents and `git diff` output inside its own context. None of that reaches you. When each finishes, it hands back a short, structured report: the critical issues first, then warnings, then suggestions.

Your main conversation receives three tidy summaries instead of the entire diff, then does the one job that genuinely needs the whole picture: merging them, resolving overlap, and ranking every finding into a single list you can act on. Three reviewers did the reading in parallel. One synthesizer gave you the answer. Your context holds the conclusion and almost none of the noise it took to reach it.

I TIP

Fan-out works best when the slices are genuinely independent. If reviewing the auth code requires knowing what the API reviewer concluded, they are not independent, and forcing them to run in parallel just produces two half-informed reports. Split along seams that do not talk to each other, and let the synthesis step be the only place the pieces meet.

! GOTCHA

Changes a subagent makes to your files are **not covered by checkpointing or /rewind**. If you let a subagent edit code and then want to undo it, `rewind` will not save you. Use `git` to revert subagent-made changes. This is a strong reason to keep most fan-out agents read-only and reserve write access for the few that truly need it.

! GOTCHA

A custom subagent and general-purpose both receive your `CLAUDE.md` and git status, so your project rules reach them. The built-in Explore and Plan agents skip both to stay fast and cheap. If a rule absolutely must govern an Explore or Plan run, restate it in the task you give them, because they will not have read it from your memory files.

CHAPTER 15 · CHECKLIST**You can now**

- ✓ Explain what a subagent is and why delegating heavy work preserves your main context and sharpens focus.
- ✓ Decide when to delegate a task to a subagent and when to just do it inline.
- ✓ Write a custom agent as a Markdown file with frontmatter for its name, description, tools, and model.
- ✓ Use the `tools` allowlist to build a real read-only permission boundary the subagent cannot cross.
- ✓ Invoke an agent three ways: natural language, an `@`-mention, or as the whole session with `--agent`.
- ✓ Fork the current conversation with `/subtask`, and resume a completed subagent to continue its work.
- ✓ Fan several subagents out in parallel and name the concurrent, per-session, and nesting-depth limits that govern them.

Chapter Sixteen

MCP in Depth

How to wire any tool or data source you own into Claude Code through the Model Context Protocol, from a one-line command to a server you write yourself.

Back in Chapter 5 you connected Claude to a few outside systems through the connector menu, clicking a service into place and letting Claude read your calendar or your docs. That was the app-level view: a curated list, a sign-in button, done. This chapter goes under that surface. The connector you clicked was a Model Context Protocol server, and once you are working in Claude Code you can add any server you like, point it at any tool or database or internal API you own, and reach systems no menu will ever list for you.

The payoff is direct. Instead of copying a Jira ticket, a Sentry stack trace, or a row from Postgres into a prompt, you connect the system once and Claude reads and acts on it in place. The same protocol lets you expose your own tools: a proprietary API, an internal deploy pipeline, a private database. You write the server one time and it works in Claude Code, in Claude Desktop, and in any other application that speaks MCP.

This chapter covers the model that makes that possible, the exact commands to connect each kind of server, where those definitions live and which one wins, how a remote server signs you in, how to manage and trust what you have connected, and finally how to build a minimal server of your own so you can see that authoring one is well within reach.

16.1 The client, host, and server model

MCP is an open standard Anthropic built and donated to the community for connecting an AI application to external tools and data. Anthropic's own image for it is a USB-C port for AI applications: a single standardized way to plug systems in, so one server written for one purpose works with any application that speaks the protocol, with no custom glue code for each pairing. Three roles do the work, and keeping them straight makes everything that follows easier to reason about.

KEY TERM

Host, client, server the **host** is the AI application coordinating everything, which here is Claude Code itself. For each server it connects to, the host creates one **client**, a dedicated connection that talks to exactly that server. The **server** is the separate program that actually provides the capabilities, whether it runs on your own machine or out on the network.

A server exposes its capabilities as a small set of primitives, and knowing the three main ones tells you what to expect the moment a server connects. Each surfaces differently inside Claude Code.

QUICK REFERENCE · WHAT A SERVER EXPOSES AND HOW IT APPEARS

Primitive	What it is	How it shows up in Claude Code
Tools	Functions Claude can call to take action: file ops, API calls, database queries.	Ordinary tools Claude can invoke, named <code>mcp__<server>__<tool></code> .
Resources	Read-only, file-like data: file contents, records, API responses.	Reference with <code>@server:protocol://path</code> , like <code>@github:issue://123</code> , using the same autocomplete as mentioning a file.
Prompts	Reusable interaction templates the server ships.	Become slash commands: <code>/mcp__server__promptname</code> , with optional space-separated arguments.

The connection itself rides on a transport, and which transport a server uses decides how you add it in the next section. This is the one piece of vocabulary worth pinning down before you type a single command.

KEY TERM

Transport the channel a client and server use to exchange messages. A **stdio** transport launches the server as a local process and talks to it over standard input and output, one client to one server. An **HTTP** transport reaches a server across the network at a URL, where one server can serve many clients at once.

16.2 Connecting servers

Every server is added with `claude mcp add`, and the `--transport` flag tells Claude Code how to reach it. Two transports carry almost all real work: local stdio for a program on your machine, and remote HTTP for a cloud service. Start with the remote case, since it is the shorter command.

terminal · add a remote HTTP server

```
# A hosted service, reached at a URL
claude mcp add --transport http notion https://mcp.notion.com/mcp

# Same thing, with a bearer token sent on every request
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer your-token"
```

A local server is one you run as a process on your own machine. The shape is the same until the `--` separator, which splits Claude's own flags from the command that actually starts the server. Everything after `--` belongs to the server.

terminal · add a local stdio server

```
# Pattern: claude mcp add [options] <name> -- <command> [args...]
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio
airtable \
  -- npx -y airtable-mcp-server
```

MCP IN DEPTH

One quirk of the stdio form is worth internalizing now so it never bites you. The `--env` flag accepts several `KEY=value` pairs, and you have to place at least one other option between `--env` and the server name. Skip that and the CLI misreads your server's name as yet another environment pair. The example above does this correctly by putting `--transport stdio` in between.

When a server needs a shape the flags cannot express, or you would rather paste a full definition, `claude mcp add-json` takes a raw JSON object. This is also how you reach the less common transports the flag form does not cover.

terminal · add a server from raw JSON

```
claude mcp add-json weather-api \  
  '{"type":"http","url":"https://api.weather.com/mcp","headers":  
  {"Authorization":"Bearer token"}}'
```

I

TIP

MCP also defines an SSE transport and a WebSocket transport, and you will meet SSE in older setups. SSE is deprecated in the current docs, so prefer HTTP for anything new and keep SSE only for a legacy server that has not migrated. Reach for WebSocket solely when a server must push events to you unprompted, since it cannot do OAuth and is not offered by `claude mcp add -transport`. For the ordinary case, HTTP is the right default.

16.3 Scopes and the .mcp.json file

Every server you add is stored at a scope, and the scope decides who sees it. This mirrors the memory hierarchy from Chapter 9 almost exactly. A `--scope` flag picks the level; leave it off and you get local, which is private to you and to the current project.

QUICK REFERENCE · MCP SERVER SCOPES

Scope	Available in	Shared with team	Stored in
Local (default)	This project only	No	<code>~/.claude.json</code> , under the project's path

Project	This project only	Yes, via version control	<code>.mcp.json</code> in the project root
User	All your projects	No	<code>~/.claude.json</code>

terminal · the same server at three scopes

```

claude mcp add --transport http stripe https://mcp.stripe.com
# local (default)
claude mcp add --transport http shared --scope project
https://example.com/mcp # writes .mcp.json
claude mcp add --transport http hubspot --scope user
https://mcp.hubspot.com # every project

```

Project scope is the one that matters for a team. It writes a `.mcp.json` file in your project root, and because that file lives in the repo, everyone who clones the project gets the same servers. The format is a keyed object under `mcpServers`, where each entry names its `type` and the fields that transport needs.

.mcp.json

```

{
  "mcpServers": {
    "api-server": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": { "Authorization": "Bearer ${API_KEY}" }
    }
  }
}

```

That example shows the feature that makes a committed `.mcp.json` safe to share. Values can reference environment variables. `${API_KEY}` expands to the variable's value, and `${API_BASE_URL:-https://api.example.com}` uses the variable if set and the default otherwise. Expansion works in `command`, `args`, `env`, `url`, and `headers`. If a referenced variable is unset and has no default, the config still loads and `claude mcp list` reports a missing-variable warning.

! GOTCHA

Never paste a real token straight into a committed `.mcp.json`. That file goes into version control, so a hardcoded secret is a secret leaked to everyone with repo access and everyone in its history. Use `${VAR}` expansion for every credential and keep the actual values in your environment, out of the file. The `env-var` syntax exists precisely so a shared config can carry no secrets at all.

When the same server name exists at more than one scope, the precedence is fixed: **local beats project beats user**, then plugin-provided servers, then your `claude.ai` connectors. The winning scope's entry is used whole. Fields are never merged across scopes, so a partial override does not exist. The most specific definition simply wins.

! GOTCHA

A project server someone else committed into `.mcp.json` does not connect silently. The first time you open a project carrying one, Claude Code asks for your one-time approval, and until you give it the server shows as pending. This is a real security gate: a shared repo config launching a server on your machine is the same class of risk as a CI script. Read what the entry runs before you approve it. If you approve too fast and change your mind, `claude mcp reset-project-choices` clears your approvals so you can decide again.

16.4 Authentication and OAuth

A remote server usually sits behind a sign-in. MCP handles this with OAuth, the same browser-based flow you use to let one web app connect to another, so you never hand a raw password to the server or to Claude Code. Claude Code marks a server as needing authentication when a request comes back `401` or `403`, and `claude mcp list` shows that state plainly with a needs-authentication marker.

When you see it, you have two ways in. Run `/mcp` inside a session and follow the browser flow it opens, or trigger the same flow from the shell.

I terminal · sign in to a remote server

```
claude mcp login sentry           # open the browser OAuth flow
claude mcp login sentry --no-browser # print the URL instead, for SSH
sessions
claude mcp logout sentry          # clear stored credentials
```

The browser opens, you approve the connection with the service directly, and Claude Code stores the resulting credentials so you are not asked again. The `--no-browser` variant prints the URL rather than launching a browser, which is what you want on a remote box over SSH where no browser exists to open.

I TIP

You can narrow what a sign-in grants. A server entry in `.mcp.json` accepts an `oauth` block with a `scopes` string, so you can request read access to channels and nothing more when that is all Claude needs. Requesting the least access that does the job is good hygiene here for the same reason it is everywhere else: a connection that cannot do a dangerous thing cannot be tricked into doing it. Some servers also need pre-registered callback ports or client credentials; the auth docs cover those cases, which shift often enough that you should read the current page rather than trust a fixed recipe.

16.5 Managing servers with `/mcp`, and who to trust

Once servers are connected, `/mcp` is your control panel. Run it inside Claude Code to see every server's connection status, the number of tools each one exposes, and a way to authenticate any that need it. It also lets you toggle a server on or off without deleting its configuration, which is the clean way to silence a noisy server for a session and bring it back later. From the shell, `claude mcp list` gives the same health at a glance and `claude mcp get <name>` prints the details of one.

I TIP

Do not ration servers out of a fear of context bloat. Claude Code defers MCP tool schemas by default, loading only tool names and each server's short instructions at startup and fetching a full schema when Claude actually reaches for that tool. Adding many servers therefore costs almost nothing up front. This deferred loading has provider-specific fallbacks and can be off on some non-first-party deployments, so if you run Claude Code against an unusual backend, confirm it is on before you assume a large server list is free.

Connecting a server is an act of trust, and it deserves a moment of real thought each time. An MCP server runs with genuine tool-call power inside your session. It can read what you point it at and act through the tools it exposes, which is the entire point and also the entire risk.

! GOTCHA

Only connect servers you trust, and understand why. Anthropic's own guidance is blunt: verify you trust each server before connecting it, because a server that fetches external content can expose you to **prompt injection**. A web page, an issue comment, or a database row that a server pulls in can carry text crafted to hijack Claude's next action. A third-party server you have not vetted is running with your access. Read the source or trust the publisher before you add it, treat any server that fetches outside content with extra care, and grant it the narrowest reach that still does the job.

The recurring question of what to reach for and when has a clean answer here. Use a `claude.ai` connector or a directory-listed server for the mainstream services you would recognize by name, where a known publisher stands behind it. Reach for a self-built or self-hosted server, covered next, when the system is yours and no public server exists. And when a repo hands you a `.mcp.json` you did not write, read it before you approve it, every time.

16.6 Building a minimal custom server

The reason to write your own server is simple: your internal systems will never appear in any public directory. A server is the bridge from Claude Code to a private API, an internal database, or a bespoke pipeline. The encouraging part is how little code it takes. The official Python quickstart builds a working weather server in a single short file, and the core of it is small enough to read in one sitting.

 terminal · project setup

```
uv init weather && cd weather
uv venv && source .venv/bin/activate
uv add "mcp[cli]"
touch weather.py
```

The server needs Python 3.10 or newer. Inside `weather.py`, you create a server object, decorate a plain async function as a tool, and run it over stdio. The function's type hints and docstring are read automatically to generate the tool's schema and the description Claude sees, so a clear docstring is not documentation you add later. It is the interface.

```
weather.py

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather")

@mcp.tool()
async def get_alerts(state: str) -> str:
    """Get weather alerts for a US state.

    Args:
        state: Two-letter US state code (e.g. CA, NY)
    """
    # ... fetch and format alerts for `state` here ...
    return "No active alerts for this state."

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

That is a complete server. Register it with Claude Code the same way you added any local server earlier, pointing the command at your file.

```
terminal · register your server
```

```
claude mcp add --transport stdio my-server -- python weather.py
```

GOTCHA

Never call `print()` in a stdio server. A stdio server talks to Claude Code over standard output using JSON-RPC, and a stray `print` injects text into that stream and corrupts it, producing failures that look like anything except their real cause. Send diagnostics through the standard `logging` module instead, which writes to standard error and leaves the protocol channel clean. HTTP-based servers do not have this restriction.

I TIP

The MCP SDKs are moving quickly, and the exact import path for the server class is one of the details that shifts between versions. The `FastMCP` class shown here is the current Python entry point; some older tutorials wire the server up differently. Before you ship, check the import path against the version of the `mcp` package you actually installed. A TypeScript SDK offers the same shape for teams that prefer Node, and the MCP Inspector, run with `npx @modelcontextprotocol/inspector`, gives you a local web UI to call your tools by hand before Claude ever touches them.

CHAPTER 16 · CHECKLIST

You can now

- ✓ Name the host, client, and server roles and say which primitive surfaces as a tool, a resource, and a slash command.
- ✓ Add a remote HTTP server and a local stdio server, and know why the `--` separator and `--env` placement matter.
- ✓ Choose the right scope for a server and predict which definition wins when a name collides.
- ✓ Read and write a `.mcp.json` file, and keep every secret out of it with `${VAR}` expansion.
- ✓ Sign in to a remote server through OAuth from `/mcp` or the shell, including over SSH.
- ✓ Use `/mcp` to inspect, toggle, and authenticate servers, and vet a server for prompt-injection risk before trusting it.
- ✓ Build a minimal custom server, register it with Claude Code, and avoid the stdio `print` trap.

Chapter Seventeen

Worktrees, Checkpoints & Sessions

How to run two jobs at once without collision, rewind a bad turn without panic, and pick up any past conversation exactly where you left it.

Real work is not one clean line. You are three prompts into a refactor when a production bug lands, so you want a second Claude on the bug without losing the refactor. An edit goes sideways and you want to step back two turns without unpicking it by hand. You close the terminal at six and want the whole conversation back at nine the next morning. These are not edge cases. They are Tuesday.

Claude Code has a durable answer for each one. Worktrees let you run parallel sessions whose files never touch. Checkpoints snapshot your files before every prompt so you can rewind. Sessions persist to disk so you can resume, continue, or branch a past conversation. Together they turn a single fragile chat into something you can run in parallel, undo, and reopen at will.

This chapter covers all four mechanisms plus one more: headless mode, the one-shot `-p` form introduced back in Chapter 8, seen here as the engine for automation and unattended runs. Real commands throughout. Where a detail shifts between releases, this chapter tells you the concept and points you at the docs for the current number.

17.1 Parallel worktrees, isolated by file

A subagent (Chapter 16) isolates the *work* by giving a task its own context window. It can still write to the same files as everything else. A worktree isolates the *files*. It is the tool for running two real jobs against one repo at the same time without either stepping on the other.

KEY TERM

Git worktree a second working directory checked out from the same repository, on its own branch, sharing the one underlying `.git` history and remote. Edits in one worktree are physically separate files from edits in another, so two Claude sessions can build in parallel and never collide.

Picture the repo you know: one folder, one branch checked out, one set of files on disk. A worktree is a sibling folder with a *different* branch checked out, backed by the same project history. Commit in one and the object lands in the shared `.git` store, so both worktrees see it. The files on disk stay separate. That separation is the whole point. Session A refactoring billing and session B patching auth are editing two different directories that happen to be the same project.

Claude Code has a built-in flag that does the `git worktree add`, the `cd`, and the launch in one move. Give it a name and you are dropped into an isolated session.

terminal

```
# create + enter an isolated worktree session
claude --worktree feature-auth
# short form
claude -w feature-auth
```

That creates the worktree under `.claude/worktrees/feature-auth/` at the repo root, on a new branch named `worktree-feature-auth`. Open a second terminal, run the same command with a different name, and you now have two Claude sessions working the same repo in parallel with zero file contention. Omit the name and Claude generates one for you, like `bright-running-fox`.

WORKTREES, CHECKPOINTS & SESSIONS

Reuse a name and Claude reopens that existing worktree instead of making a new one.

▶ WORKED EXAMPLE · SPIN UP A PARALLEL BUG-FIX WHILE A REFACTOR RUNS

You are mid-refactor in your main checkout when a bug report arrives. You do not want to stash, switch branches, and lose your place. So you leave the refactor session running and open a fresh terminal.

terminal (second window)

```
claude -w hotfix-login
```

Claude creates `.claude/worktrees/hotfix-login/` on branch `worktree-hotfix-login`, branched fresh from `main`, and lands you in it. You fix the bug, commit, and push a PR from that worktree. Your refactor session in the other window never noticed. When you exit the hotfix session and its worktree is clean, Claude offers to remove it and the branch for you. The refactor picks up exactly where you left it, because you never touched it.

By default a new worktree branches from `fresh`, meaning the repo's default branch such as `main`. When you want the worktree to carry your current unpushed commits instead, set `worktree.baseRef` to `head` in settings, which branches from your local `HEAD`. You can also branch straight from a pull request. Quote the argument, because a bare `#` is a comment to your shell.

terminal

```
# branch a worktree from PR #1234
claude --worktree "#1234"
```

You can also stay in one session and ask Claude to "work in a worktree" mid-conversation. It uses the `EnterWorktree` tool to move you into a fresh one, switches between worktrees with the same tool, and steps you back out with `ExitWorktree`. For the cases where you need a specific existing branch or a location outside the repo, the raw git commands are always there underneath.

terminal

```
# a worktree on a new branch, in a sibling folder
git worktree add ../project-feature-a -b feature-a

# from an existing branch
git worktree add ../project-bugfix fix-issue-456

cd ../project-feature-a && claude

# housekeeping
git worktree list
git worktree remove ../project-feature-a
```

TIP

Add `.claude/worktrees/` to your `.gitignore` so worktree contents do not show up as untracked files in your main checkout. For the gitignored files a worktree genuinely needs to run, like `.env`, create a `.worktreeinclude` file at the repo root in `.gitignore` syntax. Claude Code copies anything it lists into every new worktree it creates.

TIP

Worktrees and subagents compose. A subagent isolates the work; a worktree isolates the files. Set `isolation: worktree` in a subagent's frontmatter and that agent runs in its own worktree, so a fleet of parallel agents can each edit freely without a single merge conflict between them. Reach for it when you fan out mechanical work across many files at once.

GOTCHA

Cleanup depends on how the session ran. An interactive, unnamed, clean worktree is removed automatically on exit. A named one, or one with uncommitted work, prompts you to keep or remove it. A non-interactive `claude -p` run has no exit prompt and never auto-cleans, so you remove those yourself with `git worktree remove`. A periodic background sweep clears out old subagent and background worktrees past `cleanupPeriodDays`, but it skips anything holding uncommitted work and never touches worktrees you made yourself with `--worktree`.

I UNDER THE HOOD

Some things are shared across every worktree of a repo no matter how it was created: the `.git` directory itself, project-scope plugins, and, in recent builds, your saved "don't ask again" permission approvals. Approve a Bash command inside a worktree and the rule is written to the main checkout's `.claude/settings.local.json`, so it applies everywhere. That approval-sharing behavior is version-dependent, so if your team relies on it, confirm it against your installed version before you count on it.

17.2 Checkpoints and the rewind flow

Worktrees keep parallel work apart. Checkpoints let you walk a single session backward in time. Claude Code automatically snapshots the state of the files it edited right before each prompt you send. Every prompt creates one checkpoint, silently, with no action from you. When a turn goes wrong, you rewind to the snapshot from before it.

KEY TERM

Checkpoint an automatic snapshot of the files Claude edited, taken just before each user prompt. It captures your working state so you can restore it later. It is local undo for a session, saved alongside the conversation so a resumed session can still rewind into its own history.

The rewind menu is the front door. Open it with `/rewind`, or press `Esc` twice on an empty prompt input. The menu lists every prompt you sent in the session. Pick a point, then choose what to restore.

QUICK REFERENCE · REWIND MENU ACTIONS

Action	What it does
Restore code and conversation	Revert both files and chat to that point.
Restore conversation	Rewind the chat, keep your current code.
Restore code	Revert the files, keep the conversation.

Summarize from here	Compress everything from this point forward into a summary, keeping earlier context.
Summarize up to here	Compress everything before this point, keeping later messages, staying at the end.
Never mind	Cancel and change nothing.

The two code-restore options only appear when the checkpoint you picked actually has tracked file changes to revert. Reach for rewind the moment a turn produces edits you do not want and correcting them in place would be fiddlier than starting that turn over. It is faster than reading a diff and hand-reverting, and it takes the conversation back with it so Claude is not working from a mental model you already discarded.

▶ DO THIS · UNDO A TURN THAT WENT SIDEWAYS

- 1 Stop and open the menu.** On an empty input, press `Esc` twice, or type `/rewind`. If your input still has text, double-Esc clears the text instead, so clear it first, then press `Esc` twice again.
- 2 Pick the prompt before things went wrong.** The list is every prompt you sent. Choose the last good one.
- 3 Choose your restore scope.** Take **Restore code and conversation** to fully reset that turn, or **Restore code** when you want the files back but want to keep discussing what happened.
- 4 Continue from the clean state.** Re-prompt with a sharper instruction. The bad turn is gone from your files as if it never ran.

! GOTCHA

Checkpoints only track edits Claude made through its own Read, Edit, and Write tools. File changes made by a Bash command are **not** tracked. An `rm file.txt`, an `mv`, or a `cp` run through Bash cannot be undone by `/rewind`. Subagent edits are not restored by the main session's rewind either, and neither are files you changed outside Claude Code. For anything Bash or a subagent touched, git is your undo, not checkpoints.

I TIP

Think of checkpoints as local undo and git as permanent history. Checkpoints are per-session, auto-pruned after a retention window, and capped at a fixed number of recent snapshots per session, so they are for fast in-session recovery. They are not a backup and not a substitute for committing. The exact retention count and cleanup window are version-sensitive, so check the checkpointing docs for the current numbers if you are counting on them.

! GOTCHA

On restore, symlinked and hard-linked files are skipped, and you get a warning like `Restored the code, but skipped N files`. The debug log at `~/.claude/debug/<session-id>.txt` names which ones. If your project leans on linked files, verify their state by hand after a rewind. Do not assume the restore was total.

17.3 Sessions: resume, continue, branch

Every Claude Code conversation is written to disk as you go. That is what lets you close the terminal and pick the whole thing back up later, complete with its history, tool calls, and results. Three verbs cover the lifecycle: continue the most recent one, resume a specific one, or branch a copy to try a different path.

QUICK REFERENCE · RESUMING AND CONTINUING

Command	What it does
<code>claude --continue</code> (or <code>-c</code>)	Resume the most recent session in the current directory.
<code>claude --resume</code> (or <code>-r</code>)	Open the interactive session picker.
<code>claude --resume <name></code>	Resume a session you named, directly.
<code>claude --resume <session-id></code>	Resume by ID, which also reaches <code>-p</code> sessions that never show in the picker.
<code>claude --from-pr <number></code>	Open the picker filtered to sessions linked to that PR.

`/resume` or `/resume`
`<name>`

Switch to another conversation from inside an active session.

Resume restores a great deal automatically: the full conversation history, the model, the permission mode, an agent identity if you launched with `--agent`, an active goal, and non-expired scheduled tasks. A couple of things need re-enabling by hand for safety, namely `plan` and `bypassPermissions` mode. Resume by ID or by name must be run from the same project directory the session started in, because the lookup is scoped to that directory and its worktrees.

GOTCHA

Some launch flags are **not** restored on resume and must be re-passed every time: `--mcp-config`, `--settings`, `--plugin-dir`, `--fallback-model`, and any directories you added with `--add-dir`. Your standard `settings.json` and `settings.local.json` are re-read fresh at launch, so those carry over. Background Bash and Monitor tasks do not survive a resume either. If a resumed session suddenly cannot see an MCP server, a re-passed flag is almost always the reason.

Name a session and you get a stable handle you can resume by. Name it at startup with `-n`, or rename it live with `/rename`, or highlight it in the picker and press `Ctrl + R`.

terminal

```
# name a session at startup
claude -n auth-refactor
```

GOTCHA

An unnamed session gets two labels, and neither is a valid resume handle. One is an auto-generated display name like `my-app-3f`. The other is an AI-written title a small model produces from your first prompt. They help you spot a session in the picker, and that is all. Only a name you set yourself, with `-n`, `/rename`, or `Ctrl + R`, works as a resume argument.

WORKTREES, CHECKPOINTS & SESSIONS

The picker is quick once you know its keys. Arrows navigate, `Enter` resumes, `Space` previews. Any printable character starts a search, and pasting a PR or MR URL finds that link's session. `Ctrl + A` shows all projects, `Ctrl + W` shows all worktrees of this repo, and `Ctrl + B` filters to your current git branch.

Branching is the third verb, and it is how you explore a fork in the road without burning your original path. It copies the transcript up to a chosen point into a new session ID and switches you into the copy. The original is untouched on disk and still resumable, so you can try a risky approach in the branch and walk back to the trunk if it does not pan out.

```
terminal / session

# from inside a session
/branch try-streaming-approach

# or at launch, fork the continued session
claude --continue --fork-session
```

I TIP

When you branch, in-flight background subagents and Bash commands keep running, and their output lands in the **new** branch, not the original. If you branch specifically to escape a long-running task, remember it followed you into the copy. Plan for its output to arrive where you now are.

I UNDER THE HOOD

Sessions live at `~/.claude/projects/<project>/<session-id>.jsonl`, one JSON object per line, where `<project>` is your working-directory path with non-alphanumeric characters replaced by dashes. The format is internal and changes between releases, so do not parse those files directly in scripts. For programmatic access use `/export`, `claude -p --output-format jsonl`, or the Agent SDK. Retention follows `cleanupPeriodDays`, you can relocate storage with `CLAUDE_CONFIG_DIR`, and `--no-session-persistence` suppresses the write for a single run.

17.4 Background tasks that keep running

Some commands take minutes: a dev server, a full test suite, a docker build, a terraform apply. You do not want Claude frozen while they run, and you do not want to babysit them. Background execution moves a long-running Bash command off the main thread so Claude keeps working and checks back on the output when it is ready.

You trigger it two ways. Press `Ctrl + B` to send a running Bash tool call to the background, or just ask in plain language, "run the dev server in the background." Claude returns a background task ID immediately and moves on. The task's output streams to a file that Claude reads back with its Read tool whenever it needs to see progress.

TIP

Tmux users, press `Ctrl + B` twice. Tmux claims the first press as its own prefix key, so a single press never reaches Claude Code. This one trips up a lot of people who swear the shortcut is broken.

To see what is currently running, use the `/tasks` command, which lists both background shells and running subagents. Note that this list is separate from Claude's own to-do checklist. If you are following older community write-ups, you may see the name `/bashes` for roughly the same idea. Treat that as the predecessor term for what `/tasks` now covers, and check your installed version, because this naming has shifted between releases.

Shell mode is the close cousin worth knowing. Prefix a line with `!` and Claude Code runs the command verbatim without interpreting or approving it, since you chose the command, not the model. The command and its output go straight into the conversation context, with real-time progress, and the same `Ctrl + B` backgrounding works.

```
! session (shell mode)
```

```
! npm test
! git status
```

! GOTCHA

Background tasks are tied to the session that started them. They are auto-cleaned when Claude Code exits, unless you background the whole session instead of exiting. A task is also terminated if its output exceeds a large size cap, on the order of several gigabytes, and on some platforms an idle session's background tasks can be reaped under memory pressure. None of this is a durable job scheduler. For work that must outlive the session, use a real scheduler or CI, not a backgrounded Bash call.

17.5 Headless mode for automation

Chapter 8 introduced `-p` as the way to get a one-shot answer from Claude without entering the interactive UI. That same flag is the foundation for automation. A `claude -p` call runs non-interactively, does its work, prints a result, and exits, which makes it a building block you can drop into scripts, pipelines, and CI jobs.

terminal

```
# one-shot, non-interactive, with a tool allowlist
claude -p "Find and fix the bug in auth.py" --allowedTools
"Read,Edit,Bash"
```

For scripted and CI use, add `--bare`. It skips auto-discovery of hooks, skills, plugins, MCP servers, auto-memory, and CLAUDE.md, so only the flags you pass explicitly take effect. That gives you a determinism a shared CI runner can depend on. In bare mode, authentication has to come from an `ANTHROPIC_API_KEY` or an `apiKeyHelper`, since keychain and OAuth reads are skipped along with everything else.

terminal

```
claude --bare -p "Summarize this file" --allowedTools "Read"
```

GOTCHA

The relationship between `-p` and `--bare` is in motion. Bare mode is described as the recommended mode for scripted and SDK calls, and it is slated to become the *default* for `-p` in a future release. Until it is, a plain `-p` still auto-discovers your local hooks, memory, and MCP config, which is exactly what you do not want polluting a CI run. Verify the current default for your installed version, and pass `--bare` explicitly whenever you need CI-safe behavior today.

Output format is what makes headless mode composable. The default is plain text. Ask for `json` and you get a structured object with the `result`, the `session_id`, and cost and usage metadata, which is what you parse in a script. Ask for `stream-json` and you get newline-delimited events in real time. The examples below cover the three moves you will reach for most: pipe data in, pull a clean field out, and continue a headless conversation across calls.

terminal

```
# pipe a file in, capture the answer
cat build-error.txt | claude -p 'explain the root cause of this build
error' > output.txt

# structured JSON out, extract one field with jq
claude -p "Summarize this project" --output-format json | jq -r
'.result'

# continue a headless conversation by capturing the session id
session_id=$(claude -p "Start a review" --output-format json | jq -r
'.session_id')
claude -p "Continue that review" --resume "$session_id"
```

The continue-by-ID pattern is how you script a multi-step job that has to remember its own earlier steps. Capture the `session_id` from the first call's JSON, then feed it back with `--resume` on each follow-up. As with interactive resume, run every call from the same directory the session started in, because ID lookup is scoped to that project directory and its worktrees.

For unattended runs, you also have to answer the permission prompts that would normally stop and ask you. Pre-approve the exact tools a job needs with

`--allowedTools`, which takes the same permission-rule syntax as the rest of Claude Code, or set a permission mode outright.

terminal

```
claude -p "Run the test suite and fix any failures" --allowedTools
"Bash,Read,Edit"
claude -p "Apply the lint fixes" --permission-mode acceptEdits
```

I TIP

Watch the space in a permission rule. `Bash(git diff *)` with a space before the `*` matches `git diff` and its arguments. Written as `Bash(git diff*)` with no space, it also matches `git diff-index` and other commands you never meant to allow. In an unattended run that difference is the gap between a tight allowlist and an accidental open door.

! GOTCHA

A background Bash task started during a `claude -p` run is killed a few seconds after Claude returns its final result. There is a short grace period so a task that finishes right at the end still delivers its output, and background subagents get a longer wait, but a long-lived server backgrounded inside a headless call will not keep running once the call returns. Headless mode is for work that completes and reports. Persistent services belong elsewhere.

CHAPTER 17 · CHECKLIST

You can now

- ✓ Explain what a worktree is and why running Claude in separate worktrees isolates files so two jobs never collide.
- ✓ Spin up an isolated parallel session with `claude --worktree`, and clean it up correctly when done.
- ✓ Rewind a bad turn with `/rewind`, and choose whether to restore code, conversation, or both.
- ✓ Say what checkpoints do not track, and reach for git instead of rewind for Bash and subagent changes.
- ✓ Continue, resume by name or ID, and branch a session, and name a session so it has a real resume handle.
- ✓ Push a long command to the background, watch it with `/tasks`, and know when it will be reaped.
- ✓ Drive Claude headlessly with `-p`, choose an output format, chain calls by session ID, and pre-approve tools for unattended runs.

PART IV

Putting It Together

Individual tools are the vocabulary. Workflows are the language. This final part wires the pieces into real operator setups you can copy: a self-maintaining project, a research and review pipeline, a scheduled agent, a safe-by-default configuration. It closes with the practice that keeps you sharp as the product moves under your feet.

Chapter Eighteen

Operator Workflows

Five copy-ready recipes that wire the individual tools from Part III into repeatable, end-to-end setups you can paste into a real project today.

Every tool in this manual so far has been a single instrument. A memory file remembers. A hook enforces. A subagent explores without flooding your context. A skill packages a procedure. Each one is useful alone, and each one is far more useful in a chord. The operators who get real leverage out of Claude Code are the ones who stopped thinking about features and started thinking about workflows: two or three tools wired together so a whole outcome happens on its own.

This chapter is five of those workflows. Each is a complete recipe with real config, built from tools you already met in earlier chapters and cross-referenced back to where each one was taught. Nothing here is new machinery. The value is in the wiring.

Read them as patterns, not scripts. Copy the shape, then swap in your own file paths, formatter, and rules. The map below is the whole chapter on one page: a recurring need on the left, the combination that solves it in the middle, and the chapters that taught the parts on the right.

QUICK REFERENCE · WHICH COMBINATION SOLVES WHICH NEED

Recurring need	Combine	Taught in
A project enforces its own standards	CLAUDE.md, a <code>PostToolUse</code> hook, a custom command	Ch 9, 14, 12

OPERATOR WORKFLOWS

Gather facts, then verify them	A forked research skill, a read-only reviewer subagent	Ch 13, 15
Work happens while you sleep	Headless <code>claude -p</code> on a schedule	Ch 17
A setup that resists misuse	Permission <code>deny</code> rules, a <code>PreToolUse</code> guard hook	Ch 10, 14
A multi-step job runs identically every time	An orchestrator skill that chains the steps	Ch 13

18.1 The self-maintaining project

The goal here is a project that holds its own standards without you policing them. Three layers do it, each from a different chapter, each covering a gap the others leave. CLAUDE.md (Chapter 9) states the standard so Claude knows it. A hook (Chapter 14) enforces one slice of it automatically. A custom command (Chapter 12) audits the whole thing on demand.

Start with the standard itself. CLAUDE.md is context: Claude reads it and usually follows it, which is exactly right for conventions that need judgment.

CLAUDE.md

```
# Project standards
- Format all TypeScript with Prettier before it lands.
- API handlers live in `src/api/handlers/`.
- Run `npm test` before every commit.
```

The formatting rule is the one you never want to depend on judgment for. Promote it from a request to a guarantee with the auto-formatter hook from Chapter 14. A `PostToolUse` hook on the edit tools reads the path Claude just wrote and runs Prettier over it, every time, with no model decision involved.

.claude/settings.json

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": "jq -r '.tool_input.file_path'
| xargs npx prettier --write" }
      ] }
    ]
  }
}
```

The third layer is the on-demand audit. A custom command turns "check that we still meet our own rules" into one word you type. This uses the legacy `.claude/commands/` format from Chapter 12, with the `!`command`` injection that runs a shell command and drops its output into the prompt before Claude reads it.

.claude/commands/check.md

```
---
allowed-tools: Bash(npm run *), Bash(npx *)
description: Audit the project against its own standards
---

## Current state
- Lint: !`npx eslint . --max-warnings 0`
- Types: !`npx tsc --noEmit`
- Tests: !`npm test`

## Task
Report anything above that failed, then fix it to match the rules in
CLAUDE.md.
```

Now the three layers cover each other. CLAUDE.md keeps the standard in front of Claude on every turn. The hook holds the one rule that must never slip. The `/check` command gives you a full sweep whenever you want one. The project maintains itself, and you moved from enforcing standards to just declaring them.

I TIP

Keep the hook narrow. The matcher `Edit|Write` fires the formatter only after the two write tools, so it never runs on the dozens of reads and searches in a session. A guard that runs on everything is the fastest way to make a project feel slow (Chapter 14).

18.2 A research-and-review pipeline

Research and verification are two different jobs, and doing them in one pass is how wrong answers slip through. This pipeline splits them. A forked skill (Chapter 13) fans out to gather findings, then a read-only subagent (Chapter 15) checks each finding against the file it claims to come from. Gathering is noisy and cheap. Verifying is careful and separate.

KEY TERM

Pipeline two or more agents wired in sequence, where the output of one becomes the input to the next. The first stage produces a draft, and a later stage does nothing but check it, so no single agent is trusted to both write and grade its own work.

The gathering stage is a skill that runs in a forked subagent. The `context:fork` field sends the whole job to a separate context window, so the twenty files it reads never touch your main conversation. Pointing it at the built-in `Explore` agent keeps it fast.

.claude/skills/gather/SKILL.md

name: gather

description: Research a question across the codebase, returning findings with file citations.

context: fork

agent: Explore

Research \$ARGUMENTS thoroughly:

1. Find every relevant file with Glob and Grep.
2. Read the code and note the exact path and line for each point.
3. Return a list of findings, each with the source you drew it from.

The verifying stage is a subagent whose tool list holds only `Read`, `Grep`, and `Glob`. That allowlist is a real boundary, not a suggestion: a fact-checker that cannot write cannot quietly rewrite the thing it was supposed to audit.

.claude/agents/fact-checker.md

name: fact-checker

description: Verifies claims against the files they cite. Use after any research pass.

tools: Read, Grep, Glob

model: sonnet

You verify claims. For each claim you are handed:

1. Open the cited file and confirm the claim is accurate.
2. Mark it Confirmed, Wrong, or Unverifiable, with the exact line you checked.

Report only the verdicts and the evidence behind them.

► DO THIS · RUN THE PIPELINE

- 1 **Fan out.** Run `/gather how does session auth work`. The forked skill explores in its own context and returns a cited list of findings, keeping the file contents out of your window.

- 2 **Hand the findings to the checker.** Say *"use the fact-checker subagent to verify each of these against its source."* The read-only agent reopens every cited file and grades each claim.
- 3 **Keep only what survived.** You end with a list of Confirmed findings and a short list of Wrong ones to discard, produced by two agents that never shared a context.

! GOTCHA

The `Explore` agent skips `CLAUDE.md` and `git status` to stay lean, so a rule that lives only in your memory file will not reach the gathering stage. If a convention matters to the research, restate it inside the skill body or the delegation prompt. `Explore` is also one-shot and cannot be resumed, so use `general-purpose` for the fork when you need to send follow-up questions to the same researcher (Chapter 15).

18.3 A scheduled or background agent

Some work should happen without you in the chair. Triage the new bug reports overnight. Summarize yesterday's commits before you wake up. Chapter 17 covered the unattended modes; this recipe wires one to a clock. The engine is headless mode: `claude -p` runs a single prompt non-interactively and exits, which is exactly what a scheduler needs.

The command below is the whole agent. The `--bare` flag strips auto-discovery so the run is deterministic and CI-safe, `--allowedTools` pre-approves the exact tools it may use without a prompt, and `--output-format json` gives you a parseable result with cost and session metadata.

```
❯ triage.sh
```

```
#!/bin/bash
claude --bare -p "Triage open issues labeled bug and draft a one-line
fix plan for each" \
  --allowedTools "Read,Grep,Glob,Bash(gh *)" \
  --output-format json > triage-result.json
```

Wrap that in whatever scheduler your machine already runs. On Linux or macOS a cron entry fires it every morning. On Windows, Task Scheduler does the same. The agent wakes, does its job against the tools you allowed, writes its result, and exits, all with nobody watching.

I TIP

The JSON output carries a `session_id` and a `total_cost_usd` field. Pipe it through `jq` to log both, so an unattended agent leaves a paper trail of what it did and what it cost. You can resume that exact session later with `claude --resume` and its ID from the same project directory (Chapter 17).

! GOTCHA

Bare mode is deliberately forgetful. It skips `CLAUDE.md`, hooks, skills, and auto-memory, so anything the run depends on has to be passed on the command line. In bare mode, authentication also comes from an API key in the environment, not from your saved login. And a skill marked `disable-model-invocation: true` will not auto-run from a scheduled task at all, by design. If your scheduled agent needs a skill, drive it explicitly in the prompt and make sure that skill is allowed to run.

18.4 A safe-by-default configuration

A shared repo should be hard to misuse, including by an over-eager agent on a bad afternoon. Two layers make it so, and they belong to two chapters. Permission rules (Chapter 10) declare what is off limits. A guarding hook (Chapter 14) enforces the rule that absolutely cannot slip. This is belt and suspenders, and it holds where either layer alone might not.

Start with the permission layer. A `deny` rule in settings blocks a matching tool call outright, and Claude cannot switch it off by changing permission mode.

```
.claude/settings.json
```

```
{
  "permissions": {
    "deny": [
      "Bash(rm -rf *)",
      "Agent(Explore)"
    ]
  }
}
```

Permission rules cover the tool call. They do not read a file's path the way a script can, and the matcher catalog for file scopes is worth learning from Chapter 10 directly, since guessing at it invites mistakes. For the one rule you refuse to leave to a matcher, add the hard gate from Chapter 14: a `PreToolUse` hook on the edit tools that inspects the target path and exits 2 the moment it sees a protected file.

```
.claude/hooks/protect-files.sh
```

```
#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

PROTECTED_PATTERNS=( ".env" "package-lock.json" ".git/" )

for pattern in "${PROTECTED_PATTERNS[@]}; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
    exit 2
  fi
done

exit 0
```

Wire it to the edit tools with the same settings file, and the two layers now reinforce each other. The permission rules state the policy in a form the whole team can read. The hook guarantees the one line that matters, even in a mode that would skip the permission check.

I TIP

Commit both layers to `.claude/settings.json` so every teammate and every worktree inherits the same guardrails. Keep truly personal, machine-only exceptions in the gitignored `.claude/settings.local.json` instead, so your private tweaks never weaken the shared policy (Chapter 10).

! GOTCHA

Hooks tighten, they never loosen. A `PreToolUse` hook returning `deny` cannot be overridden by switching permission mode, which is why the gate is trustworthy. The reverse does not hold: a hook that says `allow` does not punch a hole through a `deny` rule in your permission settings. Treat the hook as a way to add restrictions on top of the permission system, never as a way to relax it (Chapter 14).

18.5 A multi-skill workflow

Some jobs have to run the same way every single time. A release. A data import. Anything where step three depending on step two is the whole point. The reliable way to get that is one orchestrator skill (Chapter 13) that lays out the steps in order and can call the smaller skills you already built, so the sequence is written down once and repeated exactly.

Here is a release orchestrator. It reuses the `/check` command from the first recipe as its opening step, which is the payoff of building your tools as composable pieces. Because it has side effects, it is marked `disable-model-invocation: true`, so only you can fire it, never Claude on its own.

```
I .claude/skills/ship/SKILL.md
```

```
---
name: ship
description: Run the full release checklist in order, the same way
every time.
disable-model-invocation: true
allowed-tools: Bash(npm *) Bash(git *)
---
```

Ship the release in this exact order. Stop and report if any step fails.

1. Run the /check audit and confirm the project is clean.
2. Bump the version and update the changelog.
3. Build the app, then run the full test suite.
4. Tag the commit and push.

Now a release is one word: `/ship`. The steps live in the file, so the process does not drift as you get tired or distracted. When a step needs to change, you edit the skill once and every future release picks up the new order.

I TIP

For a quick, throwaway chain you do not need a file for, stack skills inline at the start of a message: `/gather /fact-check the auth flow` loads both skills and passes the trailing text to each. Reach for an orchestrator skill when the sequence is worth saving and repeating; reach for inline stacking when you just want two skills on one job right now (Chapter 13).

! GOTCHA

An orchestrator skill is instructions, not a script. Claude reads the numbered steps and follows them, with the same "usually, not always" caveat that governs every memory file. If a step in the middle must be blocked or must always fire no matter what, back it with a hook. Reserve the skill for the shape of the workflow, and let hooks hold the steps that cannot be skipped.

That is the whole idea of this chapter in one sentence. Skills give a workflow its shape, permissions and hooks give it its guardrails, subagents keep its

context clean, and a scheduler lets it run without you. You already had all four. Now you can wire them.

CHAPTER 18 · CHECKLIST

You can now

- ✓ Build a self-maintaining project by combining a CLAUDE.md standard, a `PostToolUse` hook, and a custom audit command.
- ✓ Split research from verification with a forked gathering skill and a read-only reviewer subagent that never shares its context.
- ✓ Run an unattended agent on a schedule with headless `cLaude -p`, bare mode, and an explicit tool allowlist.
- ✓ Harden a shared repo with permission `deny` rules backed by a `PreToolUse` guard hook, and say why the two layers reinforce each other.
- ✓ Chain a multi-step job into one orchestrator skill so it runs the same way every time, and know when inline skill stacking is enough instead.
- ✓ Choose the right combination for a recurring need using the map at the top of the chapter.
- ✓ Explain which layer gives a workflow its shape and which layer gives it its guarantees.

Chapter Nineteen

The Operator's Practice

The handful of habits that keep you sharp as an operator while the tools keep changing under your feet.

You have reached the end of the manual, and the tools you learned here will keep changing. New models will ship. Commands will get renamed. A flag you memorized this month may move next month. That churn is real, and it is also fine, because the thing that makes you an operator was never the memorized flag. It was the way you work.

This closing chapter steps back from the machinery and names the practice underneath it. The small recurring habits that separate a power user from a casual one. A way to stay current without drowning in changelogs. A concrete checklist for the first hour on a new machine or project. And a short map of where to go deeper once you close the book.

There is almost no new syntax here. Everything below is a distillation of what the previous eighteen chapters taught, arranged so you can carry it in your head and act on it every day.

19.1 The habits that make an operator

Watch someone who is genuinely good with Claude and you notice they do a few unglamorous things by reflex. None of them is clever. All of them compound. Four matter most, and each one is the through-line of a chapter you already read.

They write things down as memory. The first time an operator corrects Claude, they answer the question. The second time, they put the answer in a file. A convention, a build command, a hard rule, a path that is never obvious from the code goes into CLAUDE.md or auto memory (Chapter 9), so it loads on its own tomorrow. The casual user re-explains the same context every morning and calls it the cost of doing business. The operator refuses to pay that tax twice.

They start clean. A fresh session begins with an empty head, and that blank slate is a feature worth protecting. When a conversation has wandered through three unrelated problems, the context is full of noise that quietly degrades every answer that follows. An operator finishes a task, starts a new session for the next one, and lets memory reload the durable context automatically. Clean sessions stay predictable.

They reach for plan mode on risky work. Before Claude touches billing code, a database migration, or anything with a blast radius, the operator drops into plan mode (Chapter 11) and reads the plan first. It costs thirty seconds. It catches the misread instruction, the wrong file, the destructive command before any of it runs. The habit is cheap. The mistake it prevents is not.

They verify what matters. Claude is a capable assistant that is sometimes confidently wrong, so an operator checks the output that carries consequences. They read the diff before they commit. They run the test. They confirm the file actually changed. Verification is scoped to the stakes: a throwaway script gets a glance, a production change gets real scrutiny.

TIP

If you adopt only one habit from this book, make it the second one: **start a new session when the task changes.** It is the single cheapest way to raise the quality of every answer, it costs you nothing once memory is set up, and it is the habit newcomers are slowest to build because a long thread feels productive even as it fills with clutter.

These four habits reinforce each other. Good memory makes clean sessions painless, because a fresh start still knows your project. Plan mode and verification are the same instinct pointed at two moments: one before the

work, one after. Together they are the whole operator's stance in a sentence. Give Claude the context, let it work, and check the parts that count.

19.2 Staying current when the product moves fast

Claude ships quickly, and trying to track every release is a losing game that also misses the point. The reason this manual leaned so hard on mental models is that the models are what last. A specific flag is perishable. The idea behind it usually is not.

KEY TERM

Durable mental model the concept behind a feature, which survives across releases, as opposed to the exact command or number that implements it, which does not. Learn why permissions have a deny-list and you can find the current syntax in a minute. Memorize only the syntax and you are stranded the day it changes.

Hold that distinction and staying current gets much lighter. You are not trying to memorize a moving target. You already own the map. What you top up is the coordinates. When a command feels off or a default seems to have shifted, you know exactly which idea it belongs to, and you go look up the one detail that changed.

Two sources are worth trusting as the current truth. The first is the official documentation, which is always ahead of any book, including this one. When this manual said "check the docs for the exact limit," it meant that sincerely: printed numbers go stale, and the docs are where the live figure lives. The second is the companion update page for this book, which tracks the specifics that drift so the chapters you read stay anchored to reality. Treat the book as your mental model and those two pages as the errata that keep it accurate.

You do not need a daily ritual for this. A light, occasional pass is enough, and it is worth being deliberate about cadence so keeping current never becomes a second job.

QUICK REFERENCE · A LOW-EFFORT KEEP-CURRENT ROUTINE

Cadence	What to check	Why it is worth it
---------	---------------	--------------------

When something breaks	The docs page for that one feature	A renamed flag or changed default is a one-minute fix once you know where to look
Monthly	The release notes and this book's update page	Catches the handful of changes that actually affect your setup
On a new model	The model's own notes on strengths and limits	Capabilities shift with each release; your prompts may want tuning
Rarely, on purpose	One deep feature you have never used	Widens what you reach for, instead of only maintaining what you know

I TIP

Claude can help you stay current about Claude. When a command behaves in a way you do not expect, ask it directly what changed, or point it at the relevant docs page and have it summarize the current behavior. The model reads faster than you do, and it is genuinely good at telling you which of your habits a new release affects.

! GOTCHA

Chasing every changelog is its own trap. If you rebuild your setup around each new feature the week it lands, you spend more time tuning tools than using them. Let a change prove it earns a place in your workflow before you adopt it. A stable setup you understand beats a bleeding-edge one you have to relearn every Monday.

19.3 The day-one setup checklist

Here is the practical payoff of the whole book on one page. When you sit down at a new machine or open a new project, this is the order that gets you from zero to a real operator setup fastest. Each step points back to the chapter that covers it in full, so you can go as deep as you like on any line.

▶ DO THIS · SET UP A NEW MACHINE OR PROJECT

- 1 **Install the desktop app and confirm your plan.** The desktop app gives Claude local file access and the global shortcut a browser tab cannot have, and the operator surfaces need a paid plan (Chapter 1). Do this first; everything else assumes it.
- 2 **Write your user-scope preferences once.** Put the rules that follow you across every project, your tone, your standing constraints, your defaults, into `~/.claude/CLAUDE.md` (Chapter 9). You author this a single time and reap it everywhere.
- 3 **Run `/init` in the project, then trim.** Let Claude draft a starting CLAUDE.md from the codebase, then cut it to the pitfalls, conventions, and reasons it could not have guessed (Chapter 9). A short, specific memory file beats a long vague one.
- 4 **Set a safe permission baseline.** Configure `settings.json` so routine actions run freely and the genuinely dangerous ones are denied or need approval (Chapter 10). Ten minutes here buys you a session you can trust to move quickly.
- 5 **Add a hard rule for the one thing that must never happen.** Memory shapes behavior; a `PreToolUse` hook enforces it (Chapter 14). If there is a single irreversible action you cannot risk, block it with a hook so no polite request can be ignored.
- 6 **Capture your repeated work as a command or skill.** The workflow you run three times a week belongs in a slash command or a master command (Chapter 12), and a multi-step procedure belongs in a skill that loads only when needed (Chapters 4 and 13).
- 7 **Confirm it all loaded with `/context`.** Check that your memory files, rules, and settings actually show up (Chapter 9). If a file is not in the list, Claude never read it, and nothing above it is doing anything yet.

I TIP

Do not do all seven on day one for every project. The first two are worth doing once on any new machine. Steps three through seven pay off most on a project you will return to for weeks. For a throwaway experiment, an `/init` and a sane permission default are plenty. Match the setup effort to how long the project will live.

19.4 Where to go next

You now have the full vocabulary. The natural next move is to deepen the few pieces that map to your actual work, and you can do that in any order. Three directions repay the time most.

Go deeper on automation. If your days are full of repeated procedures, live in skills and commands until building one is second nature (Chapters 4, 12, and 13). This is where the largest hours-per-week savings hide, because every workflow you encode is a task you never steer by hand again.

Go deeper on delegation. If your work is large and parallel, spend real time with subagents (Chapter 15) and the workflow patterns earlier in this part. Learning to hand off a self-contained job and trust the result is the skill that lets one operator do the work of several.

Go deeper on safety. If you run Claude on anything that matters, master permissions and hooks (Chapters 10 and 14) until a safe-by-default configuration is something you stand up without thinking. Confidence to move fast comes from knowing the dangerous paths are already closed.

! GOTCHA

The most common failure from here is not using any of it. You will finish this book, feel equipped, and quietly drift back to plain chat for everything because it is familiar. Pick one habit from 19.1 and one setup step from 19.3 and use them on your very next real task, today. Capability you never reach for is the same as capability you never had.

A closing word, operator to operator. The tools in this manual are genuinely powerful, and they will keep getting more so, which means the ceiling on what one person can do keeps rising. What decides how much of that ceiling you actually reach is not which release you are on. It is whether you build the small habits that let you trust the work: writing context down, starting clean, planning the risky moves, and checking what counts. Do those, keep your mental models sharp, and look up the perishable specifics when you need them. The product will change many times over. You will already know how to operate whatever it becomes. Now go build something.

CHAPTER 19 · CHECKLIST

You can now

- ✓ Name the four operator habits: write context down, start clean, plan risky work, verify what matters.
- ✓ Separate durable mental models from perishable specifics, and know which one to memorize.
- ✓ Keep current with a light routine anchored to the official docs and this book's update page.
- ✓ Stand up a new machine or project in order, from desktop app and memory to permissions and hooks.
- ✓ Match your setup effort to how long a project will actually live.
- ✓ Choose where to go deeper, whether that is automation, delegation, or safety.
- ✓ Operate Claude as a power tool across chat, Projects, Cowork, and the command line, and keep operating it as the product changes.

REFERENCE

Appendices

The fast-lookup half. Cheat sheets, command and flag references, the hook-event and settings keys, a glossary, and the sources behind every claim in this manual. Keep these open in a second window while you build.

Appendix A

The Day-One Setup Checklist

The highest-value setup moves, in order, for a new machine or a new project. Each points back to the chapter that explains it.

This is the short path from a fresh install to an operator-grade setup. Work down the list once per machine, then once more per serious project. Everything here is taught in full earlier in the book, so treat the chapter references as the place to go when a step needs more than a line.

A.1 On a new machine

► DO THIS · ONE-TIME MACHINE SETUP

- 1 Install the desktop app and Claude Code.** The app for local file access and Quick Entry, the command-line tool for the operator work. See Chapter 1 and Chapter 8.
- 2 Sign in and confirm your plan.** Pro or higher unlocks Cowork and the command-line agent. Chapter 1.
- 3 Write your user-level CLAUDE.md.** Put your personal, cross-project preferences in `~/.claude/CLAUDE.md` so they follow you everywhere. Chapter 9.

- 4 **Set safe default permissions.** A conservative user-level `settings.json` with a sensible deny list keeps you out of trouble before you tune per project. Chapter 10.
- 5 **Connect only the tools you trust.** Add the connectors and MCP servers you actually use, and no more. Chapters 5 and 16.

A.2 On a new project

► DO THIS · PER-PROJECT SETUP

- 1 **Run `/init`.** Generate a starting project CLAUDE.md, then cut it down to pitfalls and non-obvious conventions. Chapter 9.
- 2 **Split stable from volatile.** Keep CLAUDE.md short and import a separate status file for day-to-day state. Chapter 9.
- 3 **Scope project permissions.** Add a project `settings.json` that allows the project's real tools and denies the dangerous ones. Chapter 10.
- 4 **Add a guarding hook if the project has protected files.** A `PreToolUse` gate is the only thing that hard-blocks an action. Chapter 14.
- 5 **Capture your repeatable prompts as commands or skills.** Anything you type twice becomes a custom command or a skill. Chapters 12 and 13.
- 6 **Use plan mode for risky changes.** Force a read-only plan before large or unfamiliar edits. Chapter 11.
- 7 **Confirm what loaded.** Run `/context` to verify your memory and rules are actually in play. Chapter 9.

I TIP

Do not try to build the perfect setup on day one. Ship the basics above, then add a rule, a command, or a hook each time you catch yourself repeating a correction. The setup that grows from real friction beats the one you imagined in advance.

Appendix B

Slash-Command Reference

The built-in commands worth committing to muscle memory, grouped by the job they do.

Type `/` at the start of a message in Claude Code and the menu of every available command appears. This card covers the durable built-ins, the ones that have earned a place in daily use and are unlikely to vanish between releases.

The list here is the important set current as of this edition, not the complete inventory. The live, authoritative list is always one keystroke away: run `/help` in any session to see exactly what your version ships. Custom commands, plugin commands, and MCP prompts appear in the same menu and are covered in their own chapters.

! GOTCHA

The built-in command list moves. Commands are added, renamed, promoted from fixed CLI logic into bundled skills, and occasionally retired, often release to release. Treat this appendix as a durable map of what the commands are for, and treat `/help` in your live session as the authority on what exists and what it is called today.

B.1 Session control

Start, save, resume, and move a conversation between surfaces.

QUICK REFERENCE · SESSION CONTROL

Command	What it does
<code>/clear</code>	Start a new conversation with empty context. Aliases <code>/reset</code> , <code>/new</code> .
<code>/resume</code>	Return to an earlier conversation.
<code>/rewind</code>	Roll code and conversation back to a checkpoint.
<code>/export</code>	Export the conversation as plain text.
<code>/exit</code>	Exit the CLI. Alias <code>/quit</code> .

B.2 Context and memory

Manage what the model is holding in view and what it carries between sessions.

QUICK REFERENCE · CONTEXT AND MEMORY

Command	What it does
<code>/context</code>	Visualize context usage as a colored grid and warn when you approach the window limit.
<code>/compact</code>	Summarize the conversation to free context. Takes optional focus instructions.
<code>/memory</code>	Edit your <code>CLAUDE.md</code> memory files and view or toggle auto-memory.
<code>/init</code>	Initialize a project with a starter <code>CLAUDE.md</code> guide.

B.3 Configuration and permissions

Set how Claude behaves and how freely it may act.

QUICK REFERENCE · CONFIGURATION AND PERMISSIONS

Command	What it does
<code>/model</code>	Switch the active model, saved as the default. No argument opens a picker.
<code>/config</code>	Open the Settings UI or set a key directly, e.g. <code>/config theme=dark</code> . Alias <code>/settings</code> .

SLASH-COMMAND REFERENCE

<code>/permissions</code>	Set approval rules for file access, tool use, and other sensitive operations.
<code>/plan</code>	Switch into plan mode before a large change.
<code>/mcp</code>	Manage MCP server connections and OAuth.
<code>/hooks</code>	Browse configured hook events, matchers, and their sources, read only.
<code>/add-dir</code>	Add a working directory for file access this session.

B.4 Review and shipping

Check the work before it goes out. Several of these run as bundled skills.

QUICK REFERENCE · REVIEW AND SHIPPING

Command	What it does
<code>/diff</code>	Open an interactive diff viewer with per-turn diffs.
<code>/code-review</code>	Review the current diff for bugs and cleanup.
<code>/security-review</code>	Check the diff for vulnerabilities.
<code>/review</code>	Fast, single-pass, read-only review of a GitHub PR.

B.5 Information and diagnostics

See where you stand and check that the setup is healthy.

QUICK REFERENCE · INFORMATION AND DIAGNOSTICS

Command	What it does
<code>/help</code>	Show help and the full list of available commands. The authority on your version.
<code>/status</code>	Show current session status and model.
<code>/usage</code>	Show token usage and cost for the session. Alias <code>/cost</code> .
<code>/doctor</code>	Run a setup checkup covering installation health and configuration cost. Alias <code>/checkup</code> .
<code>/copy</code>	Copy the last response to the clipboard.

Appendix C

Hook-Event Reference

The lifecycle events Claude Code fires, when each one triggers, and which ones can stop the action.

A hook fires on a named lifecycle event, deterministically, whether or not the model wants it to. This card lists the core events current as of this edition. The catalog grows between releases, so the hooks documentation is the live authority for the full set.

C.1 Events

Every event below is a key inside the top-level `hooks` object in settings. The block column tells you whether a hook on that event can deny or halt the action, or only observe and inject context.

QUICK REFERENCE · CORE HOOK EVENTS

Event	When it fires	Can block?
<code>SessionStart</code>	A session begins or resumes.	No, context only
<code>UserPromptSubmit</code>	You submit a prompt, before Claude processes it.	Yes
<code>UserPromptExpansion</code>	A typed command or skill name expands into a prompt, before it reaches Claude.	Yes, blocks the expansion
<code>PreToolUse</code>	Before a tool call executes.	Yes, can deny

HOOK-EVENT REFERENCE

<code>PermissionRequest</code>	When a permission dialog is about to appear.	Yes, can auto-decide
<code>PermissionDenied</code>	A tool call was denied by the auto-mode classifier.	Can return retry
<code>PostToolUse</code>	After a tool call succeeds.	Yes, <code>decision: block</code>
<code>PostToolUseFailure</code>	After a tool call fails.	Yes, like <code>PostToolUse</code>
<code>PostToolBatch</code>	After a batch of parallel calls resolves, before the next model call.	Yes, ends the turn
<code>Notification</code>	Claude Code sends a notification (permission, idle, auth, agent events).	No
<code>SubagentStart</code>	A subagent is spawned.	No
<code>SubagentStop</code>	A subagent finishes.	Yes, block to keep going
<code>Stop</code>	Claude finishes responding, not on a user interrupt.	Yes, block to keep going
<code>InstructionsLoaded</code>	A CLAUDE.md or rules file loads.	No
<code>ConfigChange</code>	A settings or skills config file changes mid-session.	Yes, block the change
<code>PreCompact</code>	Before context compaction.	No
<code>PostCompact</code>	After compaction completes.	No
<code>Elicitation</code>	An MCP server requests user input mid-tool-call.	Yes, can auto-respond
<code>SessionEnd</code>	The session terminates.	No

All matching hooks for one event run in parallel, and identical commands are deduplicated. When several `PreToolUse` hooks fire at once, the most restrictive permission decision wins, in the order `deny`, `defer`, `ask`, `allow`. Context text from every hook is kept.

GOTCHA

This table is the core set, not the complete list. The event catalog expands release to release, with agent-team, worktree, and MCP-elicitation events arriving as those features ship. Treat the hooks documentation as the authority and re-check it before you commit a hook to a real workflow.

c.2 Exit codes

A command hook signals its verdict through its exit code. Exit 1 and exit 2 are not the same, and confusing them is the most common hook-authoring mistake.

QUICK REFERENCE · COMMAND-HOOK EXIT CODES

Exit code	Effect
0	Success, action proceeds. For <code>PreToolUse</code> this does not itself approve; the normal permission flow still applies. Stdout is added to context only for <code>UserPromptSubmit</code> , <code>UserPromptExpansion</code> , and <code>SessionStart</code> .
2	Blocking. Stderr is fed to Claude as feedback. Events that cannot block (such as <code>SessionStart</code> and <code>Notification</code>) instead show stderr to you and continue.
Other, such as 1	Non-blocking error. The transcript shows a hook error plus the first stderr line, and execution continues.

Do not mix an exit-2 block with JSON on stdout. Claude Code ignores your JSON when you exit 2. For structured control, exit 0 and return JSON.

c.3 Decision and permission fields

On an exit-0 hook, the JSON you return carries the decision. The relevant field depends on the event.

QUICK REFERENCE · DECISION FIELDS BY EVENT

Event	Decision field and values
-------	---------------------------

HOOK-EVENT REFERENCE

PreToolUse	hookSpecificOutput.permissionDecision : allow , deny , ask , or defer . May also return updatedInput and additionalContext .
PermissionRequest	hookSpecificOutput.decision.behavior : allow or deny, with an optional permissionRule and updatedPermissions to switch mode.
PermissionDenied	hookSpecificOutput.retry: true to retry the denied call.
UserPromptSubmit	hookSpecificOutput.additionalContext . It must be nested; a top-level field is silently ignored.
PostToolUse	hookSpecificOutput.updatedToolOutput , or top-level decision: block .
Block-capable events	Top-level decision: block with a reason : used by UserPromptSubmit , UserPromptExpansion , PostToolUse , PostToolUseFailure , PostToolBatch , Stop , SubagentStop , ConfigChange , and PreCompact .

Universal fields apply on any exit-0 hook. `continue: false` stops Claude after the hook regardless of event-specific decisions, and `suppressOutput: true` hides stdout from the transcript. Output strings such as `additionalContext` are capped at 10,000 characters, with the excess written to a file and referenced by a preview and path.

I TIP

Hooks can tighten but never loosen. A `PreToolUse` hook fires before any permission-mode check, so its `deny` holds even under `bypassPermissions` . A hook `allow` does not override a deny rule in settings.

Appendix D

settings.json Key Reference

The configuration keys worth knowing, where the files live, and which scope wins when they disagree.

These are the durable, high-value keys, current as of this edition. Claude Code adds and renames settings often, so treat this as the map, not the territory. The official settings schema is the authority for the exact current list.

Everything below groups by what you are actually trying to do: find the right file, control what Claude may do, tune its behavior, and wire in your environment. Keys marked managed only take effect in an administrator-deployed policy file and are ignored anywhere else.

D.1 File locations and precedence

Claude Code reads settings from several scopes and layers them. A setting at a higher scope overrides the same setting at a lower one. Permission rules are the exception: allow, deny, and ask arrays merge across all scopes rather than replace each other.

QUICK REFERENCE · SETTINGS FILE LOCATIONS

Scope	Location	Applies to
Managed	System policy path (see the docs for your OS)	Every user on the machine. IT-deployed, cannot be overridden.
User	<code>~/.claude/settings.json</code>	You, across all projects.

SETTINGS.JSON KEY REFERENCE

Project	<code>.claude/settings.json</code>	Everyone on the repo. Committed to git.
Local	<code>.claude/settings.local.json</code>	You, this repo only. Gitignored.

QUICK REFERENCE · PRECEDENCE, HIGHEST WINS

Order	Source	Note
1	Managed policy	Wins over everything, including CLI arguments.
2	Command line arguments	Session-only overrides.
3	Local settings	<code>.claude/settings.local.json</code>
4	Project settings	<code>.claude/settings.json</code>
5	User settings	<code>~/.claude/settings.json</code> , lowest.

I TIP

Add `"$schema": "https://json.schemastore.org/claude-code-settings.json"` to the top of any settings file for editor autocomplete and validation. Run `/status` to see every settings source that loaded for the current session.

D.2 Permissions keys

These govern what Claude runs without asking. The rule arrays take pattern strings like `Bash(npm run *)` or `Read(./env)`. Evaluation runs deny, then ask, then allow, and the first match wins.

QUICK REFERENCE · PERMISSIONS.*

Key	What it controls
<code>permissions.allow</code>	Patterns Claude may run without a prompt, e.g. <code>["Bash(npm run test *)"]</code> .
<code>permissions.deny</code>	Patterns always blocked. A bare tool name like <code>Bash</code> hides the tool entirely.
<code>permissions.ask</code>	Patterns that always prompt, even if an allow rule would match.

<code>permissions.defaultMode</code>	Starting permission mode: <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>auto</code> , <code>dontAsk</code> , or <code>bypassPermissions</code> .
<code>permissions.additionalDirectories</code>	Extra directories Claude may read and edit beyond the working directory.
<code>permissions.disableBypassPermissionsMode</code>	Set to <code>"disable"</code> to block bypass mode from ever engaging.
<code>permissions.disableAutoMode</code>	Set to <code>"disable"</code> to block auto mode.
<code>allowManagedPermissionRulesOnly</code>	Managed only. Stops user and project files from defining any permission rules.

```
.claude/settings.json

{
  "permissions": {
    "deny": ["Read(/.env)", "Read(/secrets/**)"],
    "allow": ["Bash(npm run lint)"],
    "defaultMode": "acceptEdits"
  }
}
```

D.3 Model and session behavior

These shape how Claude reasons and how the session behaves. Note the timing: `model` and `outputStyle` are read once at startup, so a change needs `/clear` or a restart. Most other keys reload live.

QUICK REFERENCE · MODEL AND DISPLAY KEYS

Key	What it controls
<code>model</code>	The primary model. Read once at startup.
<code>fallbackModel</code>	Fallback chain when the primary is overloaded.
<code>availableModels</code>	Restrict which models are selectable, e.g. <code>["sonnet", "haiku"]</code> .
<code>effortLevel</code>	Persist a reasoning effort level such as <code>low</code> or <code>high</code> .

SETTINGS.JSON KEY REFERENCE

<code>alwaysThinkingEnabled</code>	Enable extended thinking by default.
<code>outputStyle</code>	Output format. Read once at startup.
<code>editorMode</code>	<code>"normal"</code> or <code>"vim"</code> input key bindings.
<code>cleanupPeriodDays</code>	Days before old session files are auto-deleted. Default is 30.
<code>autoUpdatesChannel</code>	Release channel: <code>"latest"</code> or <code>"stable"</code> .

D.4 Environment, memory, and integrations

These connect Claude Code to your shell, your credentials, and your other tooling.

QUICK REFERENCE · ENVIRONMENT AND INTEGRATION KEYS

Key	What it controls
<code>env</code>	Environment variables applied to every session and subprocess.
<code>defaultShell</code>	Shell for <code>!</code> -prefixed commands, <code>"bash"</code> or <code>"powershell"</code> .
<code>apiKeyHelper</code>	A command that generates the auth value, refreshed on an interval.
<code>autoMemoryEnabled</code>	Toggle the per-repo auto memory read and write. Default is on.
<code>autoMemoryDirectory</code>	Custom storage path for auto memory.
<code>claudeMdExcludes</code>	Globs of CLAUDE.md files to skip loading.
<code>disableAllHooks</code>	Turn off all hooks and the custom status line.
<code>allowedHttpHookUrls</code>	Allowlist of URL patterns that HTTP hooks may target.
<code>enableAllProjectMcpServers</code>	Auto-approve every server in a project <code>.mcp.json</code> .
<code>attribution</code>	Customize git commit and PR attribution text.
<code>companyAnnouncements</code>	Messages shown at startup, cycled at random.

! GOTCHA

This list is a curated subset, not the full schema. Individual keys are added, renamed, and deprecated across releases, so a key you saw in a blog post may be legacy. When you need the current, complete, exact set, the official settings documentation and the JSON schema are the authority. Check them before you rely on any key not shown here.

I TIP

Managed policy files parse tolerantly: one bad entry is stripped with a warning and the rest still applies. User, project, and local files are strict, so a single JSON error rejects the entire file. Run `/doctor` to see what was stripped or rejected.

Appendix E

CLI Flag Reference

The command-line flags and invocation forms worth keeping within reach when you drive Claude Code from a terminal.

These are the flags that earn their place in daily use, current as of this edition. The `claude` CLI carries more than fit here, and the set shifts between releases. Run `claude --help` for the live list on your machine, and see the CLI reference docs for the authoritative version.

Flags fall into a few natural groups: how you start a session, how you run one-shot and automated jobs, and how you point Claude at the right configuration and directories. The tables below follow that order.

E.1 Starting a session

The base command with no flags opens an interactive session in the current directory. These forms change what that session picks up when it starts.

QUICK REFERENCE · SESSION START

Flag	What it does	Note
<code>claude "task"</code>	Starts an interactive session seeded with an opening prompt.	The session stays open after the first reply.
<code>-c</code> / <code>--continue</code>	Loads the most recent conversation in the current directory.	Fastest way back into what you were just doing.

<code>-r / --resume</code>	Resumes a session by id or name, or opens a picker if you give none.	Use when the last session is not the one you want.
<code>--model</code>	Sets the model by alias (<code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code>) or full name.	Switch mid-session with <code>/model</code> .
<code>--permission-mode</code>	Sets how freely Claude may act: <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , and others.	<code>Shift+Tab</code> cycles modes inside a session.
<code>-w / --worktree</code>	Starts in an isolated git worktree so parallel jobs do not collide.	See the Glossary entry for worktree.
<code>--ide</code>	Auto-connects to an IDE when exactly one valid one is found.	For CLI runs launched near an editor.
<code>-v / --version</code>	Prints the installed version and exits.	Pairs with <code>claude doctor</code> for diagnostics.

E.2 Interactive versus one-shot

The single most useful distinction in the CLI is whether you want a conversation or a result. Interactive mode is a REPL you talk to. One-shot mode (`-p`) runs a query, prints the answer, and exits, which is what you want inside scripts and supervisors.

QUICK REFERENCE · INTERACTIVE VS ONE-SHOT

Form	Behavior
<code>claude</code>	Starts the interactive REPL. You stay in the session and iterate.
<code>claude "task"</code>	Interactive, with the first prompt already typed for you.
<code>claude -p "query"</code>	Runs headless via the SDK, prints the response, exits. The scriptable form.
<code>cat file claude -p "query"</code>	Pipes file content in as part of the one-shot query.

E.3 One-shot and automation

These flags shape a headless run: how output is formatted, how long it may run, and how much it may spend. The pairing of `-p` with `--output-format stream-json` is the standard pattern for scripts and supervisors that parse Claude's output as it streams.

QUICK REFERENCE · PRINT-MODE AND AUTOMATION

Flag	What it does	Note
<code>-p</code> / <code>--print</code>	Runs the query and exits without an interactive session.	The anchor of every automated call.
<code>--output-format</code>	Sets output shape: <code>text</code> , <code>json</code> , or <code>stream-json</code> .	<code>stream-json</code> for supervisors that read as it runs.
<code>--input-format</code>	Sets input shape: <code>text</code> or <code>stream-json</code> .	Print mode only.
<code>--max-turns</code>	Caps the number of agentic turns in a print run.	A guardrail against runaway loops.
<code>--max-budget-usd</code>	Stops the run after spending this much on API calls.	A hard ceiling on cost.
<code>--fallback-model</code>	Comma-separated chain to fall back to when the primary is overloaded.	Keeps unattended jobs alive.
<code>--allowedTools</code>	Tools that run without prompting, e.g. <code>"Bash(git log *)"</code> .	Essential for unattended runs.
<code>--disallowedTools</code>	Tools that are denied outright.	The deny-rule counterpart.
<code>--bg</code> / <code>--background</code>	Starts as a background agent and returns immediately.	Reattach later with <code>claude attach <id></code> .

E.4 Configuration and directories

These flags decide which settings and MCP servers a run loads, and which directories Claude may touch. They matter most when you want a run to ignore ambient config and use exactly what you hand it, which is common in CI.

QUICK REFERENCE · CONFIG AND DIRECTORIES

Flag	What it does	Note
<code>--settings</code>	Path to a settings JSON file, or an inline JSON string.	Override config per run.
<code>--setting-sources</code>	Comma list of which sources to load: <code>user,project,local</code> .	Trim ambient config in CI.
<code>--mcp-config</code>	Loads MCP servers from JSON files or a string.	Wire up servers for one run.
<code>--strict-mcp-config</code>	Uses only the servers from <code>--mcp-config</code> and ignores all others.	For reproducible, sealed runs.
<code>--add-dir</code>	Grants access to additional working directories.	Also available in-session as <code>/add-dir</code> .
<code>--session-id</code>	Uses a specific UUID as the session id.	For addressable, resumable runs.
<code>--append-system-prompt</code>	Appends text to the default system prompt.	A file variant takes a path.
<code>--system-prompt</code>	Replaces the entire system prompt.	Use sparingly; you lose the defaults.
<code>--effort</code>	Sets reasoning effort: <code>low</code> through <code>max</code> .	Lower effort saves cost on simple jobs.
<code>--verbose</code>	Prints full turn-by-turn logging.	The first flag to add when debugging.
<code>--safe-mode</code>	Starts with all customizations disabled.	Isolate whether your config is the problem.

E.5 Permission and bypass flags

Permission flags are worth their own table because getting them wrong has real consequences. Bypassing permissions removes the approval prompts that stand between Claude and your filesystem, so reserve it for sandboxes you trust.

QUICK REFERENCE · PERMISSION CONTROL

Flag	What it does
<code>--permission-mode</code>	Sets the mode directly, from <code>default</code> (ask on everything) to <code>acceptEdits</code> , <code>plan</code> , and the bypass modes.
<code>--dangerously-skip-permissions</code>	Equivalent to <code>--permission-mode bypassPermissions</code> . Skips all prompts.
<code>--allow-dangerously-skip-permissions</code>	Adds the bypass mode to the <code>Shift+Tab</code> cycle without starting in it.

 GOTCHA

This reference is a working subset, not the whole surface. The CLI adds and renames flags between releases, and some behaviors are gated to specific model or plan versions. Treat `claude --help` and the official CLI reference as the authority. If a flag here does not match what your terminal reports, believe your terminal.

Appendix F

Glossary

Every term this manual leans on, defined in one place and in plain language.

Definitions are written the way the book uses the words, not the way a specification would. Where a term has a chapter that covers it in full, that is the place to go deeper.

QUICK REFERENCE · TERMS A TO Z

Term	Meaning
Agent	A model that can request a tool, receive the result, and decide what to do next, looping until a job is done. Cowork, subagents, and Claude Code are all agents.
API	Application programming interface. The defined doorway a program uses to send Claude a request and get a response.
Artifact	A standalone deliverable Claude builds in a side panel, such as a document, chart, or small app, that you can edit, publish, and share.
Auto memory	Notes Claude writes about your project from your corrections and preferences, stored per repository and loaded each session.
Checkpoint	A snapshot of session state that lets you rewind Claude Code to an earlier point.
CLAUDE.md	A Markdown file of instructions you write, loaded in full at the start of every Claude Code session as standing context.
Connector	An integration that lets Claude read and act in one of your other tools or services.

GLOSSARY

Context window	The fixed budget of tokens a model can hold in view at once, shared by instructions, history, files, and the reply in progress.
Cowork	Claude's autonomous, long-running task mode. You describe a multi-step job and it runs in the background, resumable across your devices.
Extended thinking	An explicit reasoning pass a model can take before answering, useful for hard problems and wasteful on simple ones.
Headless mode	Running Claude Code as a one-shot command (the <code>-p</code> form) for scripting and unattended automation instead of an interactive session.
Hook	A command that fires on a Claude Code lifecycle event and can deterministically allow, block, or act around a tool call. The way to hard-enforce a rule.
Managed policy	Organization-wide settings or memory set by an administrator that apply to everyone on a machine and cannot be overridden locally.
Matcher	The pattern on a hook that scopes it to specific tools, so the hook only fires where you intend.
MCP	Model Context Protocol. An open standard that lets any tool or data source plug into Claude through a common interface.
Model tier	One of the family of Claude models trading capability against cost: a fast inexpensive tier, a balanced tier, and a most-capable tier.
Permission mode	A session-wide setting that governs how freely Claude Code may act, from asking on everything to accepting edits automatically.
Plan mode	A read-only phase where Claude researches and proposes a plan for your approval before it touches anything.
Project	A persistent app workspace bundling custom instructions, uploaded knowledge, and the chats held inside it.
Rules folder	The <code>.claude/rules/</code> directory of topic-scoped instruction files, optionally bound to file paths so they load only when relevant.
settings.json	The Claude Code configuration file, present at user, project, local, and managed scopes with a defined precedence order.
Skill	A packaged capability Claude loads only when a task calls for it, keeping the assistant focused and context lean.
Slash command	A <code>/</code> -prefixed command that controls a session or runs a saved workflow. You can author your own.

Statelessness	The fact that a model retains nothing between requests. Continuity comes from resending the conversation as context each time.
Subagent	A separate agent with its own clean context that does a scoped job and reports back, saving the main context and sharpening focus.
System prompt	The standing brief that frames an exchange: who Claude should be, the rules, the format. It sits above the user and assistant messages.
Token	The basic unit of text a model reads and writes, roughly a short word or a fragment of one. Cost and limits are counted in tokens.
Tool use	The pattern where the model requests that a named tool be run with specific inputs, and the surrounding program runs it and returns the result. Also called function calling.
Worktree	A second checkout of the same git repository on its own branch, so you can run separate jobs in parallel without collision.

Appendix G

Sources & Further Reading

The official pages behind every claim in this manual, grouped by area. Where this book and the live docs ever disagree, the docs win.

This manual is dated on purpose. The product moves, and these landing pages are the always-current authority. Bookmark the two documentation hubs, code.claude.com/docs for Claude Code and platform.claude.com/docs for the API, and you will rarely be more than a click from the current specifics.

Claude Code documentation

- › <https://code.claude.com/docs/en/agent-sdk/slash-commands>
- › <https://code.claude.com/docs/en/authentication>
- › <https://code.claude.com/docs/en/checkpointing>
- › <https://code.claude.com/docs/en/cli-reference>
- › <https://code.claude.com/docs/en/commands>
- › <https://code.claude.com/docs/en/costs>
- › <https://code.claude.com/docs/en/desktop>
- › <https://code.claude.com/docs/en/headless>
- › <https://code.claude.com/docs/en/hooks>
- › <https://code.claude.com/docs/en/hooks-guide>
- › <https://code.claude.com/docs/en/interactive-mode>
- › <https://code.claude.com/docs/en/jetbrains>
- › <https://code.claude.com/docs/en/mcp>
- › <https://code.claude.com/docs/en/memory>

- › <https://code.claude.com/docs/en/overview>
- › <https://code.claude.com/docs/en/permission-modes>
- › <https://code.claude.com/docs/en/permissions>
- › <https://code.claude.com/docs/en/quickstart>
- › <https://code.claude.com/docs/en/sessions>
- › <https://code.claude.com/docs/en/settings>
- › <https://code.claude.com/docs/en/setup>
- › <https://code.claude.com/docs/en/skills>
- › <https://code.claude.com/docs/en/sub-agents>
- › <https://code.claude.com/docs/en/tools-reference>
- › <https://code.claude.com/docs/en/vs-code>
- › <https://code.claude.com/docs/en/worktrees>
- › <https://code.claude.com/docs/llms.txt>

Claude Developer Platform (API, prompting, thinking, vision)

- › <https://platform.claude.com/docs/en/about-claude/models/migration-guide>
- › <https://platform.claude.com/docs/en/agent-sdk/skills>
- › <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- › <https://platform.claude.com/docs/en/api/overview>
- › <https://platform.claude.com/docs/en/build-with-claude/effort>
- › <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- › <https://platform.claude.com/docs/en/build-with-claude/pdf-support>
- › <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
- › <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
- › <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-opus-5>
- › <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-tools>
- › <https://platform.claude.com/docs/en/build-with-claude/thinking>
- › <https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost>
- › <https://platform.claude.com/docs/en/build-with-claude/vision>
- › <https://platform.claude.com/docs/en/intro>

Claude apps, pricing, connectors, blog

- > <https://claude.com/blog/best-practices-for-prompt-engineering>
- > <https://claude.com/blog/memory>
- > <https://claude.com/blog/prompt-improver>
- > <https://claude.com/docs/connectors/custom/desktop-extensions>
- > <https://claude.com/pricing>
- > <https://claude.com/resources/tutorials/use-artifacts-to-visualize-and-create-ai-apps-without-ever-writing-a-line-of-code>
- > <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Model Context Protocol (MCP)

- > <https://modelcontextprotocol.io/docs/develop/build-server>
- > <https://modelcontextprotocol.io/docs/getting-started/intro>
- > <https://modelcontextprotocol.io/docs/learn/architecture>

GitHub, tooling, and community

- > <https://agentskills.io>
- > <https://git-scm.com/docs/git-worktree>
- > <https://github.com/VoltAgent/awesome-claude-code-subagents>
- > <https://github.com/anthropics/prompt-eng-interactive-tutorial>
- > <https://github.com/anthropics/skills>
- > <https://github.com/modelcontextprotocol/inspector>
- > <https://github.com/modelcontextprotocol/mcpb>

Other

- > <https://mcp.asana.com/sse>
- > <https://mcp.hubspot.com/anthropic>
- > <https://mcp.notion.com/mcp>
- > <https://mcp.slack.com/mcp>

- > <https://mcp.stripe.com>
- > <https://support.claude.com/en/articles/10065433-install-claude-desktop>
- > <https://support.claude.com/en/articles/10166901-use-google-workspace-connectors>
- > <https://support.claude.com/en/articles/10181068-configuring-and-using-styles>
- > <https://support.claude.com/en/articles/11049762-choosing-a-claude-ai-plan>
- > <https://support.claude.com/en/articles/11176164-pre-built-integrations-using-remote-mcp>
- > <https://support.claude.com/en/articles/11647753-how-do-usage-and-length-limits-work>
- > <https://support.claude.com/en/articles/11817273-use-claude-s-chat-search-and-memory-to-build-on-previous-context>
- > <https://support.claude.com/en/articles/12012173-getting-started-with-claude-for-chrome>
- > <https://support.claude.com/en/articles/15520349-use-claude-cowork-on-web-desktop-and-mobile>
- > <https://support.claude.com/en/articles/8241126-upload-files-to-claude>
- > <https://support.claude.com/en/articles/8325606-what-is-claude-pro>
- > <https://support.claude.com/en/articles/8664678-change-the-model-effort-and-thinking-settings>
- > <https://support.claude.com/en/articles/9266767-what-is-the-claude-team-plan>
- > <https://support.claude.com/en/articles/9517075-what-are-projects>
- > <https://support.claude.com/en/articles/9519177-how-can-i-create-and-manage-projects>
- > <https://support.claude.com/en/articles/9797531-what-is-the-enterprise-plan>
- > <https://support.claude.com/en/articles/9797557-usage-limit-best-practices>
- > <https://support.claude.com/en/collections/16163169-claude-desktop>
- > <https://www.anthropic.com/engineering/desktop-extensions>
- > <https://www.anthropic.com/news/claude-code-on-the-web>
- > <https://www.claude.com/blog/improving-frontend-design-through-skills>
- > <https://your-host/mcp>